

2025 – 2026

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY: INFORMATION TECHNOLOGY

**Open Web Catcher:
Design of a Multi-Agent AI Pipeline
for Evidence Collection from
Illegal Streaming Sites**

By: Ahmed ARFAOUI

Academic supervisor: Naadine Maazoun

Corporate Internship Supervisor: Wael Hkiri

Soft Stars
Host organization

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I validate the submission of the student's report:

- **Nom & Prénom / Name & Surname:** Ahmed ARFAOUI

Encadrant Entreprise / Business site Supervisor

- **Nom & Prénom / Name & Surname:** Wael Hkiri

Cachet & Signature / Stamp & Signature

Encadrant Académique / Academic Supervisor

- **Nom & Prénom / Name & Surname:** Naadine Maazoun

Signature / Signature

Ce formulaire doit être rempli, signé et **scanné** / This form must be completed, signed and **scanned**.

Ce formulaire doit être introduit après la page de garde / This form must be inserted after the cover page.

Abstract

This report presents the design, development, and evaluation of Open Web Catcher (OWC), an agentic evidence-collection platform for illegal streaming websites. The work was carried out during a PFE internship at Soft Stars in the beIN SPORTS rights-protection context. The objective was to support evidence review by visiting target websites, classifying their pages, extracting hosting and stream evidence, detecting channel signals when visible, and preserving auditable outputs such as screenshots, traces, structured results, and provider metadata.

The internship produced two related artifacts. The first was an exploratory n8n workflow used to validate browser-agent behavior on real dynamic targets and to provide a practical backup path for dynamic evidence cases. The second was the complete OWC engineering platform composed of a FastAPI backend, Next.js operator console, PostgreSQL persistence, MCP browser tooling, Puppeteer/Chrome runtime, specialist agents, model telemetry, and review outputs. The internal name Open Web Catcher was used because the platform focuses on catching evidence that is visible only through open-web browser interaction rather than through a static HTML response.

The proposed architecture organizes evidence collection around specialized reasoning agents, browser automation tools, profile-scoped sessions, fingerprint and proxy controls, structured outputs, trace storage, screenshot capture, and evaluation metrics. The database described in this report is limited to observability, evaluation, traces, metrics, screenshots, errors, tool calls, and structured outputs; it is not the enterprise production database of the client organization.

Keywords: Open Web Catcher; multi-agent system; browser automation; illegal streaming; evidence collection; stream extraction; channel detection; observability.

Résumé

Ce rapport présente la conception, le développement et l'évaluation d'Open Web Catcher (OWC), une plateforme agentique de collecte de preuves pour les sites de streaming illégal. Le travail a été réalisé dans le cadre d'un projet de fin d'études chez Soft Stars, dans un contexte de protection des droits lié à beIN SPORTS. L'objectif était d'améliorer l'observation des pages suspectes, la classification des pages, l'extraction des liens d'hébergement et des preuves de flux, ainsi que la conservation de traces exploitables pour une revue humaine.

Le projet s'est appuyé sur deux réalisations complémentaires. La première est un workflow n8n utilisé pour valider rapidement l'idée d'un agent capable de naviguer sur des pages dynamiques et de servir de solution de secours pour les cas de preuve dynamique. La seconde est OWC, une plateforme composée d'un backend FastAPI, d'une console Next.js, d'une base PostgreSQL, d'outils MCP basés sur Puppeteer et Chrome, d'agents spécialisés, de métriques d'utilisation des modèles et d'artefacts de revue. L'architecture met l'accent sur la traçabilité, les captures d'écran, les traces d'exécution, la détection de canaux visibles, l'enrichissement fournisseur et l'observabilité.

Mots clés: Open Web Catcher; système multi-agent; automatisation navigateur; streaming illégal; collecte de preuves; extraction de flux; détection de canaux; observabilité.

Acknowledgements

I would like to express my sincere gratitude to my academic supervisor, Naadine Maazoun, for her guidance, feedback, and support throughout this PFE internship. Her remarks helped me keep the report focused on the engineering contribution, the methodology, and the clarity expected from an academic submission.

I would also like to thank my corporate supervisor, Wael Hkiri, for his technical direction, availability, and trust during the project. The discussions around real operational constraints, static-collection limitations, browser tooling, deployment choices, and evidence review shaped the work described in this report.

I am grateful to the Soft Stars team for the professional environment, the technical exchanges, and the opportunity to work on a concrete rights-protection problem connected to the beIN SPORTS context. Their feedback helped turn early experiments into a clearer platform design.

Finally, I thank my family and friends for their encouragement and patience during the internship and the preparation of this report.

Contents

Abstract	1
Résumé	2
Acknowledgements	3
List of Acronyms	15
Terminology	17
1 Introduction	21
1.1 Host Organization and Internship Context	21
1.2 Existing Solutions and Project Positioning	21
1.3 Problem Statement	22
1.4 Internship Mission and Scope	23
1.4.1 Mission Deliverables	23
1.4.2 Scope Boundaries	23
1.5 Main Contributions	24
1.6 Report Structure	24
2 Methodology, Requirements Analysis, and Design Constraints	26
2.1 DSRM as the Methodological Backbone	26
2.1.1 Why DSRM Fits the Internship	26
2.1.2 DSRM Phases Used in the Report	27
2.1.2.1 Problem identification and motivation	27
2.1.2.2 Objectives of the solution	27
2.1.2.3 Design and development	28
2.1.2.4 Demonstration	28
2.1.2.5 Evaluation	28
2.1.2.6 Communication	28
2.2 DSRM Mapping to the Internship Work	28
2.3 Project Evolution Timeline	29
2.4 Functional Requirements	30
2.5 Non-Functional Requirements	30

3	Technical Background	32
3.1	Streaming Website Structures	32
3.1.1	The normal page flow	32
3.1.2	Landing pages	33
3.1.3	Hosting pages	34
3.1.4	Embedded pages	34
3.2	DOM Structures and Web Page Representation	35
3.2.1	A simplified DOM tree	35
3.2.2	DOM signals on streaming pages	36
3.3	Streaming Protocols and Embedded Players	37
3.3.1	Common embedding patterns	37
3.3.2	HLS and MPEG-DASH	37
3.3.3	Tokens, cookies, referers, and provider checks	38
3.4	Browser Automation, Scraping, and Runtime Obstacles	38
3.4.1	Static collection versus rendered browser state	38
3.4.1.1	Rendered state as evidence	39
3.4.2	Puppeteer as a browser control layer	39
3.4.3	Ads, cookies, popups, and other annoyances	39
3.4.4	Why screenshots and network logs matter	39
3.5	Agentic Evidence Collection Concepts	41
3.5.1	Agents and ReAct-style loops	41
3.5.2	Prompt engineering, tokens, and token usage	41
3.5.3	Workflow automation tools	42
3.5.4	LangChain, tools, and MCP servers	43
3.6	Chapter Summary	44
4	System Architecture	45
4.1	System Purpose and Architecture Reading Path	45
4.2	Global Runtime Architecture	47
4.3	End-to-End Agent Pipeline	48
4.4	Agent Communication and Orchestrator Handoffs	51
4.5	Agent-by-Agent Design	53
4.5.1	Orchestrator Agent	53
4.5.2	Classification Agent	55
4.5.3	Landing-Page Agent	56
4.5.4	Hosting-Page Agent	57
4.5.5	Embedded-Page Agent	58
4.5.6	Channel Detection Across Agents	59

4.5.7	Provider Analysis and Email Generation	59
4.6	Browser Tooling and Runtime Controls	60
4.6.1	Tool Families	61
4.6.1.1	Inspection and context retrieval	61
4.6.1.2	Navigation and interaction	62
4.6.1.3	Evidence and media extraction	62
4.6.1.4	Memory and site hints	63
4.6.2	Fingerprint Handling and Proxy Rotation	63
4.7	Data, Trace, and Reviewability View	64
4.8	Chapter Summary	67
5	Implementation	68
5.1	Part 1: n8n Workflow Prototype	68
5.1.1	Purpose of the n8n phase	68
5.1.2	n8n workflow screenshots	69
5.1.3	Limitations of the n8n implementation	72
5.2	Part 2: Open Web Catcher Platform	73
5.2.1	FastAPI Backend	74
5.2.2	Next.js Operator Console	74
5.2.3	MCP Browser Containers and Puppeteer Tools	78
5.2.3.1	Tool registry and profile filtering	78
5.2.3.2	Rendered DOM context retrieval	79
5.2.3.3	Puppeteer actions and evidence feedback	80
5.2.4	Agents, Models, Memory, and Prompts	80
5.2.4.1	Design constraints implemented in the agent runtime	81
5.2.4.2	Prompt evolution in the implementation	81
5.2.4.3	Agent design principles	82
5.2.4.4	Model behavior, token usage, and rate limits	82
5.2.4.5	Prompt engineering and stopping behavior	82
5.2.5	PostgreSQL Database and Observability	83
5.3	Part 3: Deployment	84
5.4	Implementation Summary	86
6	Evaluation and Testing	87
6.1	Evaluation Objective and Business Context	87
6.1.1	Business Issue	87
6.1.2	Evaluation Question	87
6.1.3	Reviewable Evidence Bundle	88

6.1.4	Evidence-First Evaluation	88
6.2	Evaluation Protocol and KPI Definitions	88
6.2.1	Batch Test Design	88
6.2.1.1	Input Set and Repeated Runs	88
6.2.1.2	Manual Review After Automated Status	89
6.2.1.3	Website-Level Acceptance Rule	89
6.2.2	Failure Taxonomy	89
6.2.2.1	Why Failure Categories Matter	90
6.2.2.2	External Blockers Are Not Automatically Agent Faults	90
6.2.3	KPI Definitions	90
6.2.3.1	Rates Used in This Chapter	90
6.2.3.2	Current Batch Snapshot	91
6.3	Agent-by-Agent Behavior Evaluation	92
6.3.1	Agent Test Matrix	92
6.3.1.1	Why Agent-Level Evaluation Is Required	92
6.3.2	Classification and Landing Tests	92
6.3.2.1	Classification Tests	92
6.3.2.2	Landing Tests	93
6.3.3	Hosting Tests	93
6.3.3.1	Player Activation	93
6.3.3.2	Server Detection and Switching	93
6.3.3.3	Stream Capture Order	94
6.3.4	Embedded Tests	94
6.3.4.1	Iframe and Player Inspection	94
6.3.4.2	Embedded-Page Inaccessible Cases	94
6.3.5	Provider and Email Tests	94
6.3.5.1	Provider Intelligence Criteria	94
6.3.5.2	Email Readiness Criteria	94
6.3.6	Agent Dashboard Evidence	95
6.4	Run-Based Evidence Case Study	95
6.4.1	Selected Run Summary	95
6.4.2	Trace and Agent Activity	97
6.4.3	Landing Evidence	98
6.4.4	Hosting and Player Evidence	99
6.4.5	Stream, Provider, and Email Evidence	100
6.5	Dashboard Telemetry and Cost Review	101
6.5.1	Overview Metrics	102

6.5.2	Token and Runtime Cost	104
6.5.3	Model and Runtime Selection Rationale	104
6.6	Limitations and Interpretation	105
6.6.1	Operational Limitations	105
6.6.2	Interpretation of the Current Results	105
6.7	Chapter Conclusion	106
7	General Conclusion	107

List of Figures

1.1	Visual context for the host organization and the sports media-rights domain of the internship.	21
1.2	Position of the n8n landing agent as a backup to the company workflow, with OWC kept as the broader academic reference architecture.	22
1.3	Report reading map connecting the internship context, design, implementation, evaluation, and conclusion.	25
2.1	DSRM selection logic for an artifact-centered PFE project.	26
2.2	DSRM backbone used to structure the OWC problem, artifact, demonstration, evaluation, and communication.	27
2.3	Internship timeline from evidence needs analysis to evaluated OWC architecture.	29
3.1	Common navigation flow from event listing to hosting page and embedded player.	33
3.2	Browser-state view of the three page roles: landing pages discover candidate links, hosting pages activate a player context, and embedded pages provide the media request that proves playback.	33
3.3	Simplified DOM tree showing how a browser represents an HTML document.	36
3.4	Main paths from a hosting page to media evidence, with embedded pages treated as an explicit fallback when the hosting page exposes a player URL.	37
3.5	Simplified adaptive streaming hierarchy used by HLS and MPEG-DASH.	38
3.6	Observe, act, and verify loop used by browser agents.	41
3.7	Prompt and token-budget composition for an evidence-driven browser agent.	42
3.8	General MCP pattern connecting an LLM agent to scoped browser tools.	43
4.1	Use-case diagram for the active OWC console actions: launch, test, dataset management, trace inspection, evidence review, provider lookup, and configuration.	46
4.2	System component diagram grounded in the active console routes, FastAPI modules, agent runtime, browser tools, persistence, and external services.	47
4.3	Frontend route and data-flow diagram showing active console screens, the /agents redirect, and the backend route groups consumed by each screen.	48
4.4	End-to-end flowchart from workflow launch to persisted run-detail evidence, including runtime preflight, routing, conditional provider analysis, and no-stream handling.	49
4.5	End-to-end agent architecture at workflow scale.	50
4.6	Sequence diagram for launching, executing, persisting, and reviewing a workflow run.	50

4.7	Agent interconnection graph showing orchestrator-owned routing and specialist responsibilities.	51
4.8	Prompt compilation and memory inputs for each browser-facing specialist. . . .	52
4.9	Shared activity loop used by the browser-facing agents: prompt compilation, MCP session setup, model/tool turns, telemetry recording, context control, schema normalization, and optional memory writing.	53
4.10	Orchestrator activity diagram showing initialization, classification routing, specialist queues, aggregation, provider/email decisions, and final PipelineResult construction.	54
4.11	Orchestrator-agent design card.	54
4.12	Sequence diagram for orchestrator handoff, trace recording, and route updates.	55
4.13	Classification-agent design card.	55
4.14	Classification-agent sequence: inspect page state and return a routing decision.	56
4.15	Landing-page-agent design card.	56
4.16	Landing-agent sequence: convert listing-page evidence into downstream queues.	57
4.17	Hosting-page-agent design card.	57
4.18	Hosting-agent sequence: activate watch pages and return stream or iframe evidence.	58
4.19	Embedded-page-agent design card.	58
4.20	Embedded-agent sequence: stay in the player context and harvest final media evidence.	59
4.21	Provider-analysis and email-generation design card.	60
4.22	Profile-scoped MCP browser tooling and runtime controls used by OWC agents.	61
4.23	Global browser-tool flow. The agent inspects page context, selects either URL navigation or JavaScript interaction, validates the resulting evidence, and records reusable trace and memory signals.	61
4.24	Inspection and context-retrieval flow from broad page observation to scoped targets.	62
4.25	Navigation flow split between direct URL navigation and JavaScript-driven page interaction.	62
4.26	Evidence and media-extraction flow from post-action page state to reviewable artifacts.	63
4.27	Memory-tool flow showing hints as guidance that must be validated live. . . .	63
4.28	Runtime schema class diagram centered on PipelineResult, the aggregate Pydantic object that groups classification, matches, extraction results, streams, screenshots, provider analysis, email drafts, status, and metrics.	65
4.29	Persistence model for pipeline runs, extracted evidence, generated review outputs, agent telemetry, prompt compilation, and memory usage.	66

5.1	Role of the n8n workflow as a feasibility playground for browser-agent evidence collection.	68
5.3	n8n classification and landing workflow screenshots.	69
5.4	n8n hosting and embedded-player workflow screenshots.	70
5.2	Full n8n workflow shown as two readable portrait panels. The figure validates the prototype sequence from trigger and classification through landing, hosting, and embedded extraction without using a sideways report page.	71
5.5	How n8n limitations shaped the Open Web Catcher implementation.	72
5.6	Five implementation areas of the Open Web Catcher platform.	74
5.7	Main OWC operator-console screens for dashboard telemetry and live run launch.	76
5.8	OWC review screens for run results and provider enrichment, enlarged so tables remain readable in print.	77
5.9	OWC tool telemetry and runtime configuration screens, enlarged for printed review.	78
5.10	Browser tool execution loop through MCP and Puppeteer.	80
5.11	Prompt evolution from one-pass intent prompts to evidence-driven agent loops.	81
5.12	Agent reasoning loop with memory and tool feedback.	82
5.13	Summarized PostgreSQL persistence model derived from the detailed database class diagram.	84
5.14	Company deployment flow for the landing-agent backup.	85
6.1	Evaluation protocol used for website-batch regression testing.	89
6.2	Agents dashboard used for behavior evaluation. The screenshot validates that agent runs, tool calls, LLM calls, context tracking, and status are separated by agent type instead of being reduced to one final run label.	95
6.3	Selected run summary and status. The screenshot validates that the chosen run is visible in the operator console with its status, timing, model, tool profiles, and evidence counters.	96
6.4	Trace rows and agent activity for the selected run. The screenshot validates that pipeline, orchestration, and specialist-agent steps were persisted for audit and manual review.	97
6.5	Landing-page evidence for the selected run. The screenshot validates that the workflow started from a match-listing context and had to continue toward a hosting/player path rather than stopping after classification.	98
6.6	Hosting/player evidence for the selected run. The screenshot validates that the workflow reached a player state on the discovered hosting path, supporting the manual player-activation review.	99
6.7	Additional hosting evidence for the selected run. The screenshot supports the server/player review by showing a second captured browser state from the evidence path.	99

6.8	Streams and provider evidence for the selected run. The screenshot validates that stream/provider rows were persisted together with stream count and screenshot support.	100
6.9	Provider-intelligence evidence for the selected run. The screenshot validates that the playlist host was enriched into provider, organization, country, and contact metadata for review.	101
6.10	Overview dashboard metrics used in Chapter 6. The screenshot validates the current total runs, strict success rate, average latency, total cost, token volume, and tool-success cards behind the telemetry tables.	102
6.11	Token telemetry dashboard. The screenshot validates that the console separates new input, cached input, output tokens, cache-hit percentage, and average token volume for cost review.	104

List of Tables

2.1	DSRM phase mapping to project activities and outputs.	29
2.2	Functional requirements derived from the evidence-collection workflow.	30
2.3	Non-functional requirements and implementation responses.	31
3.1	Streaming-site page taxonomy and evidence role.	35
3.2	DOM signals used to understand streaming-site pages.	36
3.3	Browser-level obstacles and handling strategies.	40
4.1	Use-case responsibilities across operator and administration workflows.	47
4.2	Global runtime boundaries used by OWC.	48
4.3	Agent and tool responsibility map.	52
4.4	Channel detection responsibilities across the specialist agents.	60
4.5	Browser runtime controls that support reliable and reviewable evidence collection.	64
4.6	Runtime data boundaries and persisted review outputs.	67
5.1	n8n prototype agent roles and implementation lessons.	72
5.2	n8n prototype limitations and engineering consequences.	73
5.3	Backend API responsibilities in OWC.	74
5.4	Current Next.js operator-console surfaces and backend dependencies.	75
5.5	Browser automation engine comparison for the MCP tool layer.	79
5.6	Puppeteer MCP tool groups and responsibilities.	79
5.7	Specialist agent objectives and assigned tool surfaces.	80
5.8	Runtime constraints and their implementation responses.	81
5.9	Model-runtime concerns handled by the agent platform.	83
5.10	Prompt engineering rules enforced by the specialist agents.	83
5.11	PostgreSQL observability records and their implementation role.	84
5.12	Landing-agent deployment scope and responsibilities.	86
6.1	Website-level acceptance criteria used during batch evaluation.	89
6.2	Failure taxonomy used to separate productive evidence, external blockers, agent/runtime failures, cancelled runs, and active runs.	90
6.3	KPI definitions used to interpret batch results without hiding partials or external blockers.	91
6.4	Current batch outcome snapshot used for Chapter 6 interpretation.	92
6.5	Agent-specific tests used during manual and batch evaluation.	93

6.6	Selected-run metadata used as the main Chapter 6 evidence case study.	96
6.7	Current evidence, provider, tool, cost, and token telemetry from the overview dashboard/API snapshot.	103
6.8	Model/runtime selection rationale used instead of unsupported model-ranking metrics.	105
6.9	Limitations that remain after evidence-first evaluation.	106

List of Acronyms

Acronym	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ASB	Azure Service Bus
CDN	Content Delivery Network
CDP	Chrome DevTools Protocol
DASH	Dynamic Adaptive Streaming over HTTP
DB	Database
DNS	Domain Name System
DOM	Document Object Model
DSRM	Design Science Research Methodology
E2E	End-to-End
HLS	HTTP Live Streaming
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
IP	Internet Protocol
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LLM	Large Language Model
M3U8	UTF-8 M3U playlist format

Acronym	Meaning
MCP	Model Context Protocol
MPEG-DASH	MPEG Dynamic Adaptive Streaming over HTTP
MPD	Media Presentation Description
OWC	Open Web Catcher
OCR	Optical Character Recognition
PFE	Projet de Fin d'Etudes
RDAP	Registration Data Access Protocol
RPD	Requests Per Day
RPM	Requests Per Minute
SSE	Server-Sent Events
SQL	Structured Query Language
TPM	Tokens Per Minute
TTL	Time To Live
UI	User Interface
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
UX	User Experience
WHOIS	Domain registration lookup protocol or service

Terminology

Open Web Catcher (OWC)	The internal name of the complete evidence-collection platform built during the internship. It refers to the FastAPI backend, Next.js operator console, PostgreSQL persistence, MCP browser tooling, specialist-agent runtime, and review outputs used to catch open-web evidence from browser-visible streaming pages.
Agent	A model-driven runtime component that receives a bounded task, uses approved tools, and returns structured evidence or a failure reason.
Orchestrator	The control component that classifies the starting URL, prepares handoffs, schedules specialist agents, aggregates evidence, and decides the final run status.
Specialist agent	A browser-facing agent with a narrow responsibility, such as classification, landing-page discovery, hosting-page extraction, or embedded-player inspection.
Landing page	A page that lists events, channels, schedules, or match cards and is mainly useful for discovering downstream hosting candidates.
Hosting page	A page that usually owns play buttons, server/source controls, overlays, and redirects before a stream or embedded player becomes visible.
Embedded page	A direct player or iframe page used as a fallback extraction context when the hosting page exposes a player URL
Browser automation	Programmatic control of a real browser session to inspect rendered pages, click controls, manage frames, capture screenshots, and observe network activity.
Tool call	A browser, provider, memory, or runtime operation requested by an agent and recorded as part of the run trace.
Handoff	A short structured task brief prepared by the orchestrator for the next specialist agent, including target URL, evidence needed, route source, and navigation policy.
Trace	The ordered record of runtime events, model calls, tool calls, screenshots, status changes, costs, and final outputs for one investigation.
Observability	The ability to explain what happened during a run after it finishes, including which agent acted, which tools were used, and why a failure occurred.
Stream evidence	A URL, manifest, media request, screenshot, or player state that supports the claim that a target page exposed unauthorized streaming behavior.
Manifest	A playlist or media presentation file, such as HLS or MPEG-DASH, that points a player toward the media segments required for playback.

Iframe	An embedded browser frame that loads a player or third-party page inside the visible hosting page.
Provider enrichment	The step that resolves collected stream hosts or IP addresses into provider, country, network, or abuse-contact information.
Takedown email	A reviewable draft that groups stream evidence, provider details, screenshots, and rights-owner context for later human use.
Regression batch	A repeated set of website tests used to verify that a prompt, model, or tool change fixes new cases without breaking previously working ones.
Evidence artifact	Any persisted output used for review, including screenshots, streams, provider rows, emails, model usage, and tool results.
Runtime event	A timestamped event emitted during execution, such as agent start, tool success, screenshot capture, stream extraction, or failure.
Prompt	The role and task instruction compiled for an agent before it calls the language model.
Memory hint	A reusable site or layout note injected as soft guidance; it can help the agent but must be confirmed against live browser evidence.
Input tokens	Prompt, trace, tool-output, memory, and context text sent to the model during an LLM call.
Cached input tokens	Input tokens reused through the prompt or context cache instead of being billed and processed as fully new context.
Output tokens	Tokens generated by the model as an answer, tool decision, explanation, or structured result.
Total tokens	The aggregate token count used by the dashboard to summarize input and output usage for model calls.
Context window	The maximum amount of prompt, trace, tool output, and memory text that the model can consider in one call.
Prompt cache	A cache mechanism used to reuse repeated prompt and context content across similar model calls.
Cache hit rate	The share of model input served from cache within a specific dashboard or model-usage view. It must be read with its denominator.
LLM call	One recorded invocation of the language model, including the prompt, model name, token counters, and cost estimate.
Model spend	The recorded or estimated monetary cost of model calls for a run, batch, or dashboard snapshot.
Cost per run	Model spend normalized by the number of runs in the selected view; it is used to compare evaluation versions.
Tool success rate	Successful observed tool calls divided by observed tool calls in the same telemetry snapshot.
Stream yield	The share of runs that produced at least one stream, manifest, media, or embedded-player evidence candidate.

Strict stream success	A stricter success measure where a run counts only when it reaches reviewable stream evidence, not merely a completed status.
Pipeline run	One end-to-end investigation from a seed URL through classification, specialist routing, evidence collection, enrichment, and final status.
Active run	A pipeline run that is currently queued, running, or waiting on browser/model work.
Queued run	A run or task that has been accepted but has not yet started browser execution.
Run status	The final or current state of a pipeline run, such as success, partial, failed, cancelled, no_streams, or no_hosting_pages.
no_streams	A terminal status used when the browser path was explored but no reviewable media, manifest, stream, or embedded evidence was found.
no_hosting_pages	A terminal status used when a landing-page investigation did not discover credible downstream hosting candidates.
Evidence bundle	The grouped review output for one run, including streams, screenshots, provider rows, email drafts, status, costs, and trace links.
Provider coverage	The extent to which collected stream or host evidence could be enriched with provider, network, or registration context.
Dashboard snapshot	A live operator-console aggregate at a specific time. It should not be mixed blindly with controlled regression-batch metrics.
Persistence health	A dashboard view showing whether runs, agent runs, runtime events, model calls, tool calls, and outputs are being stored correctly.
Tokenized URL	A media or manifest URL containing temporary query parameters, signatures, or session-bound path values.
Queue message	A structured payload sent through a queue to start work or return results without blocking a synchronous web request.
agent_tasks	The Azure Service Bus input queue used in the company deployment flow to receive target URLs and task metadata.
Results queue	The configured Azure Service Bus output queue used in the company deployment flow to return matched URLs, metadata, screenshots, rejected URLs, and failure information.
Browser profile	A browser context with its own state, cookies, cache, and execution settings for a run or agent.
Profile-scoped session	A browser session tied to one profile so page state can be preserved while avoiding uncontrolled sharing across unrelated runs.
Static collection	A monitoring step that fetches pages or links without completing the full rendered browser interaction path. In this project it remains useful for discovery but can miss evidence that appears only after scripts, clicks, frames, or playback.

Rendered browser state	The visible and runtime state produced after the browser executes JavaScript, handles frames, loads media, and reacts to user-like actions. OWC treats this state as an evidence surface.
Channel detection	The process of identifying a broadcaster or channel signal from landing hints, player labels, screenshots, OCR text, structured agent output, or stream metadata. Landing hints must be verified by hosting or embedded evidence when possible.
Fingerprint profile	A browser identity profile used to keep navigator, viewport, session, and related browser-surface behavior coherent during a run. It reduces obvious automation inconsistencies but does not guarantee bypass of anti-bot systems.
Proxy rotation	The runtime selection of a network proxy for browser execution. In OWC, proxy candidates are validated and kept sticky until failure to avoid breaking session-bound pages.
Sticky proxy	A proxy selected for a browser profile and reused while it remains healthy. This prevents identity changes in the middle of one investigation.

Introduction

This PFE report presents the work carried out at Soft Stars in the context of illegal sports-streaming evidence collection for beIN SPORTS. The project started from a concrete operational need: suspicious pages could be discovered, but the evidence needed for review was often hidden behind browser-rendered interfaces, popups, iframe players, server buttons, redirects, and short-lived media manifests. The internship therefore focused on designing an evidence workflow that could observe the page like a browser user, keep a trace of what happened, and return reviewable evidence instead of only a final URL.

1.1 Host Organization and Internship Context

Soft Stars is a software company working in digital media intelligence, content protection, and technical support for rights-enforcement operations. Its work focuses on helping media organizations observe online distribution channels, identify unauthorized use of protected content, and prepare technical evidence that can be reviewed by operational, legal, or business teams.

The internship took place in a sports media-protection context connected to beIN SPORTS. In this domain, the practical problem is not only detecting that a suspicious website exists. A rights-protection workflow also needs enough evidence to understand what was visible, which channel or event was exposed, which page led to the player, which technical host or provider was involved, and whether a human reviewer can trust the result after the browser session has ended.



Figure 1.1: Visual context for the host organization and the sports media-rights domain of the internship.

1.2 Existing Solutions and Project Positioning

Before this PFE work, the company already had an operational workflow for monitoring and processing suspicious streaming websites. That existing work remained the primary company process. It was useful for broad discovery, recurring checks, and normal cases where the page structure was stable enough to process through crawler-style logic. However, many streaming sites hide the useful evidence behind browser runtime behavior: JavaScript-rendered match

cards, server buttons, popups, nested iframes, player activation steps, and short-lived m3u8 or mpd media requests. In this report, the company workflow means the existing Soft Stars operational monitoring and crawler-based processing path that remains the primary company work.

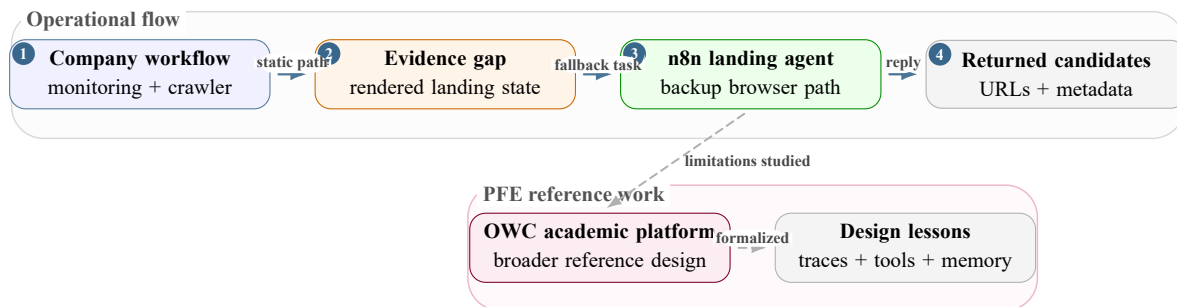


Figure 1.2: Position of the n8n landing agent as a backup to the company workflow, with OWC kept as the broader academic reference architecture.

The n8n landing agent was therefore positioned as a backup to the company workflow, not as a replacement for it. When the primary flow could not find enough landing-page evidence, it could send a task containing the target URL and execution metadata to the landing agent. The agent then opened the rendered page, inspected visible match and channel cues, followed candidate links when needed, and returned candidate URLs, metadata, screenshots, rejected links, and reasoning notes for review or downstream processing. In this report, the n8n landing agent means the focused backup browser workflow used when the company workflow needs rendered landing-page reasoning.

OWC had a different role. It was the academic and engineering reference platform built during the internship to study the n8n limitations and propose a cleaner architecture around specialist agents, browser tools, traces, memory, model telemetry, persistence, and evaluation. In other words, the deployed company-side decision stayed focused on a landing-agent backup, while OWC describes the broader design that could support a more complete future platform. In this report, OWC means that academic/reference platform, not the deployed company workflow itself.

1.3 Problem Statement

Illegal streaming websites are difficult to investigate because they combine unstable content, dynamic rendering, misleading navigation, popup traps, nested frames, and short-lived media URLs. The evidence path often has several roles:

- a landing page lists matches, events, schedules, or channel cards;
- a hosting page exposes play buttons, server controls, overlays, and iframe candidates;
- an embedded page or direct player exposes the final media behavior;
- a provider-enrichment stage connects collected stream evidence to host, network, country, or abuse-contact information.

If those roles are mixed together, the system becomes hard to debug. A failed result could mean that the page was classified incorrectly, that a landing candidate was missed, that a hosting server was not activated, that an iframe was not inspected, or that the final media request was not connected to provider evidence. The project therefore required an architecture that separates page roles, specialist agents, browser tools, screenshots, traces, model usage, provider enrichment, and final review outputs.

1.4 Internship Mission and Scope

My mission during the internship was to study, design, and validate an evidence collection workflow that uses browser-based AI reasoning to support dynamic evidence paths when the existing company workflow needs a backup. The work produced two related artifacts.

The first artifact was an n8n landing-agent workflow. It was used to validate the idea quickly in the company environment and to provide a deployable backup path for landing pages that required rendered-page reasoning. This workflow helped test whether an agent could inspect visible page state, follow candidate links, handle page-specific obstacles, and return useful match or hosting metadata.

The second artifact was Open Web Catcher (OWC), the complete platform designed during the internship as an academic/reference architecture. The internal name was chosen because the project catches evidence from the open web through browser-visible behavior rather than through a static HTML response. In this report, OWC refers to the whole project: the FastAPI backend, Next.js operator console, PostgreSQL persistence, MCP browser tooling, Puppeteer/Chrome runtime, specialist agents, model telemetry, and evidence review outputs.

1.4.1 Mission Deliverables

The internship deliverables were:

- a validated n8n landing-agent workflow used as a deployable backup to the company workflow for dynamic landing-page targets;
- an OWC architecture separating backend, frontend, database, agents, browser tools, and evidence outputs;
- browser-tool profiles for classification, landing, hosting, and embedded-player investigation;
- support for screenshots, stream candidates, provider enrichment, channel metadata, tool traces, model telemetry, and cost visibility;
- an evaluation and reporting structure based on traceability and reviewability.

1.4.2 Scope Boundaries

The project did not replace the full company enforcement process. It did not automatically send takedown messages, and it did not claim to solve every anti-bot mechanism or every illegal streaming website. The practical scope was to produce an engineering foundation that

could investigate dynamic pages, preserve evidence, expose failures clearly, and support future operational integration.

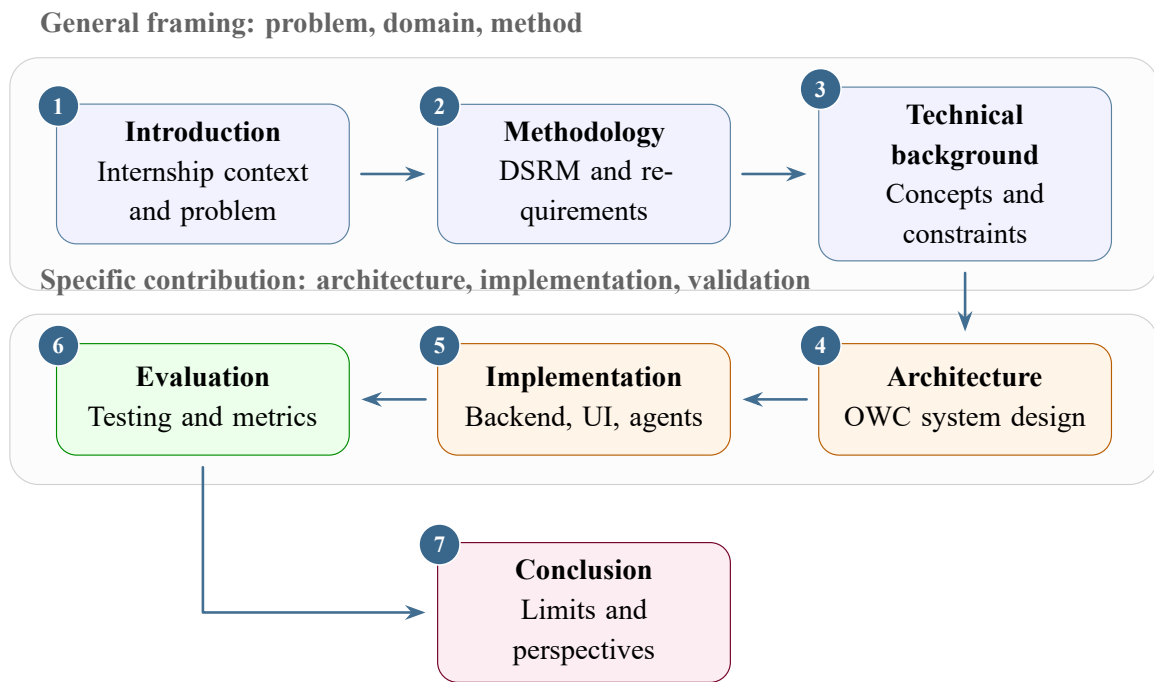
1.5 Main Contributions

The main contributions of the PFE work are:

- a clear analysis of why static page collection is insufficient for many dynamic streaming-evidence paths;
- a DSRM-based artifact design centered on Open Web Catcher;
- an n8n landing-agent workflow deployed as an operational backup to the company workflow for dynamic evidence cases;
- a multi-agent architecture that separates classification, landing discovery, hosting interaction, embedded-player inspection, provider enrichment, and review output generation;
- a browser-tool layer with profile-scoped inspection, navigation, interaction, screenshot, media extraction, memory, fingerprint, and proxy controls;
- an operator console and persistence model that make traces, screenshots, model usage, tool calls, costs, channel signals, and failure reasons reviewable.

1.6 Report Structure

Chapter 2 presents DSRM as the sole methodology and derives the project requirements. Chapter 3 introduces the technical background needed to understand streaming pages, browser automation, media manifests, agent workflows, and evidence traceability. Chapter 4 describes the OWC architecture, including the agent responsibilities, browser tooling, fingerprint and proxy handling, channel detection, persistence, and reviewability. Chapter 5 explains the implementation path from the n8n backup workflow to the OWC platform and the company deployment flow. Chapter 6 presents evaluation and testing results. Chapter 7 provides the general conclusion.



The report connects the internship problem, design choices, implementation, evaluation, and final perspectives.

Figure 1.3: Report reading map connecting the internship context, design, implementation, evaluation, and conclusion.

Methodology, Requirements Analysis, and Design Constraints

The internship was organized around Design Science Research Methodology (DSRM). This choice is deliberate: the work did not consist of studying a phenomenon from the outside or training a predictive model. It consisted of designing, building, demonstrating, and evaluating an engineering artifact for a concrete organizational problem. In this report, that artifact is Open Web Catcher (OWC), the evidence collection platform built around browser tooling, specialist agents, trace storage, screenshots, provider enrichment, and reviewable outputs.

DSRM provides the backbone for the chapter structure that follows. The problem is first identified and motivated, then solution objectives are defined, the artifact is designed and developed, the artifact is demonstrated, the artifact is evaluated, and finally the work is communicated through this PFE report and its diagrams. This phase logic follows the design-science view of information-systems artifacts proposed by Hevner et al. [7] and the DSRM process described by Peffers et al. [8].

2.1 DSRM as the Methodological Backbone

DSRM was selected because it gives equal importance to the organizational problem and to the engineered artifact. The operational problem was concrete: useful evidence often appeared only after browser rendering, interaction, iframe traversal, server switching, or media activation. The PFE work therefore needed a methodology that could justify a new artifact, not only describe a work plan.

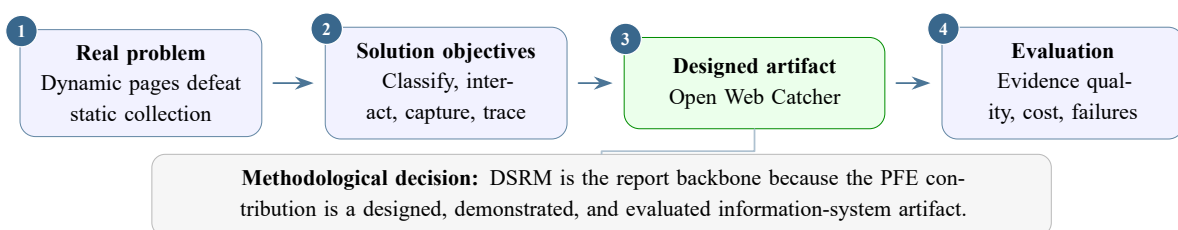


Figure 2.1: DSRM selection logic for an artifact-centered PFE project.

2.1.1 Why DSRM Fits the Internship

The project has three properties that make DSRM suitable. First, it is artifact-centered: the final contribution is a platform and workflow design, not a pure analysis. Second, the artifact is evaluated through use: it is meaningful only if it can inspect real pages, produce traces, capture

screenshots, and return evidence. Third, the organizational relevance is explicit: Soft Stars needed a more adaptable way to handle illegal streaming pages that became too dynamic for static extraction.

2.1.2 DSRM Phases Used in the Report

The six DSRM phases are not treated as abstract theory. Each one corresponds to work carried out during the internship. Figure 2.2 summarizes the adopted backbone.

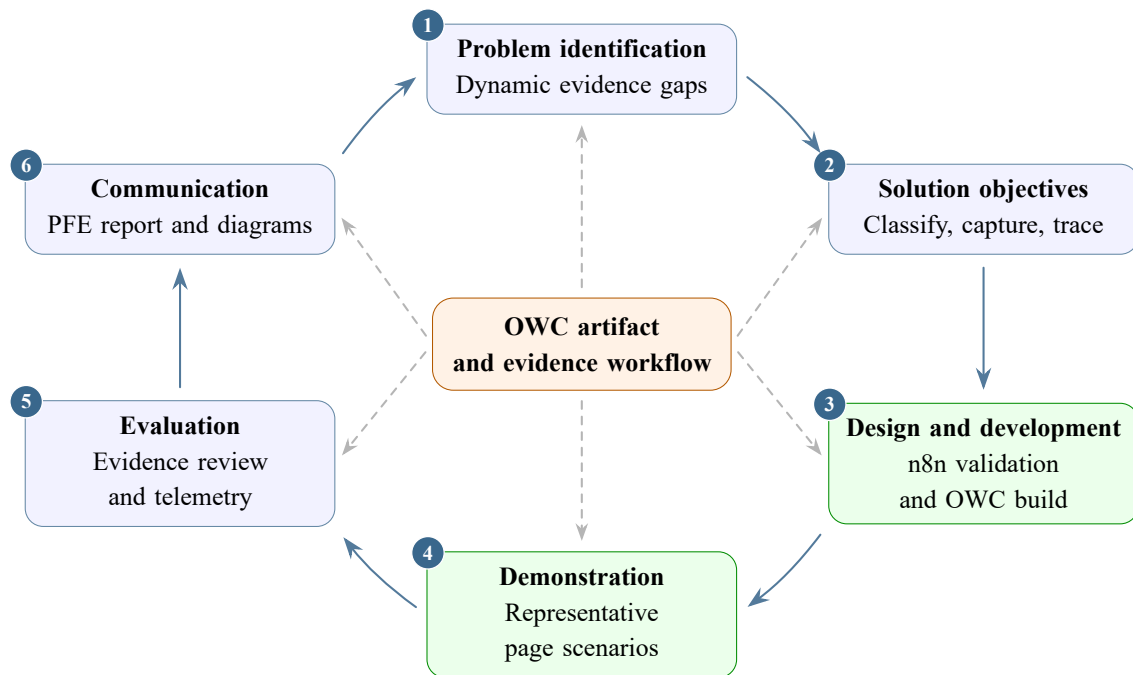


Figure 2.2: DSRM backbone used to structure the OWC problem, artifact, demonstration, evaluation, and communication.

2.1.2.1 Problem identification and motivation

The first phase established the practical problem: suspicious streaming pages could be found, but collecting reviewable evidence was difficult. Some pages exposed only lists of matches or channels. Others required a click on a server button before a player loaded. Others hid the player in an iframe or generated short-lived manifest URLs only after playback. A static HTML fetch can miss these states because it does not perform the same sequence of visual and network actions as a browser user.

2.1.2.2 Objectives of the solution

The second phase translated the problem into objectives. The artifact needed to ingest a URL, classify the page role, follow landing-to-hosting and hosting-to-embedded handoffs, activate players when necessary, collect screenshots, detect stream evidence, preserve channel and provider metadata when available, and store enough trace data for later review. These objectives became the functional and non-functional requirements listed later in this chapter.

2.1.2.3 Design and development

The third phase produced the artifacts. The n8n workflow was used first because it allowed fast operational validation of an AI-assisted landing workflow and could act as a backup path for dynamic evidence cases. OWC then formalized the same ideas in a maintainable platform with FastAPI, Next.js, PostgreSQL, MCP browser tooling, Puppeteer/Chrome, profile-scoped sessions, model telemetry, and specialist agents.

2.1.2.4 Demonstration

The fourth phase demonstrated that the artifact could operate on representative page types: landing pages, hosting pages, embedded player pages, popup-heavy pages, tokenized manifest cases, and pages where channel evidence had to be verified from visible player or screenshot context. Demonstration was not limited to a happy path; it included runs where the system returned explicit failures such as no hosting pages, no streams, or skipped downstream enrichment.

2.1.2.5 Evaluation

The fifth phase evaluated the artifact through trace inspection, tool telemetry, screenshots, token and cost records, stream yield, provider enrichment, and dashboard snapshots. The aim was not to claim that every illegal streaming page can be solved. The aim was to verify whether the artifact made the extraction process more observable, repeatable, and reviewable than a one-shot static fetch or a manually inspected page.

2.1.2.6 Communication

The final phase communicates the artifact. This report presents the problem, methodology, technical background, architecture, implementation, evaluation, and limitations. The diagrams and tables make the artifact understandable as an engineering contribution rather than reproduce internal source files.

2.2 DSRM Mapping to the Internship Work

Table 2.1 maps each DSRM phase to the work performed during the internship and to the concrete outputs used in the rest of the report.

DSRM phase	Application in this project	Main output
Problem identification and motivation	Analyze static-collection limits, dynamic page behavior, stream activation, iframe traversal, channel ambiguity, and the need for reviewable evidence.	Problem statement and motivation for OWC
Objectives of the solution	Define the extraction, traceability, screenshot, channel, provider, cost, and review requirements that the artifact must satisfy.	Functional and non-functional requirements
Design and development	Build the n8n validation workflow, then design OWC as a platform with backend, frontend, database, browser tools, agents, and observability.	n8n workflow and OWC architecture
Demonstration	Run the artifact on representative landing, hosting, embedded, popup, iframe, and tokenized-stream scenarios.	Demonstrated evidence paths and failure paths
Evaluation	Inspect traces, screenshots, stream evidence, provider enrichment, tool behavior, token usage, costs, and dashboard telemetry.	Evaluation metrics, results, and limitations
Communication	Produce the PFE report, diagrams, final PDF, and design explanation for academic and company review.	Written report and final synthesis

Table 2.1: DSRM phase mapping to project activities and outputs.

2.3 Project Evolution Timeline

The timeline follows the DSRM logic but remains concrete. The internship started with evidence needs and static-collection limits, moved to browser-agent experiments, then to the n8n landing-agent backup workflow, and finally to the OWC platform architecture and evaluation.

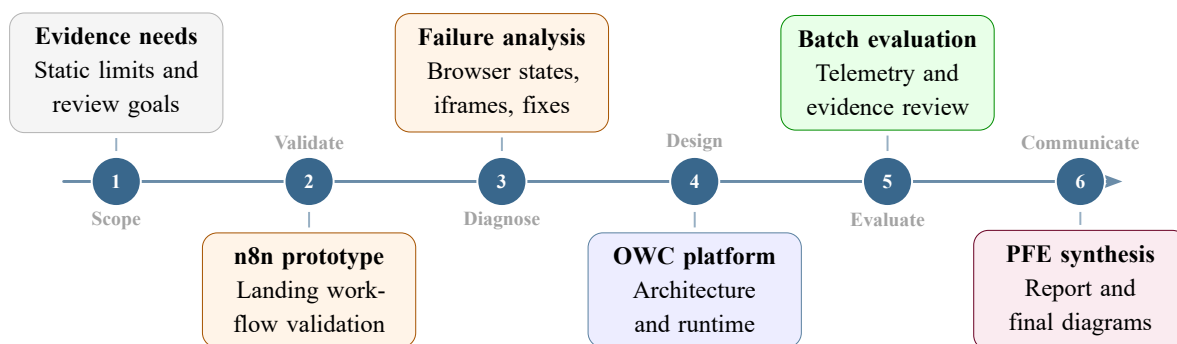


Figure 2.3: Internship timeline from evidence needs analysis to evaluated OWC architecture.

2.4 Functional Requirements

The functional requirements describe what the artifact must do to support the evidence-collection workflow.

ID	Requirement summary	Priority
FR-1	URL ingestion and run lifecycle management	Critical
FR-2	Page-type classification using rendered browser evidence	Critical
FR-3	Orchestrator handoff from classification to landing, hosting, or embedded agents	Critical
FR-4	Landing-page discovery of match, event, channel, and hosting candidates	High
FR-5	Hosting-page interaction with server/source controls, overlays, and play buttons	Critical
FR-6	Embedded-player inspection through frame trees, player state, and media signals	Critical
FR-7	Screenshot capture at important evidence and failure points	High
FR-8	Stream evidence extraction for manifests, media URLs, and embedded fallbacks	Critical
FR-9	Channel detection from landing hints, player context, screenshot/OCR text, and structured agent output	High
FR-10	Provider enrichment and reviewable evidence bundle generation	Medium
FR-11	Operator console views for runs, telemetry, screenshots, tools, settings, and provider results	High

Table 2.2: Functional requirements derived from the evidence-collection workflow.

2.5 Non-Functional Requirements

The non-functional requirements explain the qualities required for the artifact to be useful after a run finishes.

This DSRM framing defines the rest of the report. Chapter 3 introduces the technical concepts needed to understand the artifact. Chapter 4 describes the OWC architecture, including the agent routing, browser tool layer, fingerprint and proxy controls, and channel-detection path. Chapter 5 then explains how those decisions were implemented.

ID	Non-functional requirement	How it is addressed
NFR-1	Maintainability	OWC separates backend routes, frontend views, agent contracts, browser tools, runtime settings, and persistence records.
NFR-2	Observability	Runs persist runtime events, agent runs, LLM calls, tool calls, screenshots, streams, provider rows, and final statuses.
NFR-3	Traceability	Each result can be connected back to a URL, agent, tool call, screenshot, stream candidate, and final decision.
NFR-4	Reliability	Profile-scoped sessions, bounded retries, popup recovery, iframe recovery, media verification, and explicit failure states reduce silent errors.
NFR-5	Cost control	Token counters, cached input tracking, model settings, rate-limit awareness, and dashboard cost summaries make model usage visible.
NFR-6	Adaptability	Page-type specialists, browser profile controls, proxy configuration, fingerprint rotation, and memory hints support changing site behavior.
NFR-7	Deployment fit	The company deployment keeps n8n as a backup operational orchestrator and runs the landing agent through queue-driven container jobs.

Table 2.3: Non-functional requirements and implementation responses.

Technical Background

This chapter introduces the technical concepts needed before presenting the system architecture and implementation. It deliberately stays at a general level: the goal is to explain the web structures, streaming mechanisms, browser automation tools, and agentic concepts that make the project understandable. The concrete architecture, database schema, prompts, and tool contracts are described later.

The chapter moves from the target websites themselves toward the technologies used to analyze them. It starts with page structures and DOM representation, then explains streaming protocols and embedded players, then covers browser automation issues such as ads, cookies, popups, and anti-bot providers. It ends with the agentic concepts used in the project: agents, prompts, tool use, workflow automation, LangChain, MCP servers, model providers, caching, and token usage.

3.1 Streaming Website Structures

Illegal streaming websites are not a single page type. They are usually a chain of pages with different responsibilities: one page lists events, another page presents a selected match and server choices, and a final embedded page or iframe hosts the player. Understanding these page roles is necessary before discussing extraction tools, because each role exposes different evidence and requires different interaction.

3.1.1 The normal page flow

Most target websites can be described as a three-step navigation path. A landing page contains match listings. A hosting page focuses on one selected match and usually offers one or more streaming servers. An embedded page contains the actual player, often inside an iframe loaded from another domain.

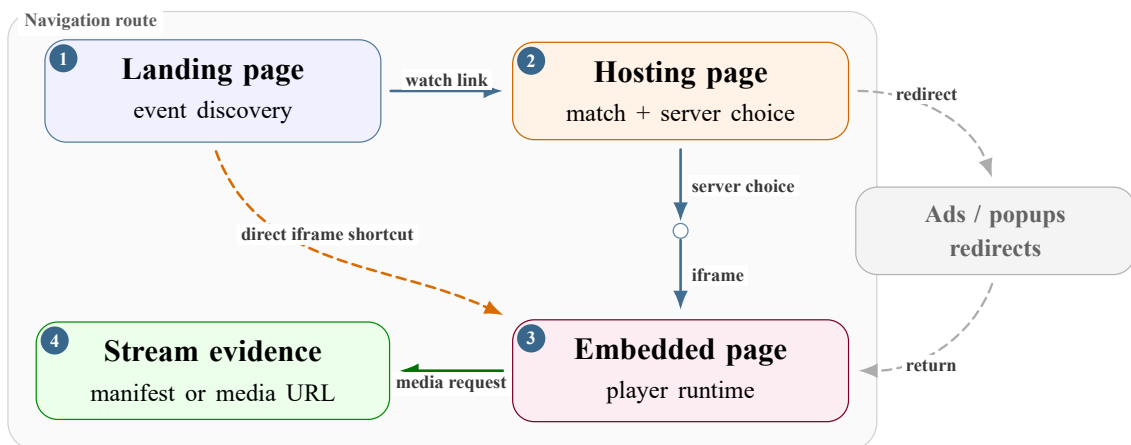


Figure 3.1: Common navigation flow from event listing to hosting page and embedded player.

Figure 3.1 shows the navigation sequence as an abstract route. The next figure uses the same sequence from the browser's point of view: it highlights what each page role exposes to the automation pipeline and what output is expected from that role before the pipeline moves on.

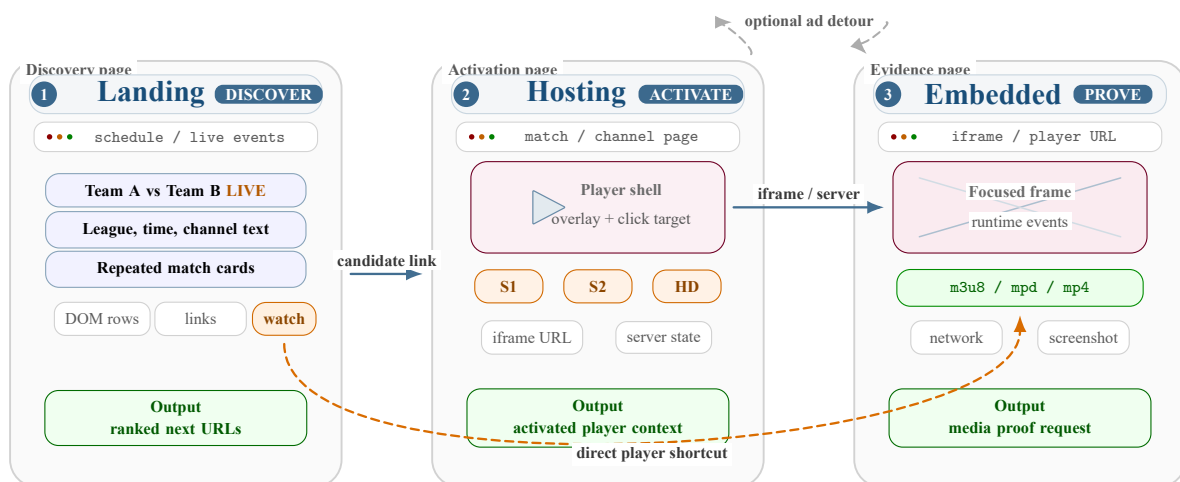


Figure 3.2: Browser-state view of the three page roles: landing pages discover candidate links, hosting pages activate a player context, and embedded pages provide the media request that proves playback.

This flow is not always clean. Some sites skip the hosting page and link directly to an embedded player. Others place several ad pages between the hosting page and the player. Still, this model is useful because it separates discovery, server selection, and media extraction into distinct technical problems.

3.1.2 Landing pages

Landing pages are the discovery layer. They usually contain match cards, schedule rows, league filters, date tabs, search boxes, or category menus. The useful output from a landing page is not the stream itself but a set of candidate hosting links and match metadata.

Common landing-page layouts include:

- **card grids:** repeated blocks with teams, time, league, and a watch link;
- **tables or lists:** one row per event, often with a direct action link;
- **tabbed schedules:** live, today, yesterday, tomorrow, or league-specific tabs loaded through JavaScript;
- **lazy-loaded sections:** match cards that appear only after scrolling or after an XHR request;
- **embedded JSON:** page data stored in `<script type=application/json>` blocks or `data-*` attributes.

The important idea is that landing pages are structurally repetitive. Even when class names change, the page still tends to contain repeated event units. This makes layout recognition more useful than relying on one fixed CSS selector.

3.1.3 Hosting pages

Hosting pages are the decision layer. They normally focus on one match and contain a player container, a server list, tabs such as Server 1 or HD, and sometimes channel names or broadcaster logos. A hosting page may already expose the stream URL, but more often it only exposes an iframe or a JavaScript player configuration.

Typical hosting-page signals are:

- one match title or two team names repeated near the player;
- server buttons, mirrors, or quality selectors;
- an iframe whose `src` points to a player page;
- player libraries such as Video.js, JWPlayer, Clappr, hls.js, or dash.js;
- visible state changes after clicking play, selecting a server, or closing an overlay.

Hosting pages are more difficult than landing pages because they require interaction. The same click can activate a real player, open a popup, switch a server, or trigger a redirect. This is why browser automation and runtime observation become necessary.

3.1.4 Embedded pages

Embedded pages are the media layer. They are often minimal pages loaded inside iframes, with little content except a player, a play button, and scripts that request the final media manifest. They may belong to another domain, which means the top page and the player page have different DOMs, cookies, and network contexts.

An embedded page is important because the final stream reference is often generated only there. A static request to the hosting page may not show any `.m3u8` or `.mpd` URL, while the embedded player requests the manifest after user activation.

The page taxonomy turns a messy website into a sequence of smaller questions: Where are the matches? Which page hosts the selected match? Which server or iframe contains the player? Which runtime request proves that media was loaded?

Page type	Main role	Useful evidence
landing_page	Discover events and candidate match pages	match links, team names, schedule metadata, listing screenshot
hosting_page	Select server and activate player	server labels, iframe URLs, player state, page screenshot
embedded_page	Load the final media runtime	stream manifest, segment requests, player diagnostics
other	Filter noise such as ads, redirects, and unrelated pages	reason for skip or blocked state

Table 3.1: Streaming-site page taxonomy and evidence role.

3.2 DOM Structures and Web Page Representation

The Document Object Model (DOM) is the browser’s tree representation of a web page. It turns HTML into nodes such as documents, elements, attributes, text nodes, iframes, buttons, images, and scripts. Browser automation tools operate on this tree when they query selectors, read attributes, click buttons, or remove overlays.

3.2.1 A simplified DOM tree

The diagram below shows a simplified DOM structure. It is intentionally basic, but it captures the idea used throughout the project: a visible page is not just text, it is a tree of nested elements with attributes and event handlers.

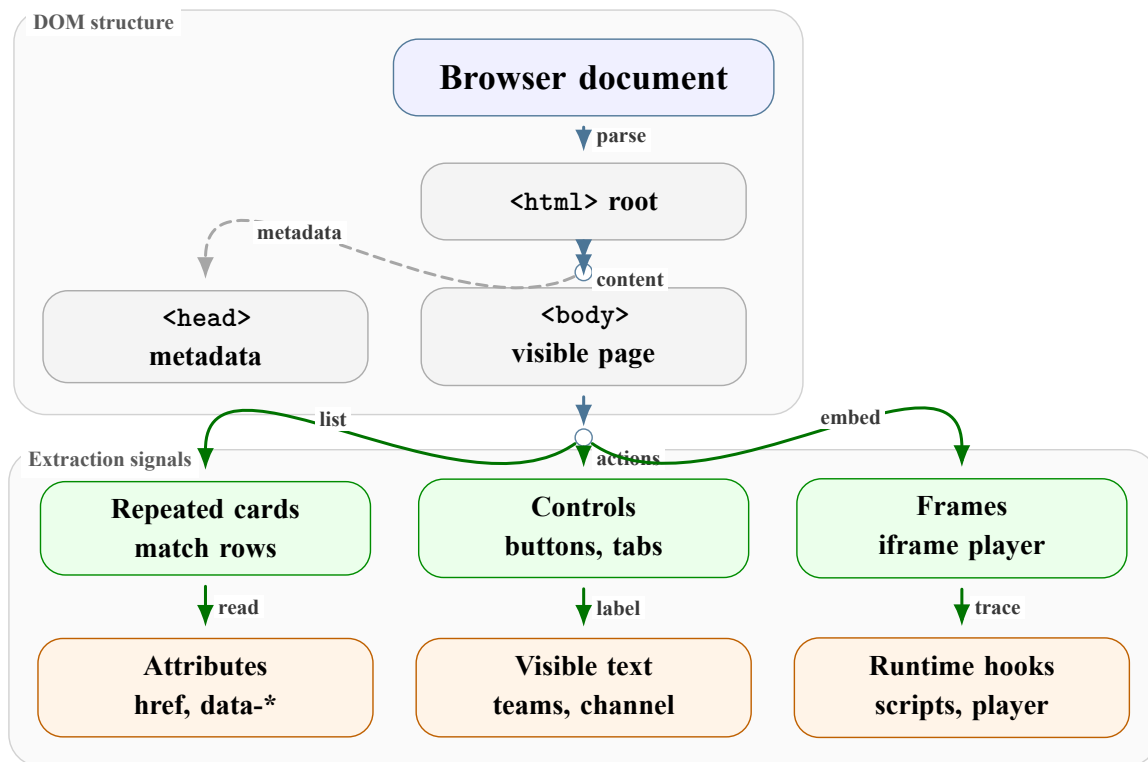


Figure 3.3: Simplified DOM tree showing how a browser represents an HTML document.

For streaming sites, the DOM is useful but incomplete. It can reveal match cards, buttons, iframes, scripts, and player containers, but it may not reveal media URLs that are generated at runtime. This is why DOM inspection is combined with screenshots and network monitoring.

3.2.2 DOM signals on streaming pages

Different page roles expose different DOM signals. Landing pages expose repeated containers. Hosting pages expose player containers and server controls. Embedded pages often expose a reduced tree with one `<video>` element or a JavaScript player root.

DOM signal	Where it appears	Why it matters
Repeated cards or rows	landing pages	indicates match listings and candidate links
<code></code>	landing and hosting pages	provides navigation targets
data-* attributes	landing and dynamic pages	may store event IDs, team names, or hidden URLs
<code><iframe src=...></code>	hosting pages	points to embedded player context
<code><video></code> or <code><source></code>	hosting or embedded pages	may expose direct media source references
script text and JSON blocks	all page types	may contain player configuration or lazy-loaded data

Table 3.2: DOM signals used to understand streaming-site pages.

The conclusion is that DOM analysis is a starting point, not a complete solution. It identifies page structure and interaction targets, while runtime browser evidence confirms what actually happened after the page executed.

3.3 Streaming Protocols and Embedded Players

The final evidence in this domain is usually not a normal web page. It is a media manifest, a player configuration, or a network request made by the browser while the player is active. This section explains the main protocols and player embedding patterns at a general level.

3.3.1 Common embedding patterns

Hosting pages usually embed players in one of three ways. The simplest case is a direct `<video>` element with a `src` or nested `<source>` tag. A common case is an `iframe` that loads a player from another domain. A more dynamic case is a JavaScript player library that inserts the video source after page load.

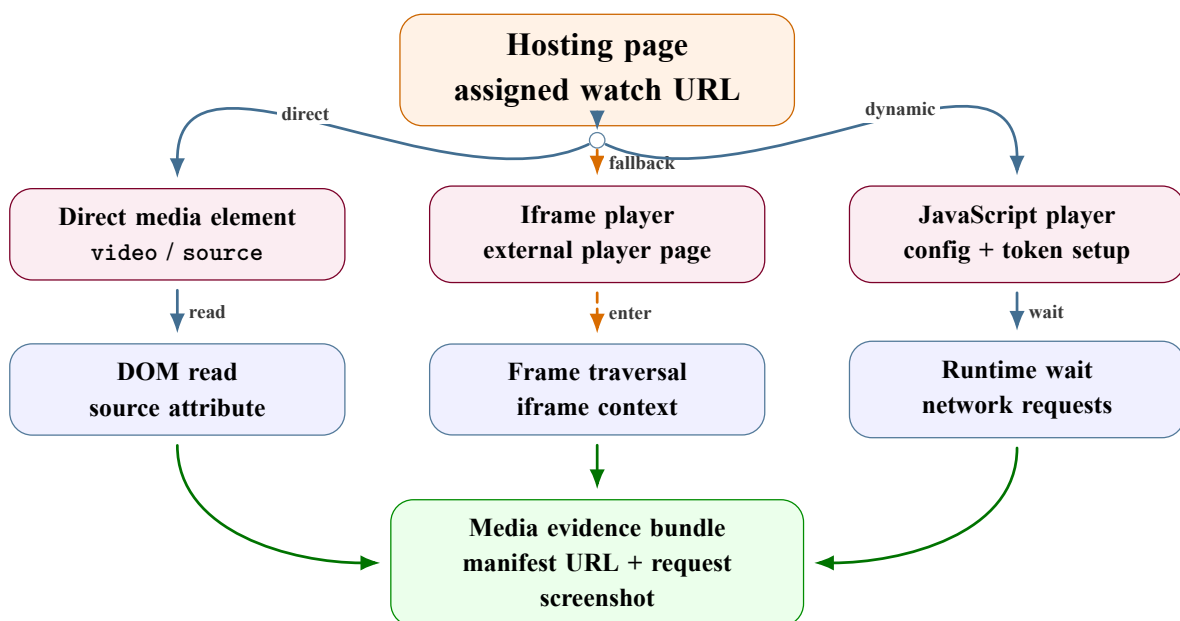


Figure 3.4: Main paths from a hosting page to media evidence, with embedded pages treated as an explicit fallback when the hosting page exposes a player URL.

This distinction matters because each embedding type requires a different observation strategy. Direct video tags can be read from the DOM. Iframes require frame traversal. JavaScript players often require runtime execution and network monitoring.

3.3.2 HLS and MPEG-DASH

HTTP Live Streaming (HLS) is one of the most common protocols for web video streaming. Its entry point is a `.m3u8` playlist that points either to variant playlists or to media segments. MPEG-DASH uses an XML `.mpd` manifest that describes periods, representations, and segment

addressing rules. Both protocols are adaptive: the player can choose different quality levels depending on network conditions [18, 19].

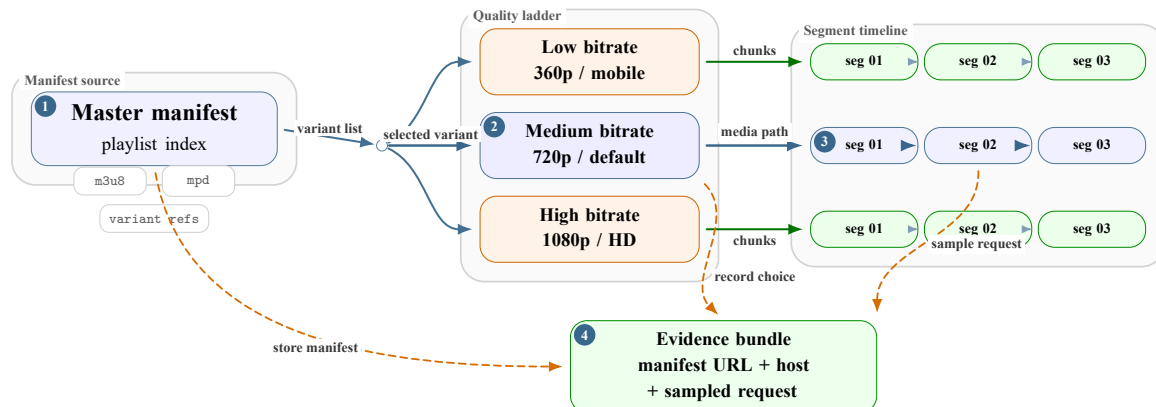


Figure 3.5: Simplified adaptive streaming hierarchy used by HLS and MPEG-DASH.

For evidence collection, the manifest URL is usually more important than downloading media segments. It proves that the page resolved to a playable stream entry point and often contains CDN, token, and path information.

3.3.3 Tokens, cookies, referers, and provider checks

Streaming manifests are frequently protected by short-lived tokens, cookies, referer checks, or provider-side gates. A URL such as `playlist.m3u8?token=...` may work only for a few minutes and only from the browser session that generated it. CDNs and fronting providers such as Cloudflare can also enforce challenge pages, bot checks, cache rules, and origin protection [17].

This is why the browser session matters. A raw HTTP request may fail even when the browser can play the stream, because the browser carries the active cookies, headers, referer, and JavaScript-generated state. The technical background therefore includes both media protocols and web-provider behavior.

3.4 Browser Automation, Scraping, and Runtime Obstacles

Static scraping means downloading HTML and parsing it. That is useful for simple pages, but it is not enough for pages that depend on JavaScript, user clicks, iframes, or runtime media requests. Browser automation tools such as Puppeteer and Playwright execute pages in real browsers and expose APIs for clicking, reading the DOM, taking screenshots, and listening to network activity [4, 5, 6].

3.4.1 Static collection versus rendered browser state

Static collection is efficient when the target evidence is present in the fetched HTML or in simple linked pages. Its limitation appears when the evidence is created only after a runtime sequence: opening a collapsed channel group, clicking a server tab, activating the player, waiting

for an iframe, and then observing the network. In those cases, the browser state is the evidence surface, not only the page source. This distinction is the technical reason why OWC adds browser agents to static monitoring instead of treating the first page source as sufficient.

3.4.1.1 Rendered state as evidence

Rendered state includes visible text, player overlays, frame trees, screenshots, media elements, and network requests that appear after interaction. It matters because the final stream or channel signal may not exist at the first page load. A screenshot, media-state record, or tool trace can therefore explain why an agent believed the page exposed unauthorized streaming behavior.

3.4.2 Puppeteer as a browser control layer

Puppeteer provides programmatic control over Chromium. In this project context, the important concept is not only “click a button” but full runtime observation:

- inspect the DOM after JavaScript has executed;
- click buttons, tabs, overlays, server selectors, and play controls;
- traverse frames and iframes;
- capture screenshots for human review;
- observe network requests through the Chrome DevTools Protocol;
- read browser-side state through page-evaluation calls.

Puppeteer is therefore the bridge between a high-level extraction objective and the actual state of the web page. It lets the system test what a user would see and what the browser actually requested.

3.4.3 Ads, cookies, popups, and other annoyances

Illegal streaming websites often monetize through aggressive ad behavior. These behaviors are not just inconvenient; they change the extraction problem because the automation may click the wrong layer, follow an ad redirect, or lose the original player context.

Ad blocking tools such as uBlock Origin can reduce noise by blocking known ad and tracking domains [20]. However, they cannot replace browser reasoning. If a site serves ads and player assets from the same domain, aggressive blocking can break the player itself. The general solution is layered: block obvious noise, keep popup recovery, and preserve evidence when the page remains blocked.

3.4.4 Why screenshots and network logs matter

DOM text alone does not prove that a player actually loaded. A screenshot can show that the player was visible or that a challenge blocked the page. A network log can show that the browser requested a media manifest. Together, DOM, screenshot, and network evidence give a more reliable picture than any one source by itself.

Obstacle	Common symptom	General handling idea
Popups and new tabs	clicking play opens an ad page instead of the player	close extra targets and retry on the original page
Overlay layers	a fake play button covers the real player	detect visible overlays and click or remove them carefully
Cookie banners	controls are blocked until consent is handled	accept, dismiss, or record as blocking evidence
Ad iframes	DOM contains many unrelated frames	rank frames by player-like signals instead of following every iframe
Cloudflare-like challenges	browser sees an interstitial instead of the page	wait, retry with normal pacing, or preserve a blocked-state result

Table 3.3: Browser-level obstacles and handling strategies.

This is also why extraction tools should avoid claiming success only because a keyword or button was found. In this domain, useful evidence is the combination of page role, visible state, player behavior, and media request.

3.5 Agentic Evidence Collection Concepts

The project uses the term *agentic* to describe an extraction approach where a language-model-driven agent observes a page, chooses a tool action, checks the result, and repeats until it has enough evidence or reaches a limit. This differs from a fixed scraper because the next action depends on the current browser state.

3.5.1 Agents and ReAct-style loops

Agentic systems commonly use a loop that combines reasoning and tool use. In a ReAct-style pattern, the agent observes the state, reasons about what to do, acts through a tool, and verifies the result before continuing. In the implementation style used by OWC, this loop is represented through model messages, tool calls, observations, and structured outputs exposed by the agent framework [11].

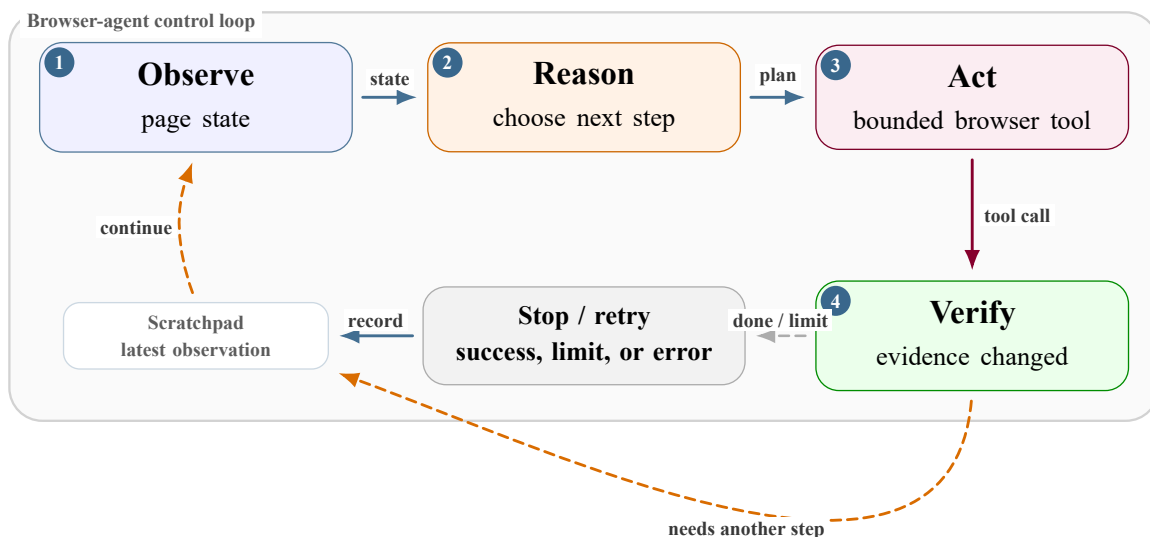


Figure 3.6: Observe, act, and verify loop used by browser agents.

For streaming sites, this loop is useful because the agent can adapt when a server button fails, a popup opens, an iframe appears, or a page turns out to be unrelated. The agent is not expected to know the full path in advance; it uses tools to discover it.

3.5.2 Prompt engineering, tokens, and token usage

Prompt engineering is the process of defining the agent's role, constraints, tool usage rules, evidence requirements, and output format. In this project domain, a prompt must prevent premature conclusions. It should tell the agent to ground decisions in visible page state, DOM evidence, screenshots, and network observations.

Tokens are the units used by language models to process prompts and outputs. Long page snapshots, repeated tool results, and verbose traces increase token usage and therefore increase cost and latency. Token management is part of the technical background because browser agents can easily spend too much context on noisy HTML, ads, or repeated page states.

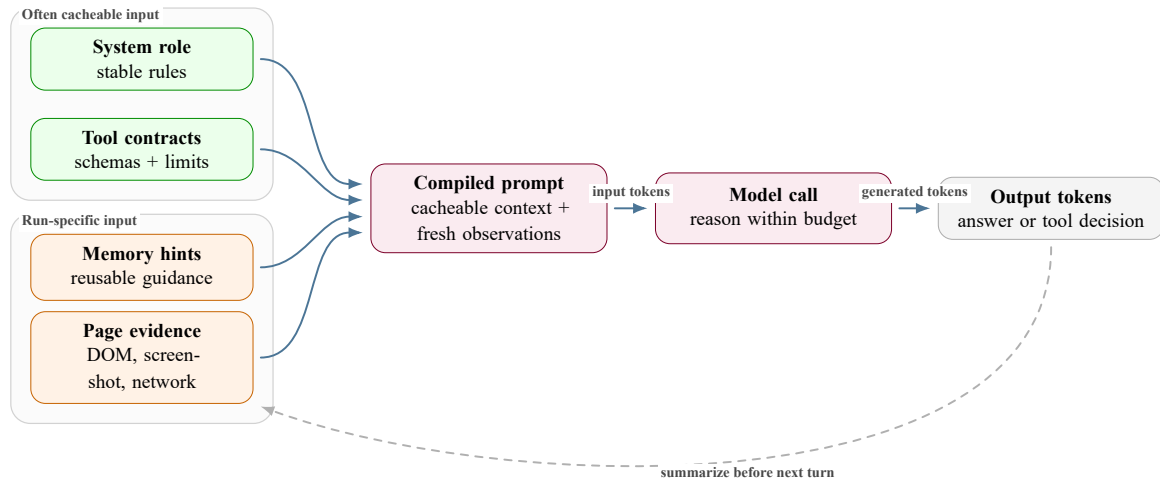


Figure 3.7: Prompt and token-budget composition for an evidence-driven browser agent.

Practical token-control ideas include:

- summarize page observations instead of passing full raw HTML every time;
- keep only evidence-relevant DOM snippets and network URLs;
- stop early when the page role or stream evidence is clear;
- separate input tokens, output tokens, and cached tokens when estimating cost;
- use provider-side caching when stable prompt sections are reused.

LLM providers such as Gemini expose model features that affect cost, context length, streaming behavior, pricing, caching, and rate limits [2, 3, 1]. Provider caching is especially relevant when the same system instructions, tool descriptions, or long context blocks are reused across many runs.

3.5.3 Workflow automation tools

Workflow automation tools let developers connect nodes for HTTP requests, browser actions, LLM calls, data transformations, and external services through a visual flow. They are useful for fast validation because a workflow can be assembled and adjusted before a full application is built. In this report, workflow automation is treated as an orchestration concept: it sequences steps, passes data between nodes, calls model providers, and makes evidence-collection logic visible during experimentation. The specific workflow tool used during implementation is introduced in Chapter 5.

3.5.4 LangChain, tools, and MCP servers

LangChain is a framework for building applications around language models, prompts, tools, memory, and structured chains. At a general level, it helps connect an LLM to external capabilities: a browser tool, a database lookup, a parser, a memory store, or a custom function. This is useful when the model should not only answer textually but take controlled actions [11].

The Model Context Protocol (MCP) is a way to expose external tools and resources to an agent through a consistent interface. In this type of project, an MCP server can expose browser tools such as inspect, click, navigate, screenshot, harvest, or media-state checks. The agent does not need to know how each tool is implemented internally; it needs a clear tool contract, input schema, and output shape [13].

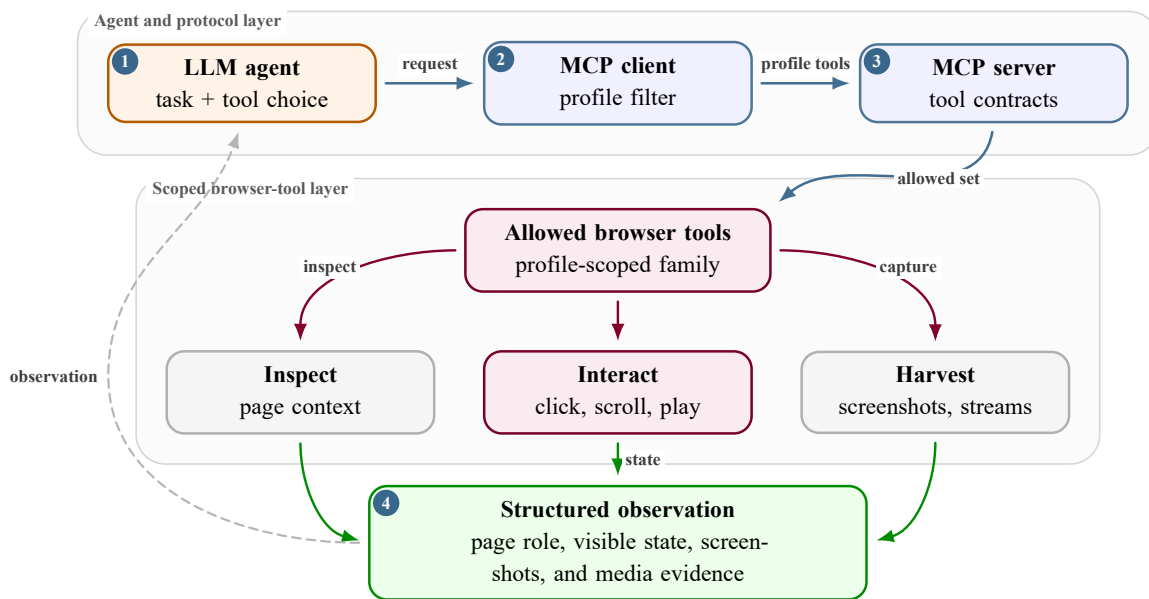


Figure 3.8: General MCP pattern connecting an LLM agent to scoped browser tools.

The important concept is separation of responsibility. The LLM decides what to try next, while tools perform bounded actions and return structured evidence. This keeps the agentic process auditable.

3.6 Chapter Summary

This chapter established the technical background for the rest of the report. It explained the page flow from landing pages to hosting pages and embedded players, the DOM structures used to inspect those pages, the streaming protocols and player embeds that carry the final evidence, and the browser-level obstacles caused by ads, cookies, popups, iframes, and provider-side checks. It also introduced the agentic concepts used later: ReAct-style loops, prompt engineering, tokens, provider caching, workflow automation, LangChain, and MCP tool servers.

The next chapters use these concepts more concretely. Chapter 4 translates them into architecture, and Chapter 5 explains how the implementation applies them in the actual evidence-collection system.

System Architecture

This chapter presents the architecture of Open Web Catcher (OWC), the evidence-collection platform developed during the internship. It starts with the user-facing system and runtime components, then narrows into the multi-agent pipeline, the handoff contract between agents, the responsibilities of each agent, and the browser-tool layer that lets agents read and act on rendered pages. It closes with the data that makes a run reviewable. Chapter 5 then maps these architectural decisions to the implementation. In this chapter, OWC means the academic/reference platform designed during the internship: the backend, console, browser tooling, specialist agents, runtime telemetry, persistence, and review outputs.

Chapter focus

The system is a staged evidence pipeline, not a single unrestricted browser agent. FastAPI starts and records runs. The orchestrator owns the LangGraph route, memory lookup, queues, handoffs, aggregation, provider analysis, and email draft generation. Four browser-facing specialists perform bounded work: classification, landing-page discovery, hosting-page extraction, and embedded-player extraction. Each specialist receives a narrow task, uses only its profile-scoped MCP browser tools, returns typed output, and leaves events that explain both success and failure. Channel detection is handled by the agents as part of evidence interpretation, while the browser runtime carries the practical controls required by this domain: rendered-page context retrieval, screenshot capture, profile-scoped sessions, fingerprint handling, and proxy rotation.

4.1 System Purpose and Architecture Reading Path

The platform is an evidence-collection workbench for illegal streaming websites. The operator launches investigations and reviews the resulting evidence. The engineer or administrator maintains models, prompts, pricing, datasets, browser runtime settings, and MCP tool configuration. External services provide target website content, model inference, screenshot storage, and provider ownership data.

Open Web Catcher Console

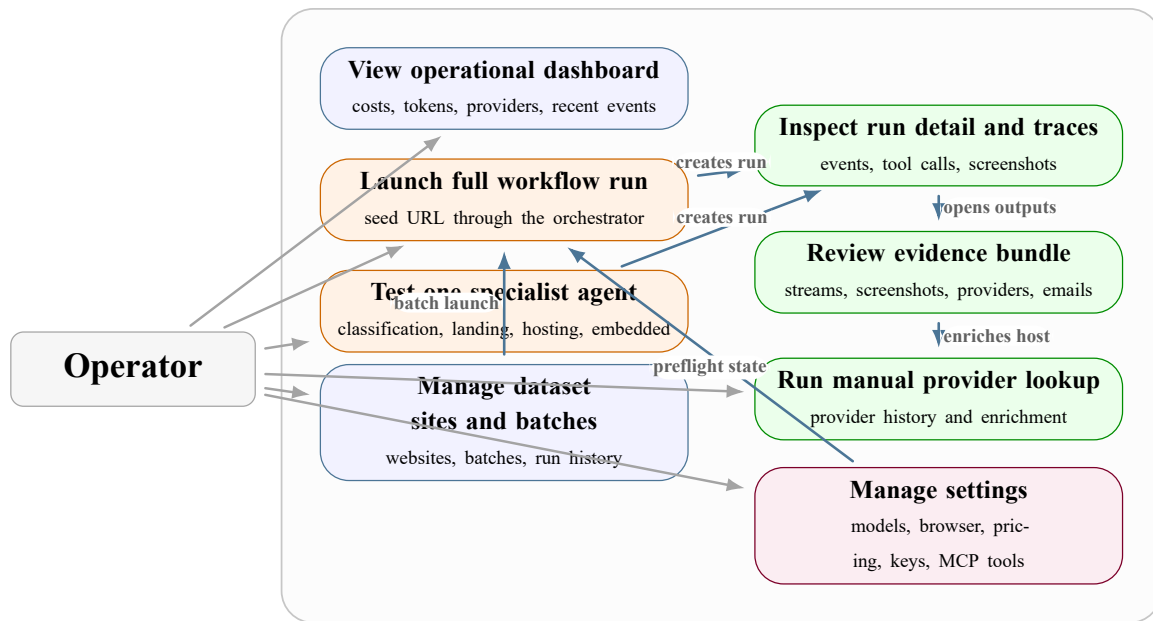


Figure 4.1: Use-case diagram for the active OWC console actions: launch, test, dataset management, trace inspection, evidence review, provider lookup, and configuration.

The architecture should be read in four layers:

1. **Product layer:** the operator console launches runs, monitors progress, and reviews evidence.
2. **Runtime layer:** FastAPI validates runtime readiness, schedules jobs, resolves settings, and persists trace data.
3. **Agent layer:** the orchestrator routes work through specialized agents instead of asking one prompt to do every task.
4. **Evidence layer:** streams, screenshots, provider rows, emails, events, model calls, and tool calls are normalized for later review.

Use-case group	User intent	System response
Launch investigation	Submit one URL, a single-agent test, or a dataset batch.	Create a traceable run, validate runtime readiness, and schedule execution.
Monitor execution	Follow agent transitions, tool calls, screenshots, events, and failures.	Stream active state and persist each important runtime event.
Review evidence	Inspect streams, screenshots, provider analysis, metrics, and takedown drafts.	Assemble a reviewable evidence bundle with final status and supporting trace data.
Configure system	Tune models, prompts, MCP tools, browser settings, pricing, and datasets.	Persist configuration and expose health or preflight feedback before runs start.

Table 4.1: Use-case responsibilities across operator and administration workflows.

4.2 Global Runtime Architecture

The runtime separates the operator console, API, agent execution, browser automation, persistence, and external enrichment services. The Next.js console does not access the database directly; it calls FastAPI endpoints. FastAPI owns background-job execution, active trace assembly, settings resolution, and database access. Browser work is delegated to profile-scoped MCP tooling so the agents receive only the tools allowed for their page type.

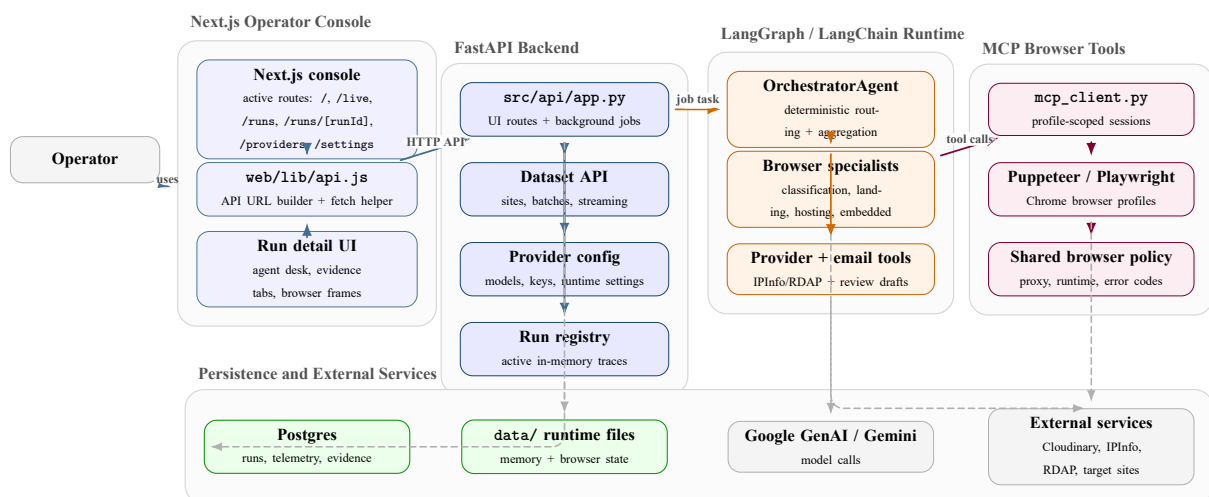


Figure 4.2: System component diagram grounded in the active console routes, FastAPI modules, agent runtime, browser tools, persistence, and external services.

The frontend route map is part of the architecture because the console is the operator's entry point into the runtime. The active screens are the dashboard, live pipeline, results, run detail, provider lookup, and settings pages; the /agents path is kept only as a redirect to /live.

Runtime area	Main responsibility	Why it is separate
Operator console	Launch workflow runs, inspect run detail, manage datasets, providers, tools, browser settings, and model configuration.	The UI stays thin; it consumes API payloads instead of owning persistence.
FastAPI backend	Validate preflight state, create jobs, run background work, assemble active and persisted traces, and expose review payloads.	Runtime control and database access remain testable from backend routes.
Agent runtime	Execute the orchestrator graph and the specialist model/tool loops.	Routing policy remains explicit instead of hidden inside one prompt.
MCP browser tooling	Provide profile-scoped page, frame, screenshot, interaction, and network tools.	Browser dependencies and page failures are isolated from the API process.
Persistence and services	Store normalized run records and call Gemini, Cloudinary, IPInfo, RDAP, and local runtime files.	The final answer and the failure path can be reviewed after execution.

Table 4.2: Global runtime boundaries used by OWC.

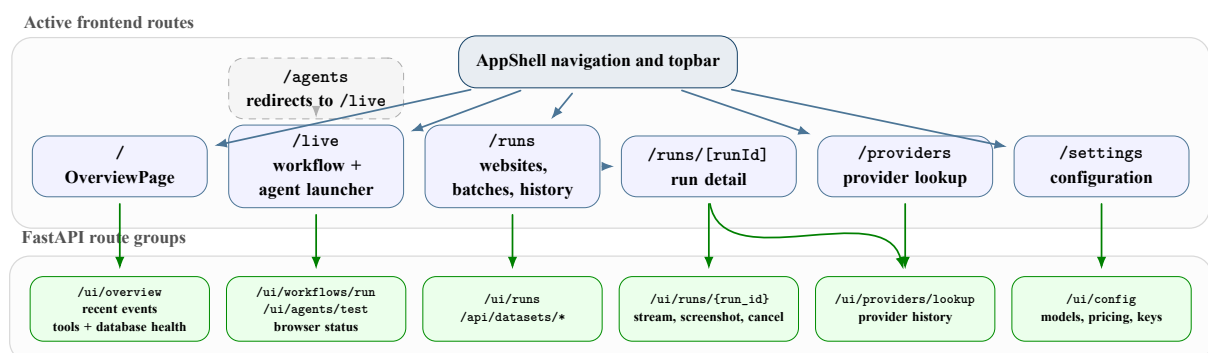


Figure 4.3: Frontend route and data-flow diagram showing active console screens, the /agents redirect, and the backend route groups consumed by each screen.

This separation is practical rather than cosmetic. Browser automation, LLM reasoning, dashboard rendering, and persistence fail in different ways. Keeping their boundaries visible makes failures easier to classify: a browser-tool timeout, a model retry, a missing stream, a skipped provider lookup, and a dashboard payload issue are different problems.

4.3 End-to-End Agent Pipeline

At large scale, the platform behaves as a controlled multi-agent pipeline, not as one unrestricted browser agent. The operator gives the system a seed URL. The API starts a run, creates an active trace, and calls the orchestrator. The orchestrator checks memory as soft guidance, calls classification, routes to the appropriate specialist path, collects evidence, and only then runs provider analysis and email generation.

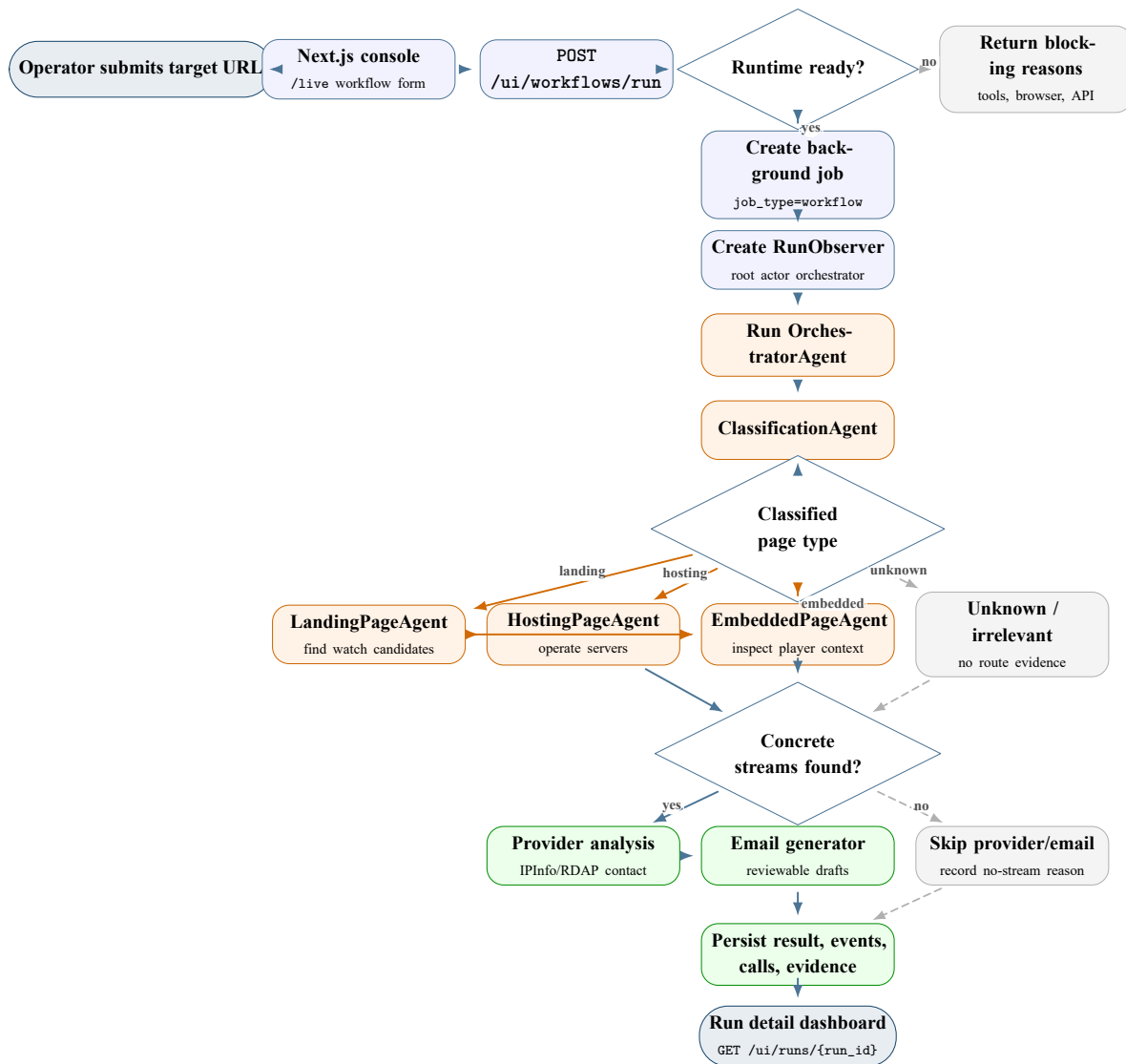


Figure 4.4: End-to-end flowchart from workflow launch to persisted run-detail evidence, including runtime preflight, routing, conditional provider analysis, and no-stream handling.

The flowchart shows the operational path and the blocking decisions. The next map expands the same route as an agent architecture, showing the specialist stages, shared browser runtime, downstream provider analysis, and final pipeline result.

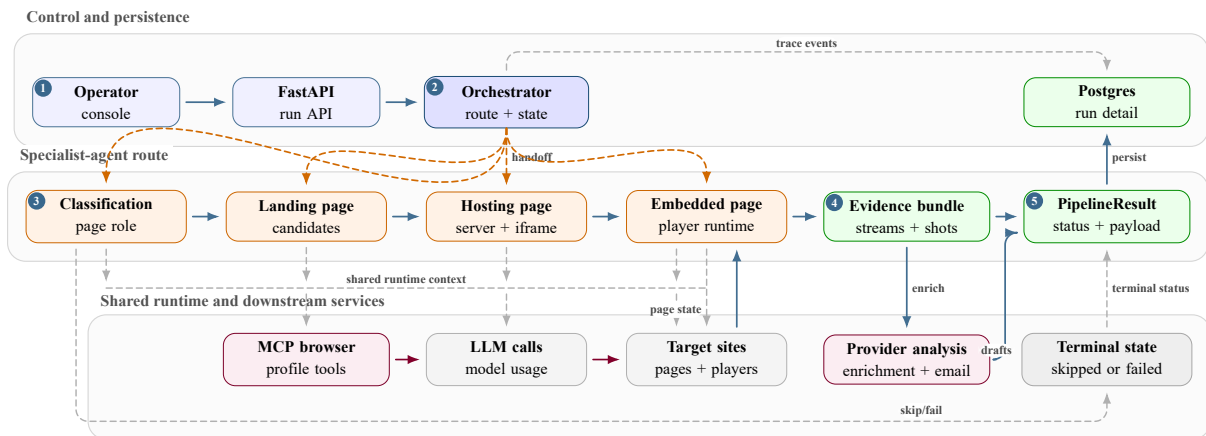


Figure 4.5: End-to-end agent architecture at workflow scale.

The sequence view shows the same workflow as a runtime contract. A run starts as an API request, becomes a background job and active trace, executes through the orchestrator graph, records model and tool activity, and is finally returned to the console as normalized run-detail data.

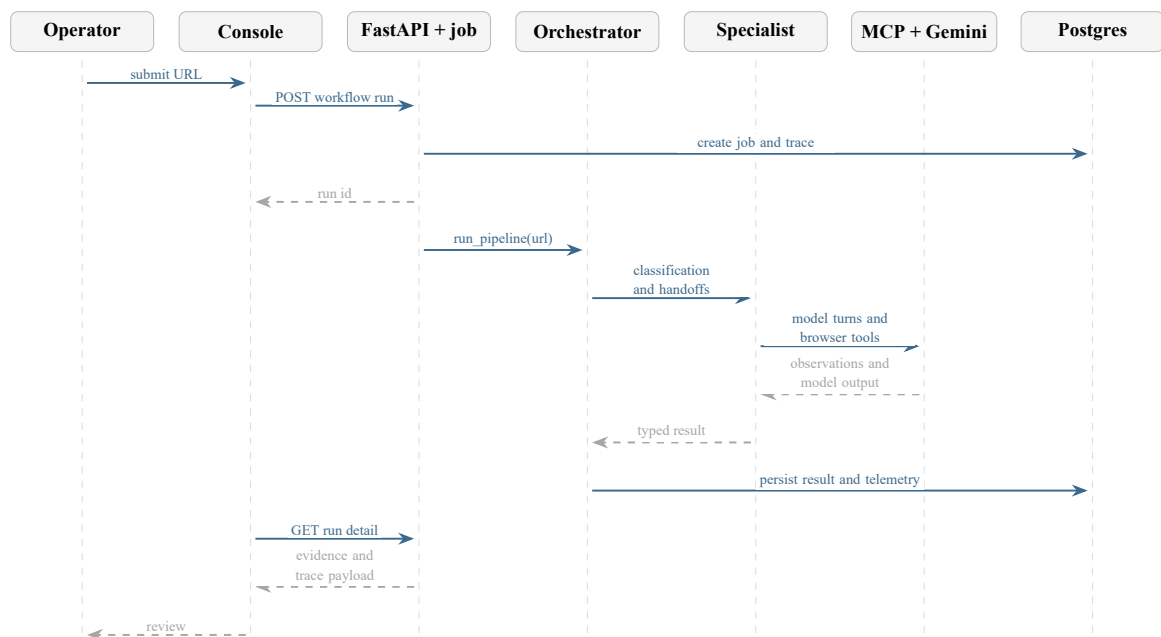


Figure 4.6: Sequence diagram for launching, executing, persisting, and reviewing a workflow run.

The pipeline has three important rules:

- **Agents do not call each other directly:** every specialist returns typed output to the orchestrator.
- **The orchestrator owns queues:** landing can add hosting or embedded targets, hosting can add more hosting or embedded targets, and embedded returns final media evidence.
- **Downstream stages require streams:** provider analysis and email generation are skipped when no stream URL exists.

4.4 Agent Communication and Orchestrator Handoffs

The orchestrator is the routing authority. It builds the LangGraph state machine, checks memory as soft guidance, calls classification, prepares handoffs, executes landing, hosting, and embedded specialists as needed, aggregates structured outputs, then runs provider analysis and email generation only when stream evidence exists. This keeps failures interpretable: a classification timeout, missing hosting candidate, hosting-player failure, embedded-player failure, and provider skip are different operational states.

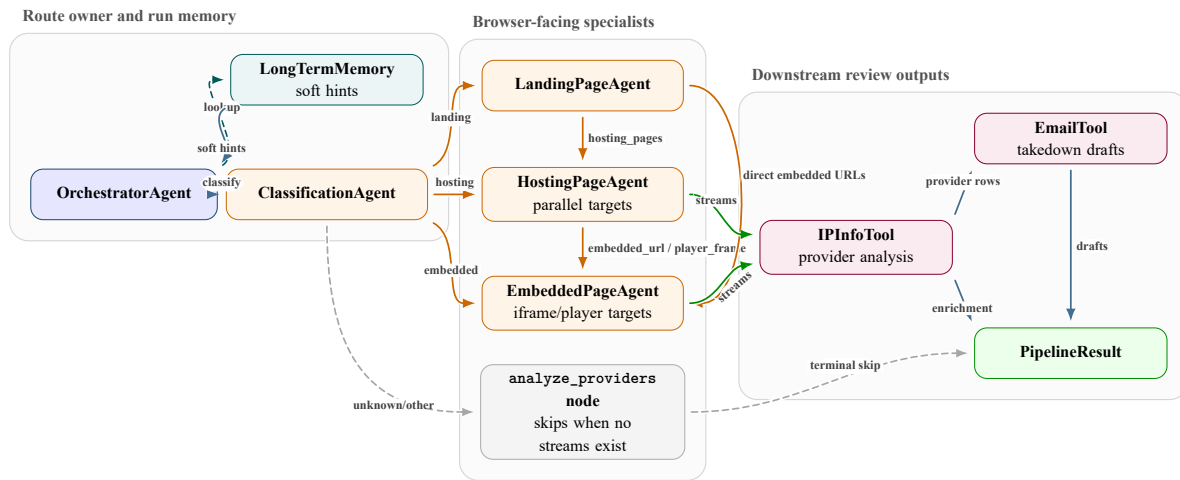


Figure 4.7: Agent interconnection graph showing orchestrator-owned routing and specialist responsibilities.

The handoff is the main communication contract between agents. It contains the root URL, target URL, page type, classification reasoning, route source, memory hints, navigation policy, required evidence, and known pattern context. A specialist receives this context inside its task brief, performs only its bounded job, and returns a schema that the orchestrator can normalize and route.

In this design, communication is visible at three levels. First, route decisions are graph edges, so the next stage is explicit. Second, handoff text gives each specialist the local context it needs without making it own the whole pipeline. Third, the observer records agent starts, model calls, tool calls, screenshots, status changes, costs, and final rollups so the dashboard can explain what happened.

Prompt compilation is also part of the communication contract. Each browser-facing specialist receives the same architectural ingredients but a different stage contract. The compiled prompt combines the base policy, the agent-specific contract, the URL task brief, runtime context, long-memory hints, and short-memory working state. Short memory is run-local: it tracks recent navigation, tool calls, selectors, candidates, streams, server records, screenshots, and the next working objective. Long memory is cross-run site memory: it stores summarized playbooks in the site-memory database and profile JSON so a future run can receive hints without trusting them blindly. Both are soft context; live browser evidence still decides the route.

Stage	Main input	Responsibility	Main output
Orchestrator	Seed URL, settings, observer, and memory hints.	Build the LangGraph route, prepare handoffs, aggregate results, compute final status, and call downstream tools.	PipelineResult, trace events, provider rows, and email drafts.
Classification	URL and profile-scoped browser context tools.	Decide page type without extracting streams or provider data.	Classification result with page type, confidence, and reasoning.
Landing	Landing URL plus orchestrator handoff.	Discover watch or hosting candidates while preserving match, iframe, channel, and player hints.	ExtractionResult with match records and hosting or embedded candidates.
Hosting	Assigned hosting URL plus evidence checklist and navigation policy.	Operate server/source controls, activate playback, collect network evidence, and return explicit embedded handoffs when needed.	Screenshots, server rows, stream URLs, diagnostics, or embedded player URLs.
Embedded	Verified player or iframe URL plus handoff context.	Stay inside the assigned player context, recover from drift, and harvest final stream evidence.	Stream URLs, screenshots, frame diagnostics, and player state.
Provider and email tools	Stream URLs, provider rows, screenshots, and extractions.	Enrich stream hosts and draft reviewable takedown notices.	ProviderInfo records and TakedownEmail drafts.

Table 4.3: Agent and tool responsibility map.

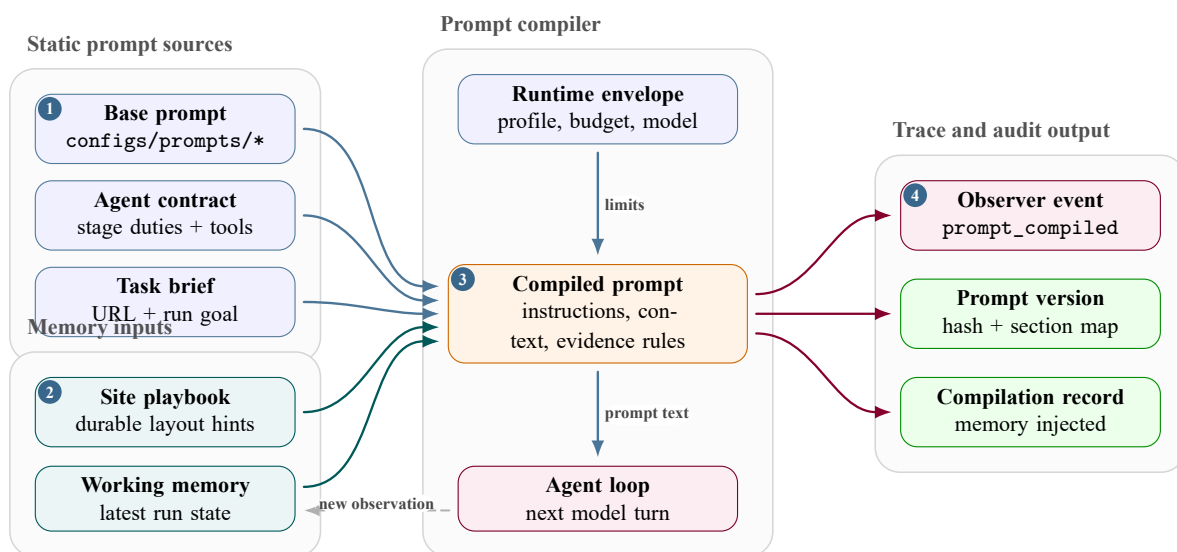


Figure 4.8: Prompt compilation and memory inputs for each browser-facing specialist.

The browser-facing specialists then reuse the same inner loop. Each one compiles its prompt, opens the correct MCP browser profile, lets the model request tools, records every model and tool turn, controls context growth, and normalizes the final answer into the stage schema.

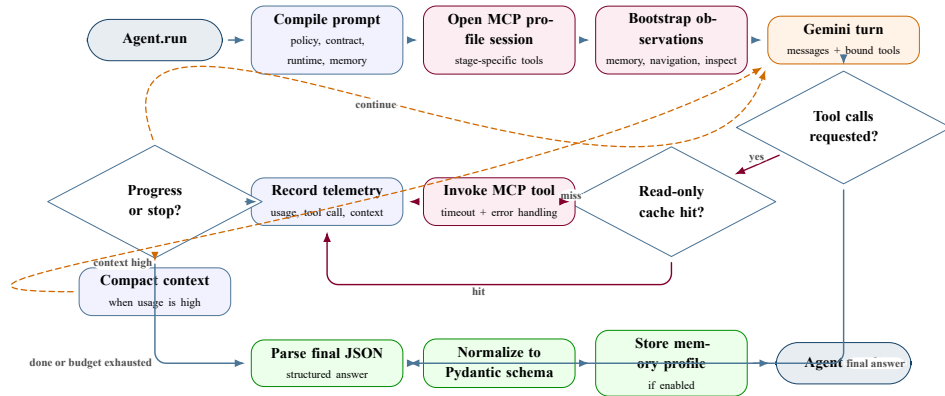


Figure 4.9: Shared activity loop used by the browser-facing agents: prompt compilation, MCP session setup, model/tool turns, telemetry recording, context control, schema normalization, and optional memory writing.

4.5 Agent-by-Agent Design

4.5.1 Orchestrator Agent

The orchestrator is the control plane for one run. It owns state, queues, handoffs, aggregation, provider and email execution, final status, and trace emission, but it does not inspect pages directly.

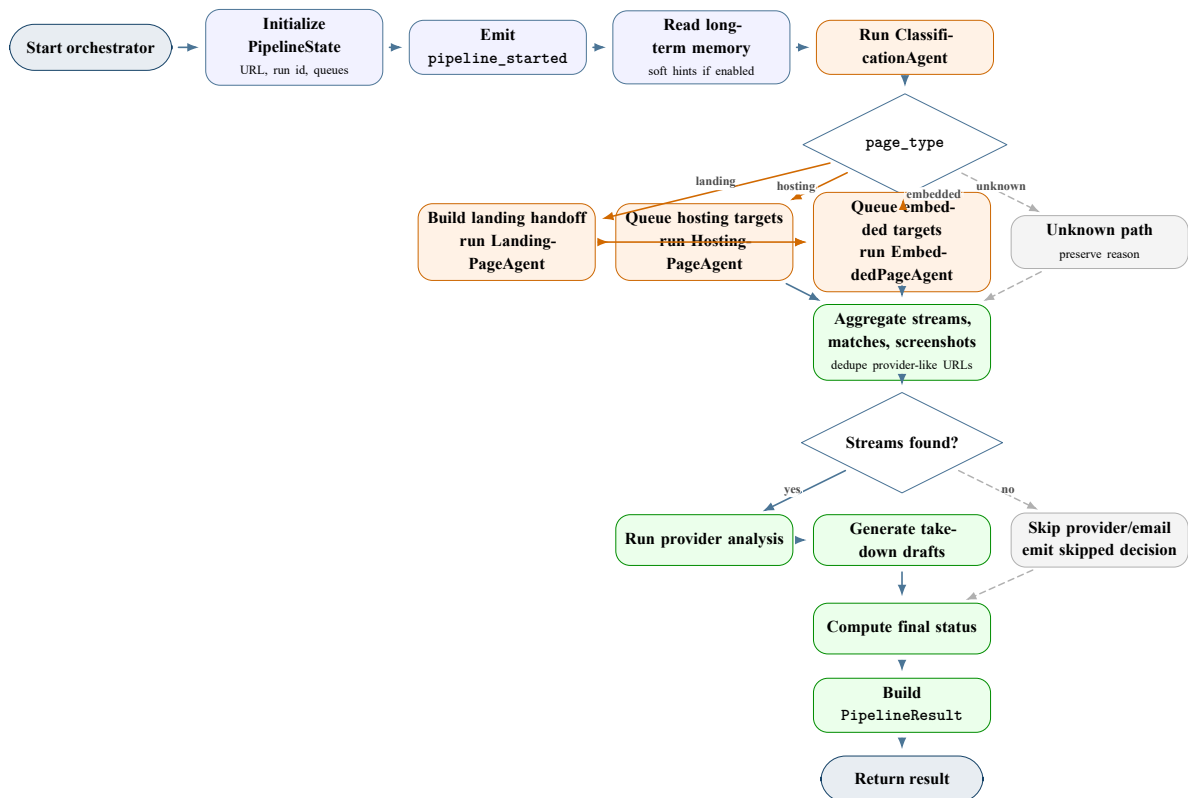


Figure 4.10: Orchestrator activity diagram showing initialization, classification routing, specialist queues, aggregation, provider/email decisions, and final PipelineResult construction.

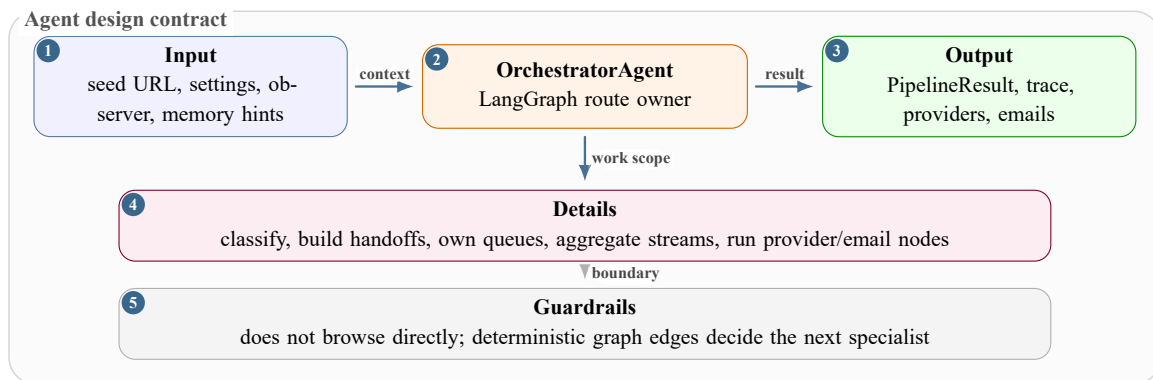


Figure 4.11: Orchestrator-agent design card.

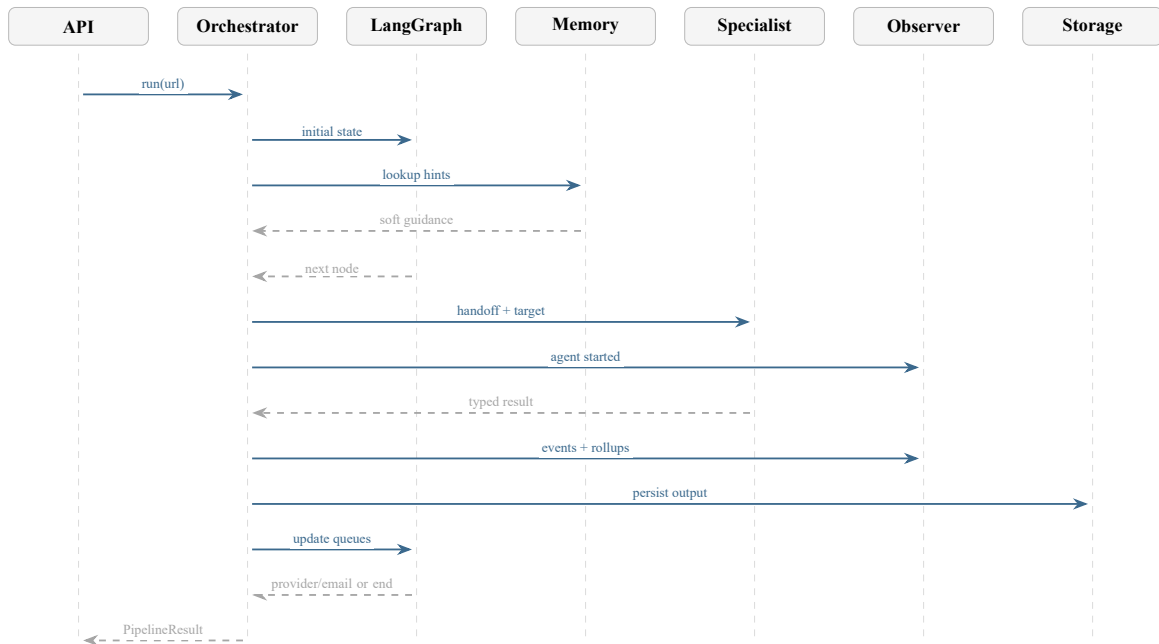


Figure 4.12: Sequence diagram for orchestrator handoff, trace recording, and route updates.

4.5.2 Classification Agent

The classification agent answers one question: what type of page is this URL? It distinguishes landing, hosting, embedded, unknown, and irrelevant pages before extraction begins.

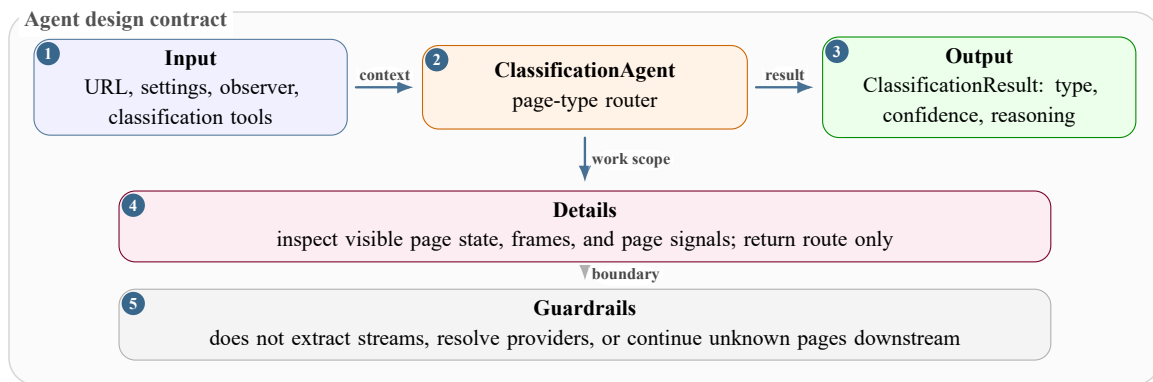


Figure 4.13: Classification-agent design card.

A landing result sends the run to the landing agent. A hosting result queues the root URL for the hosting agent. An embedded result queues the root URL for the embedded agent only when the page looks like a direct player. Unknown or irrelevant pages skip provider and email stages because no stream evidence exists.

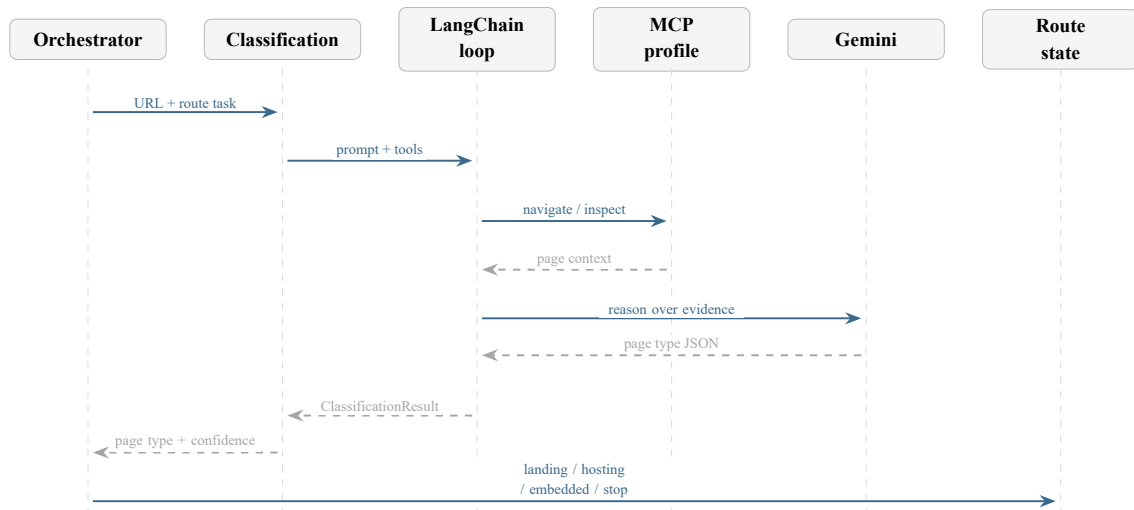


Figure 4.14: Classification-agent sequence: inspect page state and return a routing decision.

4.5.3 Landing-Page Agent

The landing-page agent turns broad site pages into bounded downstream targets. It discovers useful watch, hosting, embedded, or direct-stream candidates from homepages, schedule pages, directory pages, and match listings.

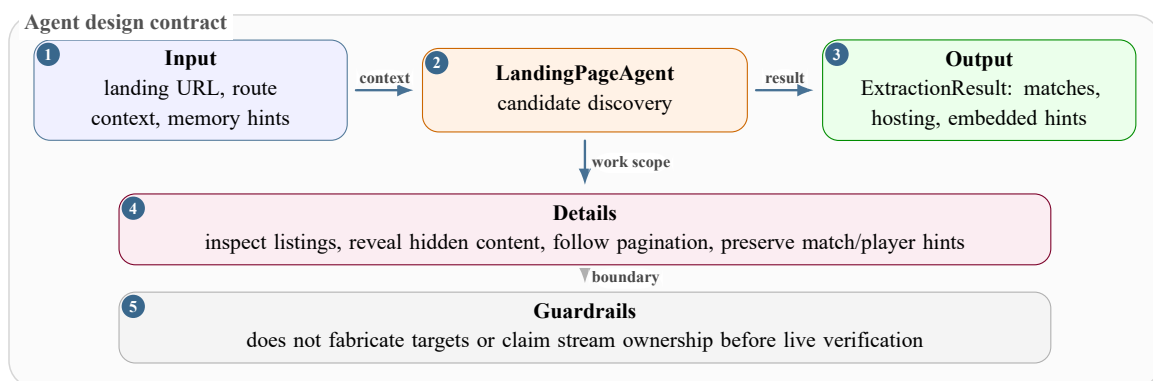


Figure 4.15: Landing-page-agent design card.

The orchestrator deduplicates landing outputs. Normal watch or server pages go to the hosting queue, direct player URLs go to the embedded queue, and provider-like media URLs are preserved as stream evidence.

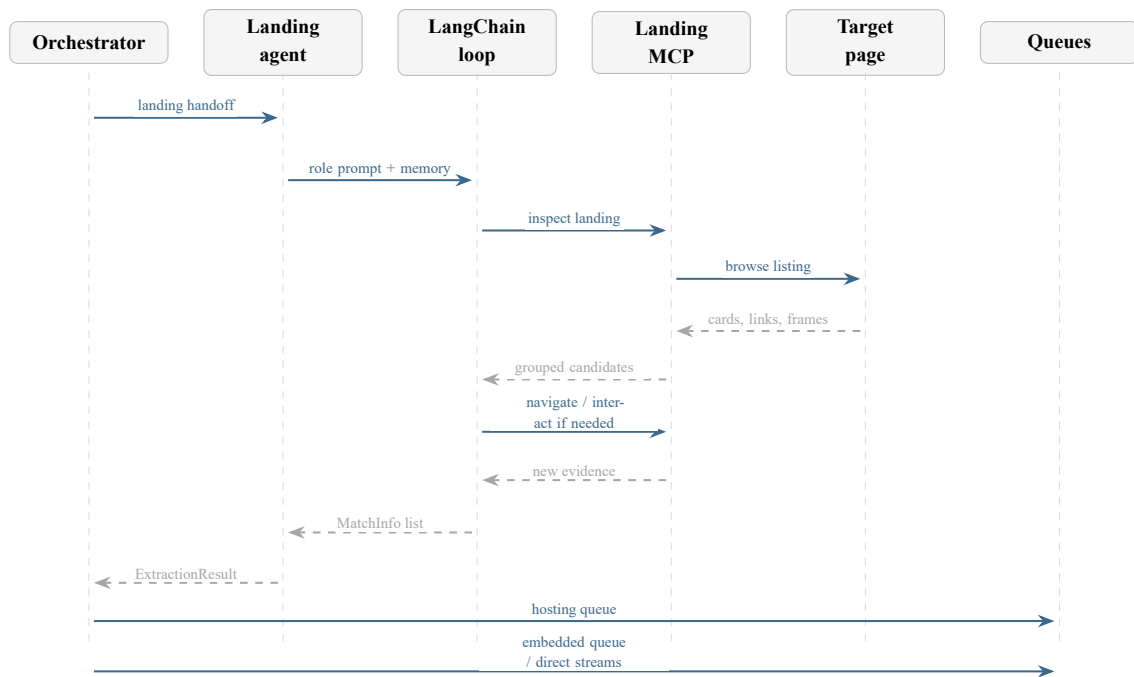


Figure 4.16: Landing-agent sequence: convert listing-page evidence into downstream queues.

4.5.4 Hosting-Page Agent

The hosting-page agent owns watch pages and server/source selection pages. It activates players, operates server controls, handles overlays, captures screenshots, and returns streams or explicit embedded-player handoffs.

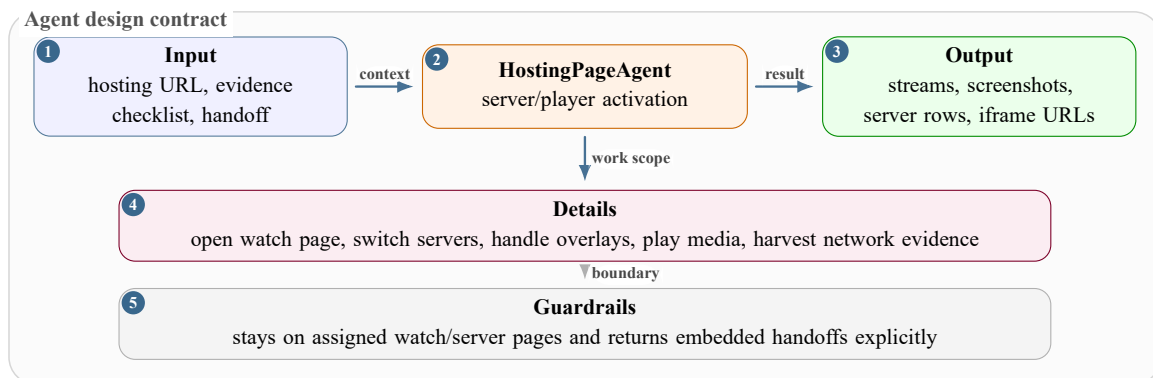


Figure 4.17: Hosting-page-agent design card.

If the page exposes an iframe or direct player URL instead of a stream, the hosting result carries that URL as an embedded handoff. If playback fails, it still returns diagnostics so the run explains why extraction stopped.

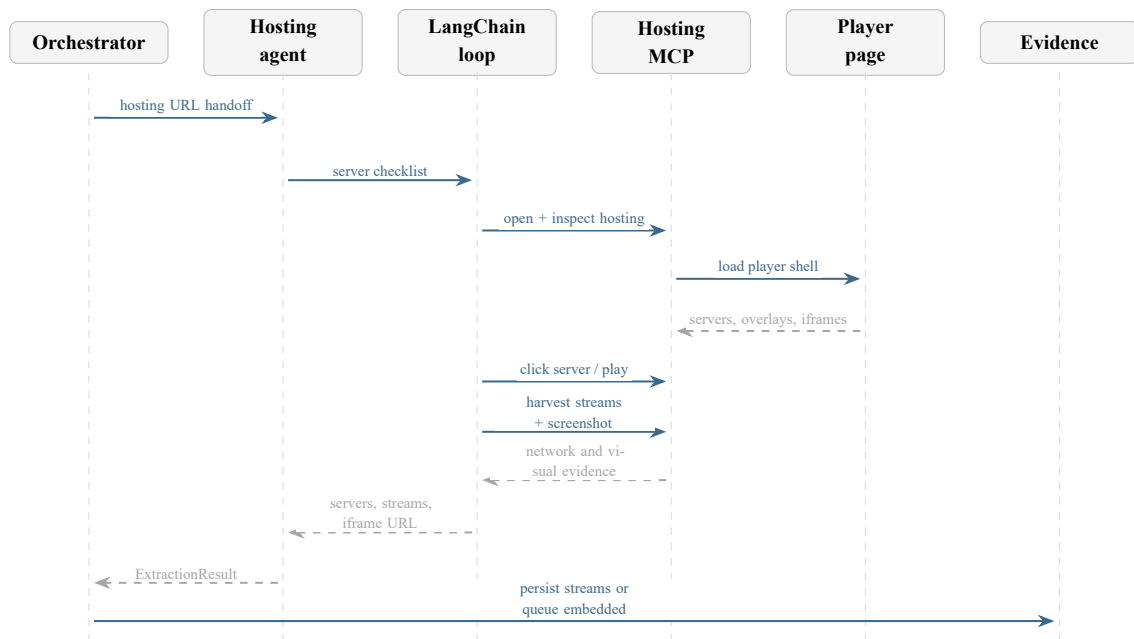


Figure 4.18: Hosting-agent sequence: activate watch pages and return stream or iframe evidence.

4.5.5 Embedded-Page Agent

The embedded-page agent owns direct player or iframe URLs. It stays inside the assigned player context, inspects frames, activates iframe-local controls, recovers from popup or redirect drift, and collects final media evidence.

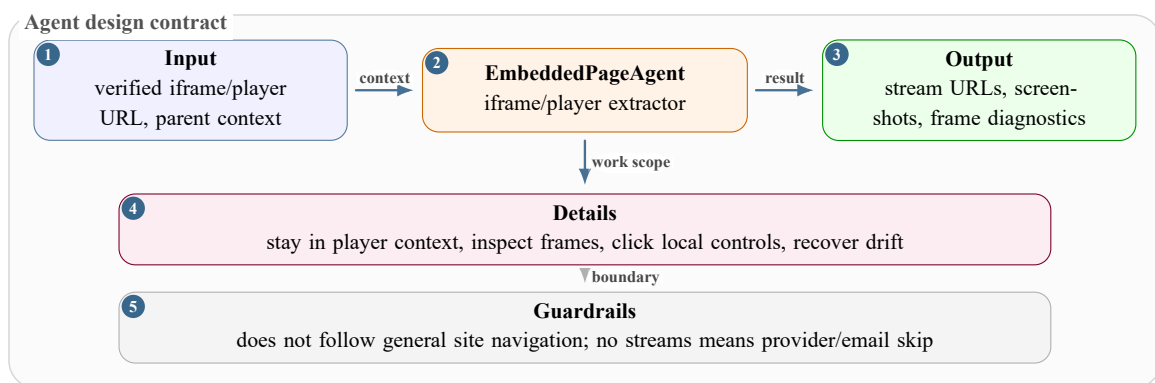


Figure 4.19: Embedded-page-agent design card.

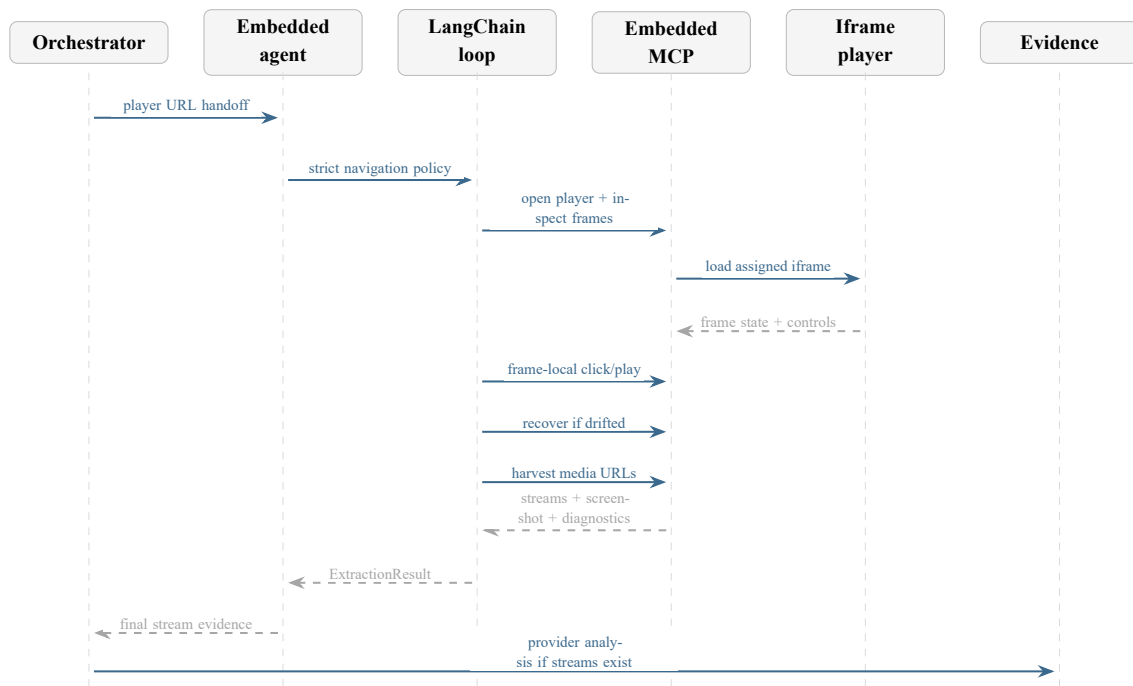


Figure 4.20: Embedded-agent sequence: stay in the player context and harvest final media evidence.

4.5.6 Channel Detection Across Agents

Channel detection is an agent responsibility, not a standalone browser tool. The tools expose page text, DOM structure, frame state, screenshots, server labels, and media requests; the agents decide whether those signals are strong enough to become evidence. This distinction matters because a channel name printed on a landing page is only a hint, while a channel label near an activated player or inside the embedded player context is stronger evidence.

The responsibility is shared across the route. Classification can preserve obvious channel text from the first page observation. The landing agent keeps channel and match hints attached to candidate hosting URLs. The hosting agent verifies whether the selected server or player still corresponds to the same channel. The embedded agent has the final opportunity to confirm, refine, or reject the channel signal from iframe-local text, player overlays, and screenshot evidence. The orchestrator then normalizes these observations as candidates, final channel fields, and confidence notes in the pipeline result.

4.5.7 Provider Analysis and Email Generation

Provider analysis and email generation are downstream evidence-packaging stages. They are not page-navigation specialists. Provider analysis reads the collected stream URLs, resolves host or IP ownership, and returns provider information and abuse contact data. Email generation groups provider rows and stream evidence into reviewable takedown drafts. These drafts are not sent automatically; they are artifacts for human review.

Agent stage	Channel evidence it can read	How the signal is treated
Classification	Page title, visible headings, URL text, and broad page context.	Preserved as an initial hint, not accepted as final proof.
Landing	Match cards, league labels, event rows, revealed tabs, and candidate hosting links.	Attached to each downstream candidate so the route keeps its sports context.
Hosting	Server labels, player container text, active source state, screenshots, and network-adjacent player metadata.	Used to verify or override landing hints after the watch page is activated.
Embedded	Iframe-local DOM, player overlays, frame title, screenshot text, and final media evidence context.	Treated as the strongest page-level confirmation before provider analysis.

Table 4.4: Channel detection responsibilities across the specialist agents.

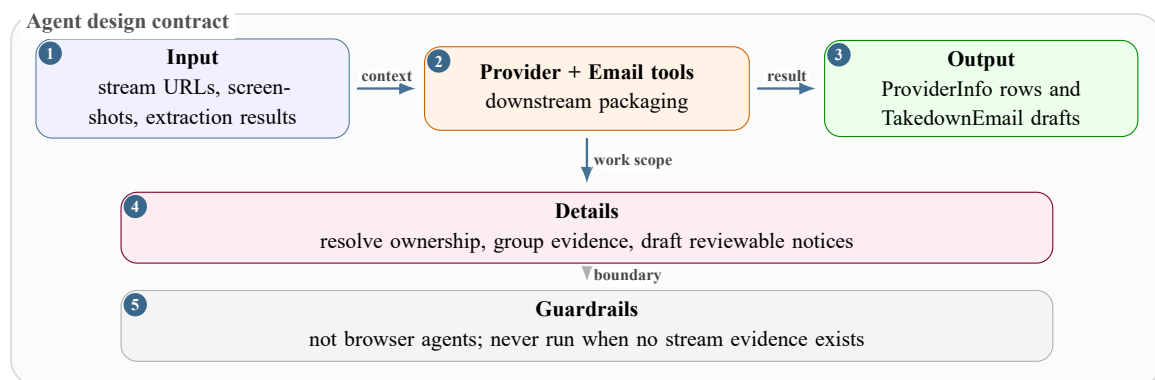


Figure 4.21: Provider-analysis and email-generation design card.

4.6 Browser Tooling and Runtime Controls

After the agent responsibilities are clear, the browser-tool layer explains how the agents observe and change real web pages. OWC does not ask an agent to reason from raw HTML alone. Illegal streaming pages are usually rendered by JavaScript, split across nested frames, and guarded by overlays, popups, server tabs, tokenized URLs, or delayed media requests. For that reason, every browser-facing specialist receives a profile-scoped MCP tool surface backed by Puppeteer and Chrome.

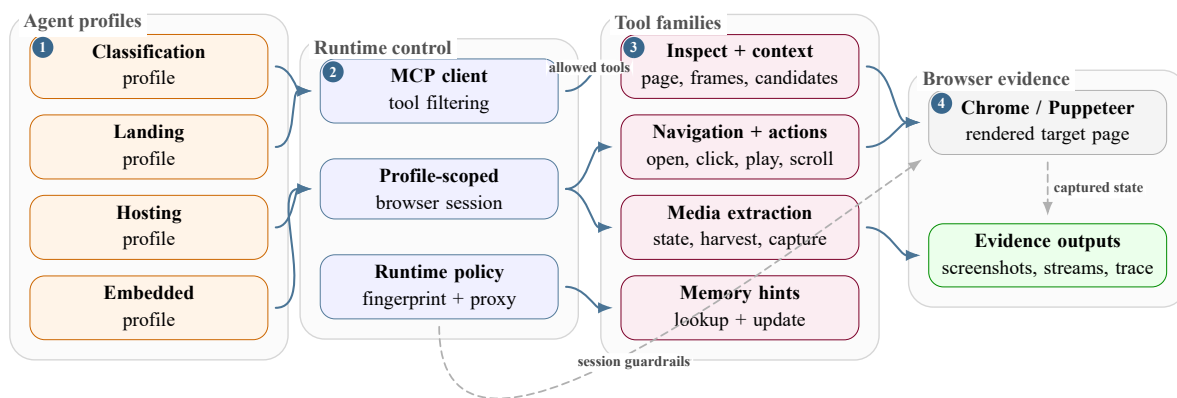


Figure 4.22: Profile-scoped MCP browser tooling and runtime controls used by OWC agents.

4.6.1 Tool Families

The Puppeteer tool layer is divided into tool families so that each agent can ask for the smallest useful observation or action. This keeps context growth under control: the model receives a compact view of the rendered page, then requests a narrower element, frame, media, or screenshot operation when it needs proof.

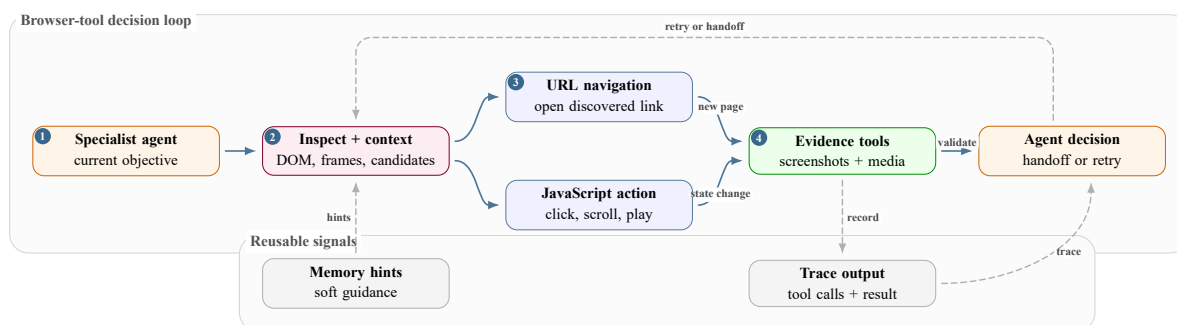


Figure 4.23: Global browser-tool flow. The agent inspects page context, selects either URL navigation or JavaScript interaction, validates the resulting evidence, and records reusable trace and memory signals.

4.6.1.1 Inspection and context retrieval

Inspection tools are the first read layer. They inspect the rendered Document Object Model (DOM), not only the first server response. Broad inspectors summarize visible text, grouped anchors, buttons, candidate cards, server controls, iframe trees, player containers, media clues, and screenshot links. Page-type inspectors specialize that same idea: the landing inspector looks for match cards and hosting candidates, the hosting inspector looks for server/source controls and player state, and the embedded inspector focuses on iframe-local player evidence.

Context tools are narrower. When an agent sees a possible target in a broad observation, it can request element details, a selector-specific subtree, a frame tree, candidate element references,

or media state. This avoids repeatedly sending a large DOM summary back to the model. The architectural rule is simple: broad context is used to choose where to look, and scoped context is used before acting.

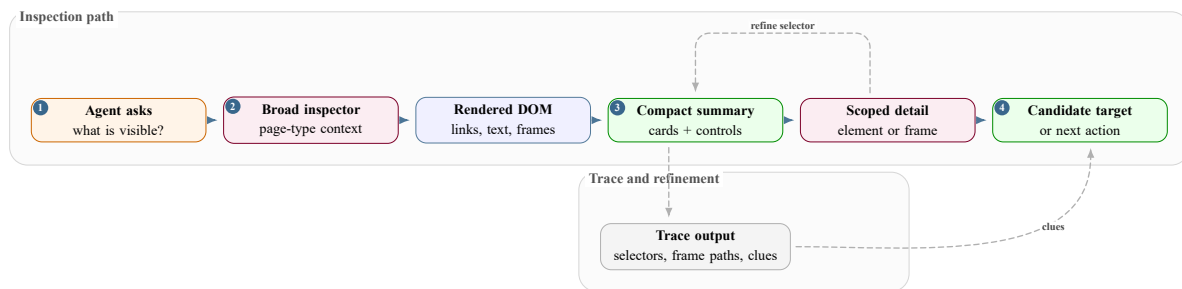


Figure 4.24: Inspection and context-retrieval flow from broad page observation to scoped targets.

4.6.1.2 Navigation and interaction

Navigation has two different meanings in OWC. When inspection returns a real link or candidate URL, the agent uses URL navigation: open the link, wait for the document to settle, then inspect the new page. When the page requires JavaScript to reveal content, the agent stays in the same browser context and uses interaction tools: click a tab or server button, scroll to trigger lazy loading, type into an input, select a control, or attempt playback. Both paths must be followed by a new observation because either a new document or a changed DOM state can invalidate the previous context.

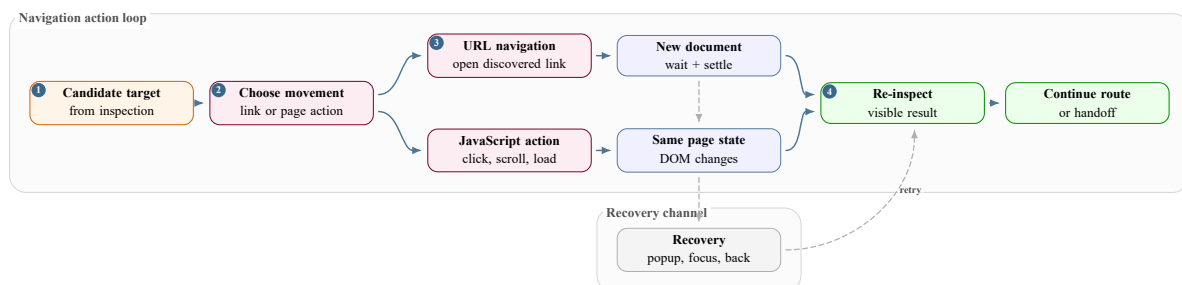


Figure 4.25: Navigation flow split between direct URL navigation and JavaScript-driven page interaction.

4.6.1.3 Evidence and media extraction

Evidence tools convert browser state into reviewable artifacts. Screenshots show what the page looked like after navigation or interaction. Media-state tools confirm whether a player is present, whether playback started, and whether the active frame changed. Stream-harvesting tools collect m3u8, mpd, mp4, or related candidates from network and browser signals. The agent still decides whether the candidate is acceptable evidence, but the tools provide the structured facts that make the decision auditable.

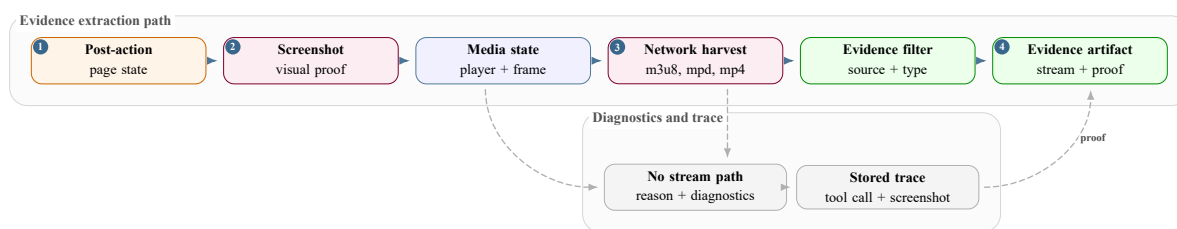


Figure 4.26: Evidence and media-extraction flow from post-action page state to reviewable artifacts.

4.6.1.4 Memory and site hints

Memory tools read and update reusable site hints such as known server tabs, useful selectors, common iframe patterns, and previous failure causes. They do not replace live browser evidence. A memory hint can tell an agent where to start, but the agent must still inspect the current page, act through Puppeteer if needed, and validate the result with screenshots, player state, or stream evidence.

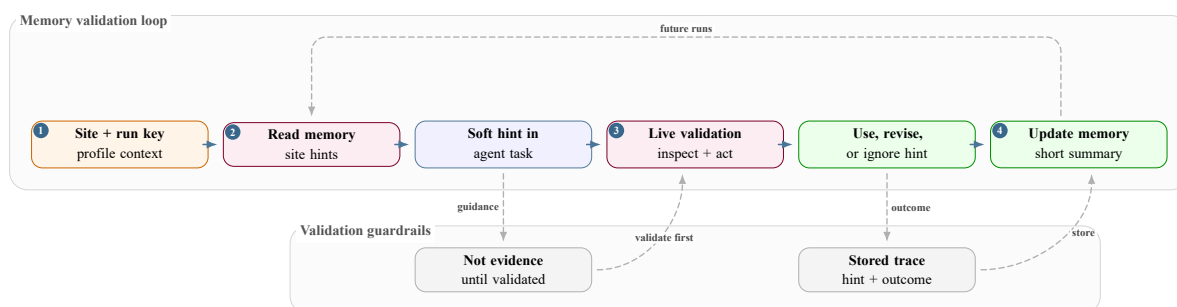


Figure 4.27: Memory-tool flow showing hints as guidance that must be validated live.

4.6.2 Fingerprint Handling and Proxy Rotation

OWC treats browser identity as a runtime setting rather than as prompt text. Browser profiles keep cookies, cache, and session state scoped to the current run or agent profile. Fingerprint handling controls properties such as browser identity, navigator surface, viewport-related behavior, and profile fallback so that repeated browser sessions do not expose an obviously inconsistent automation profile.

Proxy rotation is handled at runtime as a validated sticky strategy. Candidate proxies can be collected from configured remote or custom sources, validated against a test endpoint, and kept for the current browser profile while they remain healthy. Rotation occurs after failure instead of on every click, because changing network identity during one investigation can break cookies, session-bound player URLs, and tokenized media links. If no proxy candidate works, the runtime can fall back to direct browsing instead of incorrectly reporting the target site as unreachable.

Runtime concern	Architectural rule	Reviewable output
Tool scope	Each specialist receives only the tools needed for its page role.	Tool calls and profile identifiers attached to the run trace.
Browser profile	State is scoped by browser/profile so cookies, cache, and session-bound pages stay coherent during a run.	Profile, runtime settings, and browser events.
Fingerprint handling	Browser identity is hardened and rotated according to runtime policy, not improvised by the agent.	Runtime configuration and browser diagnostics.
Proxy rotation	Proxies are validated and kept sticky until failure; direct fallback avoids false site-down conclusions.	Proxy strategy, failures, and final navigation status.
Context retrieval	Agents request broad DOM summaries first, then scoped element, frame, or media details before acting.	Inspector output, selectors, frame paths, and observation events.

Table 4.5: Browser runtime controls that support reliable and reviewable evidence collection.

4.7 Data, Trace, and Reviewability View

The reviewability view is stored as normalized database records, not only as one large final JSON object. The central class is `PipelineRunRecord`. It stores the stable run identity, root URL, final status, aggregate counts, token totals, cost totals, and high-level classification fields. Background jobs can exist before a pipeline row is finalized, but once the run is persisted the agent, telemetry, prompt, memory, evidence, provider, screenshot, and email records attach back to `pipeline_runs`.

This is the database shape behind the run-detail console. The dashboard can show the final answer and the failure path because agent runs, model calls, browser tool calls, runtime events, prompt compilations, memory hints, streams, screenshots, providers, and email drafts are queryable as separate rows.

`PipelineResult` is the aggregate runtime object returned by the orchestrator, grouping classification, matches, extraction results, streams, screenshots, provider analysis, emails, final status, and metrics.

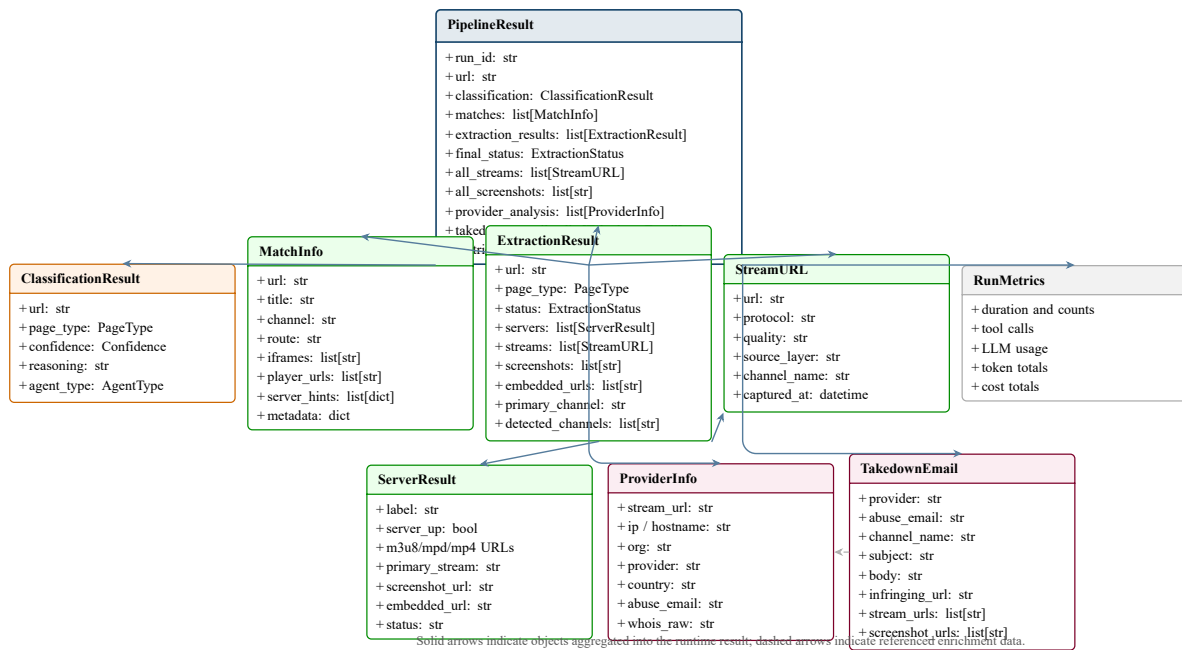


Figure 4.28: Runtime schema class diagram centered on `PipelineResult`, the aggregate Pydantic object that groups classification, matches, extraction results, streams, screenshots, provider analysis, email drafts, status, and metrics.

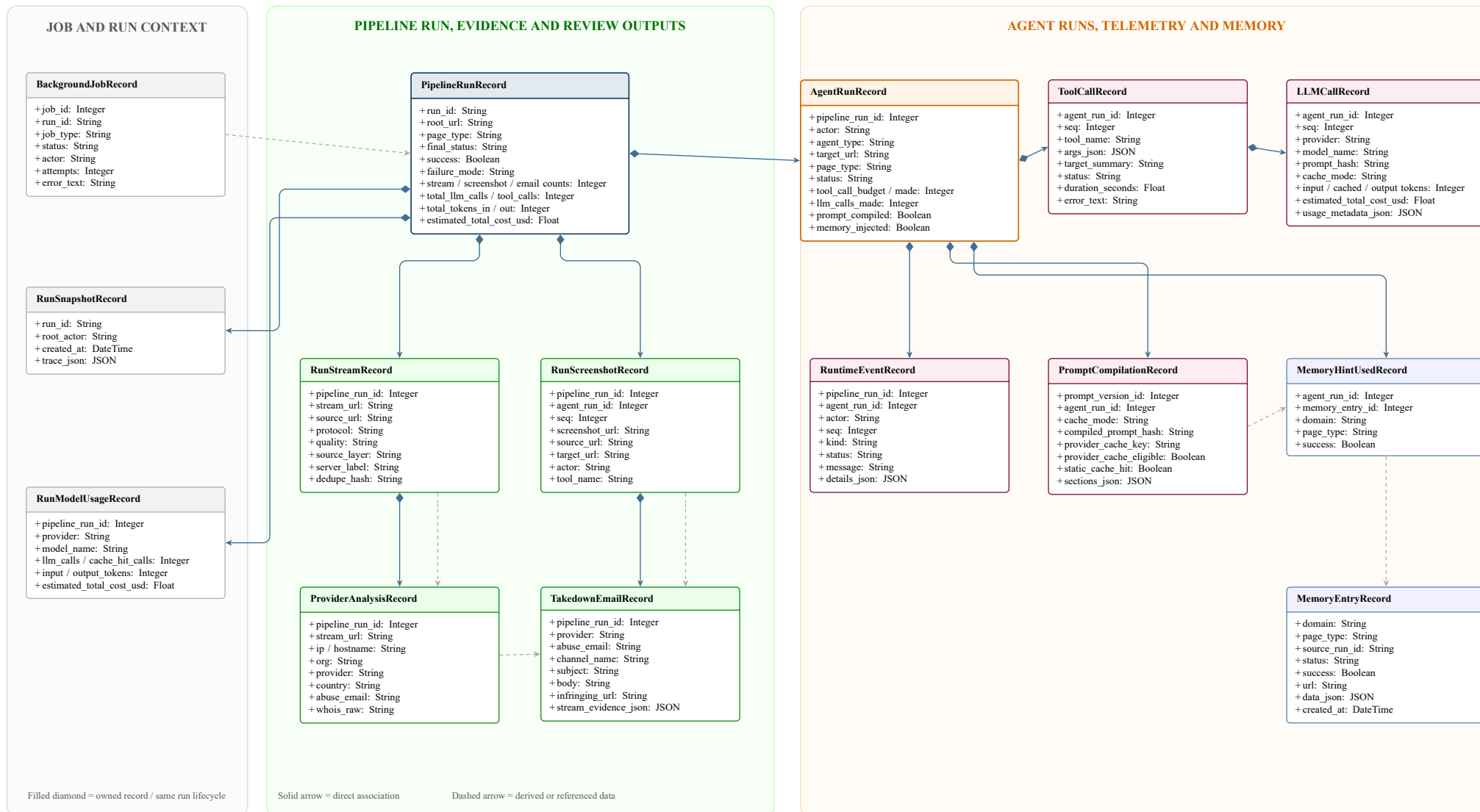


Figure 4.29: Persistence model for pipeline runs, extracted evidence, generated review outputs, agent telemetry, prompt compilation, and memory usage.

Architectural element	Boundary	Reviewable output
Orchestrator state	Current URL, classification, pending hosting and embedded queues, extraction results, provider rows, emails, and final error state.	Route decisions and final PipelineResult.
Handoff context	Root URL, target URL, page type, route source, evidence checklist, navigation policy, and memory hints.	Bounded specialist task context.
Runtime observer	Agent lifecycle, model calls, tool calls, screenshots, status changes, cancellation, and cost counters.	Events, rollups, traces, and metrics.
Persistence layer	Background jobs, run records, agent runs, evidence rows, screenshots, streams, provider rows, and email drafts.	The data behind GET /ui/runs/{run_id}.

Table 4.6: Runtime data boundaries and persisted review outputs.

4.8 Chapter Summary

The architecture is a staged, reviewable evidence pipeline. The operator starts and reviews runs through the console. FastAPI controls execution and persistence. The orchestrator uses LangGraph to route between agents and to preserve handoff state [11, 12]. Each specialist agent uses LangChain and LangGraph for the model and tool loop, interprets channel evidence according to its page role, and calls profile-scoped MCP browser tools for rendered DOM context, frame inspection, screenshots, media state, and network evidence. Browser runtime controls handle profile state, fingerprint behavior, proxy selection, popup recovery, iframe recovery, and screenshot capture. Provider and email tools run only after streams exist, and storage preserves both the final answer and the failure path. The next chapter maps these architecture boundaries to the concrete implementation.

Implementation

The implementation progressed in three major parts. The first part was an n8n landing-agent workflow used for fast exploration, validation, and backup execution when the company workflow needed rendered-page reasoning. The second part was Open Web Catcher (OWC), the academic/reference platform built around FastAPI, Next.js, MCP browser-tool containers, specialist agents, model/runtime management, and PostgreSQL observability. The third part was the deployment direction: the company kept its existing operational workflow and deployed only the landing agent as a focused backup cloud job.

This split matters because the two systems did not have the same purpose. The n8n workflow was useful for testing whether agentic browsing could handle the domain at all and whether it could recover dynamic landing-page evidence when the primary workflow missed it. OWC was built after those lessons, with cleaner engineering boundaries, persistent traces, configurable runtime settings, and a better foundation for evaluation. The deployment work then narrowed the cloud target to the part that the company actually wanted to operationalize during the internship.

5.1 Part 1: n8n Workflow Prototype

5.1.1 Purpose of the n8n phase

The n8n implementation acted as the project playground [14]. It made it possible to test the core question quickly: can an LLM-driven browser agent inspect illegal streaming websites, navigate through landing and hosting pages, activate players, and recover useful evidence? At this stage, the priority was not a clean software architecture. The priority was proving feasibility on real, unstable websites.

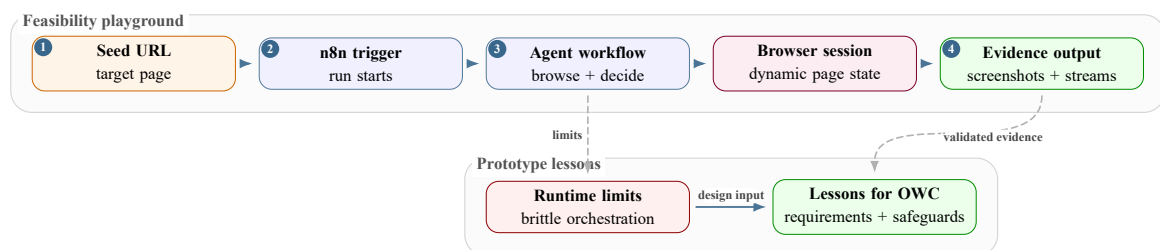


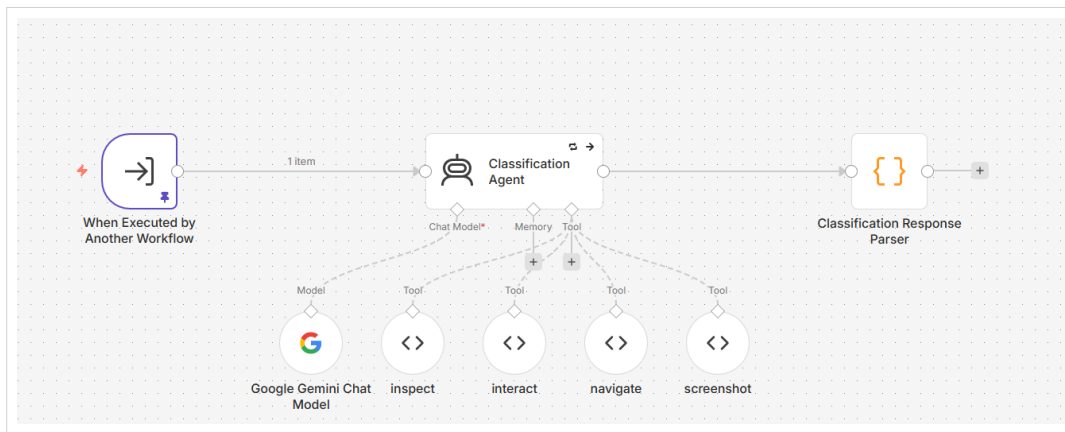
Figure 5.1: Role of the n8n workflow as a feasibility playground for browser-agent evidence collection.

The workflow helped test agent behavior before investing in a full backend, database, and operator console. It also exposed which parts of the task were hard: browser state continuity, popup handling, stream activation, memory, runtime length, and evidence validation.

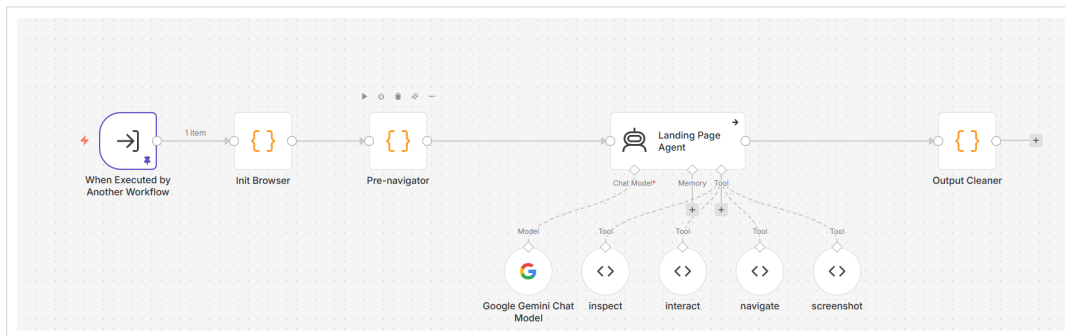
5.1.2 n8n workflow screenshots

The n8n workflow used agent-like nodes to test the same page-type responsibilities that later became specialist agents. Figure 5.2 shows the full sequential workflow used during the prototype phase. The following figures give the specialist-agent screenshots in normal stacked layouts.

The full workflow view gives the overall sequence, but it is too dense to explain each agent responsibility on its own. The next figure separates the first two prototype responsibilities: classification and landing-page exploration.



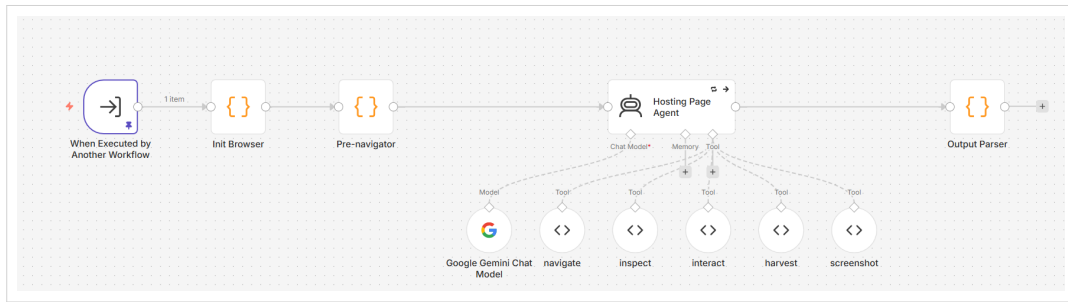
Classification workflow



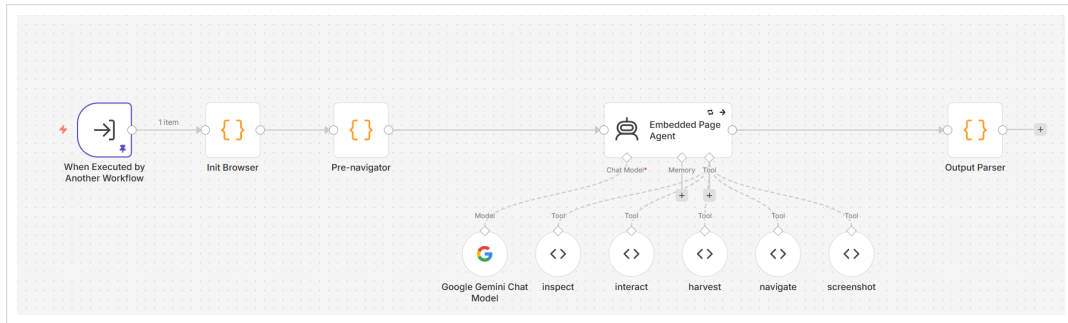
Landing-page workflow

Figure 5.3: n8n classification and landing workflow screenshots.

After classification and landing discovery, the prototype moved into the more difficult browser-interaction tasks. The hosting and embedded screenshots show the parts of the workflow that tested player activation, server handling, iframe inspection, and stream harvesting.



Hosting-page workflow



Embedded-player workflow

Figure 5.4: n8n hosting and embedded-player workflow screenshots.

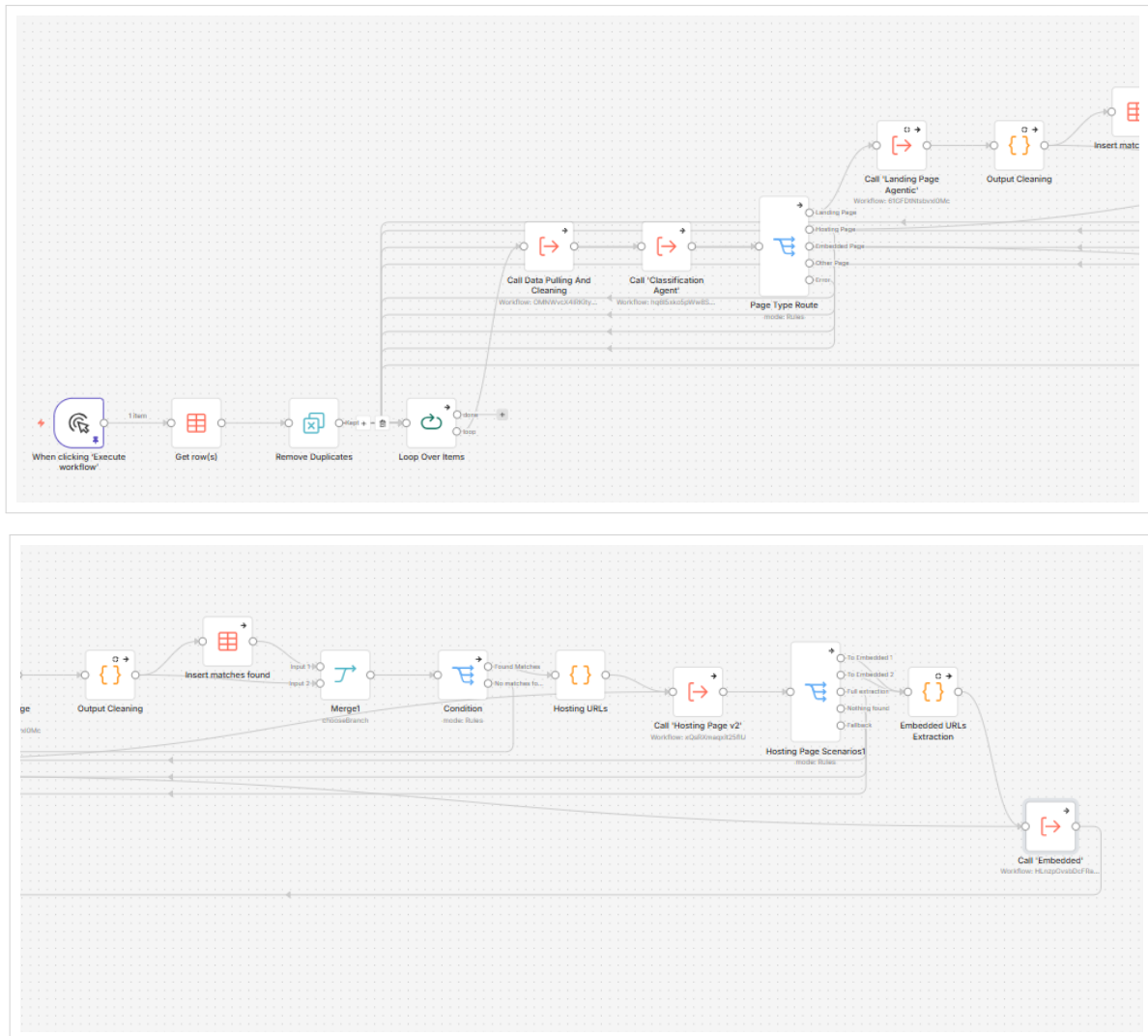


Figure 5.2: Full n8n workflow shown as two readable portrait panels. The figure validates the prototype sequence from trigger and classification through landing, hosting, and embedded extraction without using a sideways report page.

n8n agent role	What it tested	Lesson learned
Classification	Whether a page could be categorized as landing, hosting, embedded, or other from browser evidence.	Routing must be explicit and fail-closed; wrong classification wastes the whole run.
Landing	Whether listing pages could expose match cards, tabs, and hosting links.	Generic DOM inspection was not enough; landing pages needed page-type-specific extraction.
Hosting	Whether server buttons and play controls could be activated.	Stream evidence appears only after interaction; screenshot and media-state validation are necessary.
Embedded	Whether iframe and player pages could reveal final manifests.	Nested frames require a dedicated extraction path and should not be mixed with landing-page exploration.

Table 5.1: n8n prototype agent roles and implementation lessons.

5.1.3 Limitations of the n8n implementation

The n8n workflow was valuable, but it reached clear limits when the task became larger than a single experimental run. These limits explain why the second implementation was necessary.

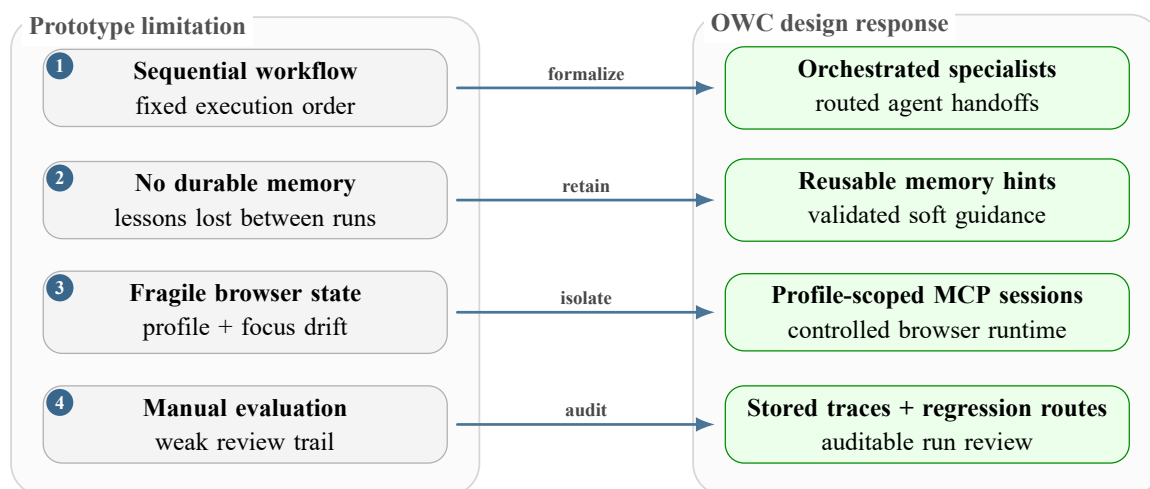


Figure 5.5: How n8n limitations shaped the Open Web Catcher implementation.

Limitation	What happened in practice	Consequence
No real evaluation layer	Runs could be inspected manually, but there was no structured evaluation harness for repeatable scoring.	Prompt changes were hard to compare objectively.
Weak memory model	The workflow did not provide a durable, domain-aware memory layer for reusable site knowledge.	Agents repeated failed actions and could not reliably reuse successful selectors or strategies.
Long runtimes	Large pages and multi-server hosting pages created long blocking executions.	Debugging became slow and failures were expensive to reproduce.
Sequential execution	The workflow behaved mostly as a sequential chain rather than parallel specialist agents.	Runs were slower than necessary, especially when multiple hosting or embedded candidates existed.
Limited development environment	n8n is a workflow editor, not a normal source controlled development environment.	Workflows had to be exported as JSON files to preserve progress and compare versions.
Fragile Puppeteer integration	Puppeteer code inside n8n nodes was brittle and tools frequently broke when browser state changed.	Browser automation needed a dedicated tool service with stable sessions.
Isolated code execution	Code executions were isolated, so browser instances could be lost between steps. A <code>puppeteer.connect</code> workaround was needed to reconnect to a running browser.	Browser continuity became a workaround instead of a clean runtime contract.
High memory usage	Long browser sessions, page artifacts, and workflow state created large memory pressure.	The prototype was not comfortable for long or batch-oriented extraction runs.
Hard failure analysis	Errors were visible, but not stored as normalized events, LLM calls, tool calls, screenshots, and costs.	Post-mortem review required manual reconstruction.

Table 5.2: n8n prototype limitations and engineering consequences.

5.2 Part 2: Open Web Catcher Platform

OWC keeps the useful extraction logic from the n8n phase but moves it into explicit software components. It is organized into five main implementation areas: FastAPI, Next.js, MCP browser containers, agents and model runtime, and PostgreSQL.

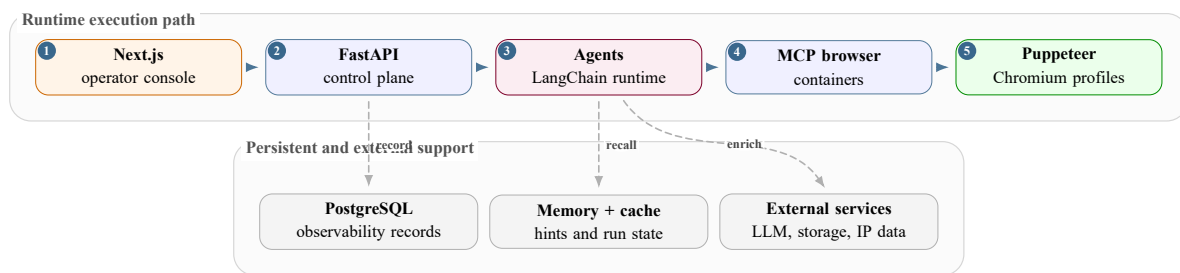


Figure 5.6: Five implementation areas of the Open Web Catcher platform.

5.2.1 FastAPI Backend

The FastAPI backend is the control plane of OWC [9]. It starts runs, checks runtime readiness, streams or returns run state, exposes settings, stores observability records, and gives the frontend a stable API surface.

Backend area	Responsibility	Why it matters
Runtime health	Check browser, MCP, settings, and launch readiness.	Failed preflight is clearer than a silently broken agent run.
Run execution	Start single URL runs and dataset-backed runs.	Keeps workflow launch separate from UI rendering.
Run inspection	Return trace, screenshots, streams, provider data, model usage, and errors.	Makes each run auditable after execution.
Configuration	Persist model, provider, pricing, browser, and tool settings.	Allows controlled experimentation without editing code every time.
MCP tool diagnostics	Expose profile-specific tool status and logged tool calls.	Separates browser/tool failures from prompt failures.
Evaluation routes	Run or review batches and reliability checks.	Provides the foundation that the n8n prototype lacked.

Table 5.3: Backend API responsibilities in OWC.

The backend also uses lightweight caching for frequently polled read endpoints. This reduces repeated database work when the frontend refreshes dashboards or run summaries.

5.2.2 Next.js Operator Console

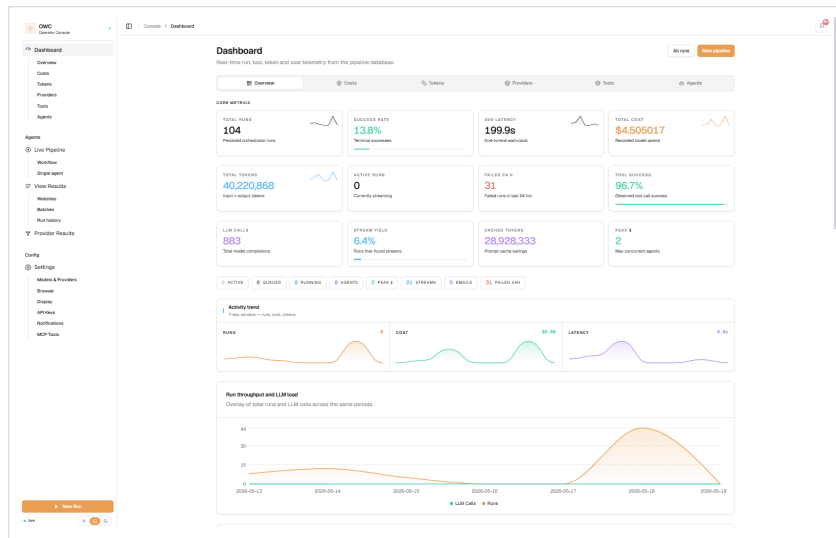
The Next.js frontend is not just a dashboard [10]. It is the OWC operator console for launching runs, following execution, reviewing evidence, and changing runtime configuration.

The design goal was operational clarity. A failed launch should show why it failed. A completed run should show what the agent actually did, which tools it called, what evidence it found, and where the model or browser runtime failed.

Frontend surface	What it shows or controls	Main backend dependency
Dashboard	Overview, cost, token, provider, tool, and agent telemetry.	Dashboard aggregation endpoints and persisted observability tables.
Live Pipeline	Workflow and single-agent launch with runtime preflight.	Workflow launch, health, and browser/MCP readiness endpoints.
View Results	Website dataset, batch execution, per-site status, and run history.	Dataset, batch, and run repositories.
Provider Results	Stream URL enrichment, IP resolution, provider distribution, country distribution, and manual lookup.	Provider-analysis records and lookup services.
Settings	Gemini model assignments, cache/runtime controls, browser settings, display preferences, API key status, notifications, and MCP tool enablement.	Configuration endpoints and runtime settings persistence.

Table 5.4: Current Next.js operator-console surfaces and backend dependencies.

Dashboard telemetry



Live workflow launcher

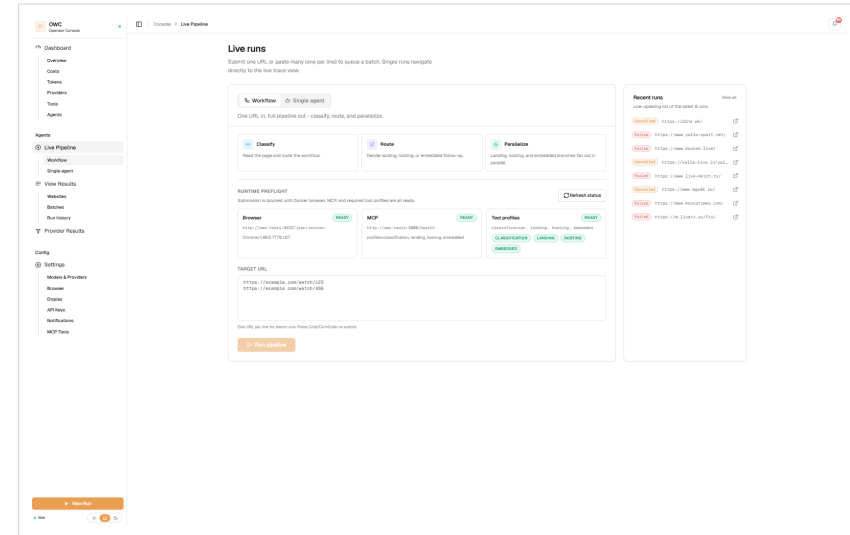
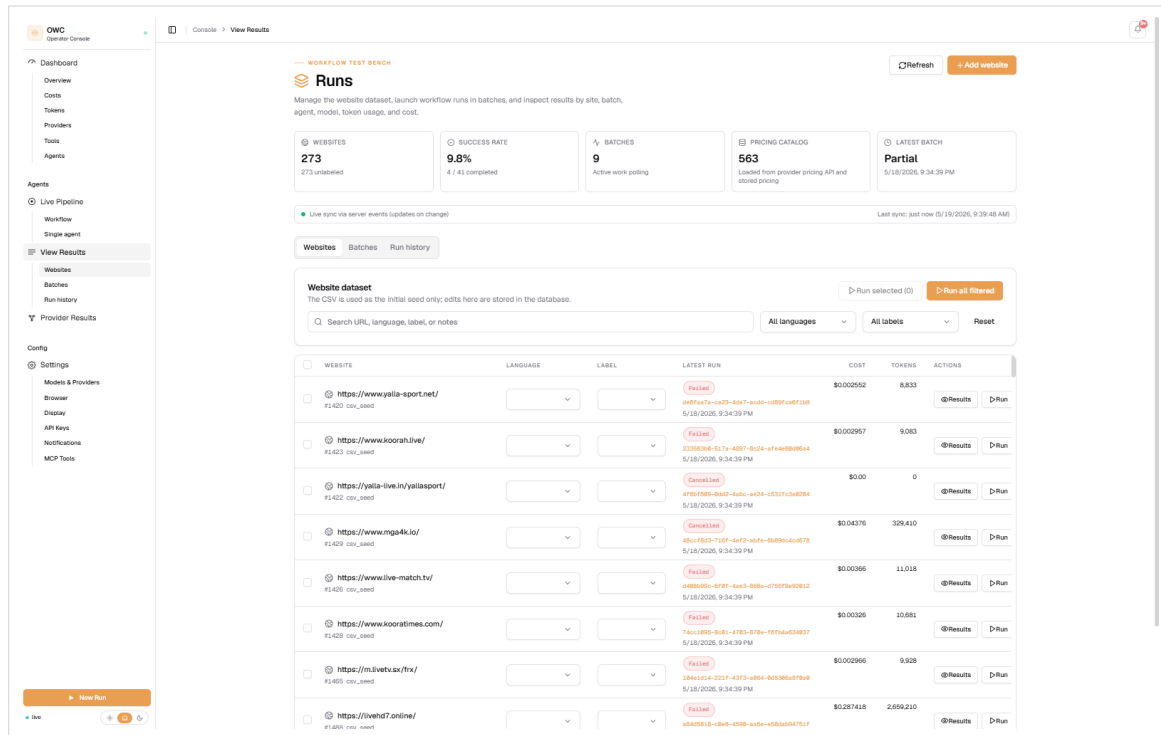
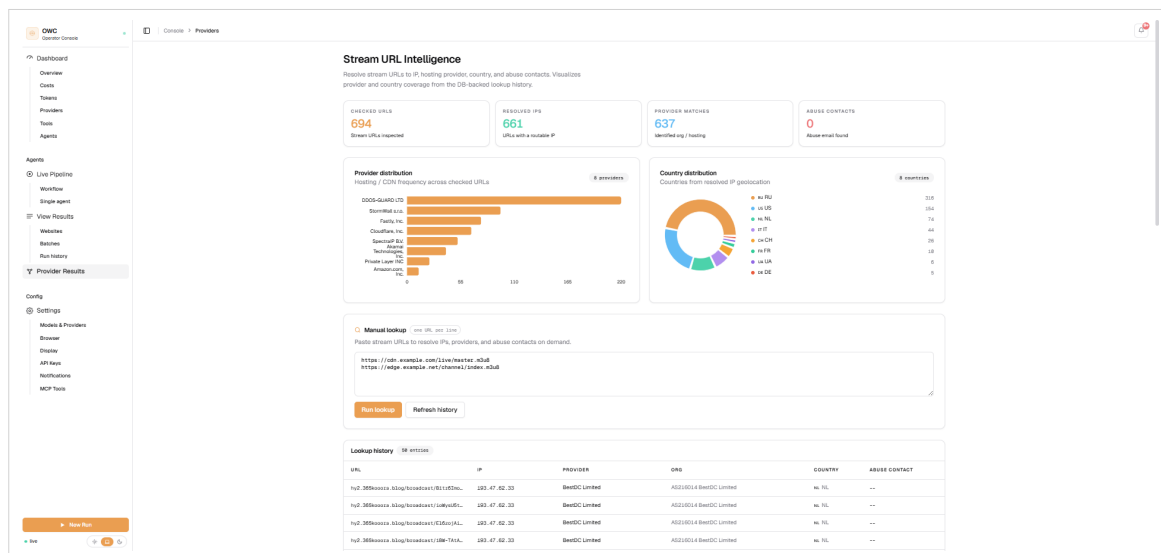


Figure 5.7: Main OWC operator-console screens for dashboard telemetry and live run launch.

The run detail and dashboard views make completed executions inspectable. The next console views focus on review and configuration screens, which are needed after a run has produced evidence.



Run and website review



Provider enrichment review

Figure 5.8: OWC review screens for run results and provider enrichment, enlarged so tables remain readable in print.

The review and settings views expose the user-facing controls. The final console screenshot group focuses on browser-tool diagnostics, which are useful when a run fails because of tool availability or profile configuration.

Browser engine	Strengths observed	Project decision
Playwright	Strong context isolation, good frame and browser-context APIs, useful for comparing behavior and validating browser policies.	Tested and kept as a useful reference path, but not the primary runtime.
Puppeteer	Direct Chrome DevTools Protocol fit, simpler integration with the existing tool registry, stable enough for the profiled MCP tool server.	Chosen as the main browser engine for the implemented agent prompts and tool runtime.

Table 5.5: Browser automation engine comparison for the MCP tool layer.

Tool group	Purpose
Navigation tools	Open URLs, go back, wait for page state, and recover from navigation drift.
Inspection tools	Read page context, landing cards, hosting server controls, embedded player state, DOM elements, and frame trees.
Interaction tools	Click by selector, text, XPath, checkbox/radio/select, coordinates, play media, type input, scroll, and swipe.
Evidence tools	Capture screenshots, media state, stream requests, player signals, and harvested m3u8/mpd/mp4 candidates.
Memory tools	Read or update reusable site hints from the memory layer.

Table 5.6: Puppeteer MCP tool groups and responsibilities.

receives broad page inspection and screenshot access. Landing receives tools for listing-page context, candidate extraction, reveal actions, scrolling, and element details. Hosting receives tools for server controls, playback activation, popup recovery, media state, and stream harvesting. Embedded receives frame-focused tools that keep the run inside the assigned iframe or player URL. This filtering prevents a landing task from drifting into provider analysis or a hosting task from rewriting the whole route.

5.2.3.2 Rendered DOM context retrieval

The implementation uses Puppeteer because the useful evidence is usually in the rendered browser state rather than the initial HTML response. Context tools execute inside the active Chrome page, read visible text and DOM structure, group anchors and buttons, inspect iframe trees, and return compact JSON observations. The agent first asks for a broad page context, then asks for a scoped element or frame detail before acting. This two-step pattern reduced repeated full-page dumps and made long runs cheaper to review because the trace shows why a selector, server tab, iframe, or candidate URL was chosen.

5.2.3.3 Puppeteer actions and evidence feedback

Action tools are implemented as browser-state changes followed by explicit evidence checks. A click, scroll, text input, server switch, play action, or popup recovery is not considered useful by itself. The agent must inspect the page again, capture a screenshot when the visual state matters, read media state when a player is involved, or harvest network candidates after playback. This is why the tool trace alternates between observation, action, and verification instead of showing one large scripted Puppeteer procedure.

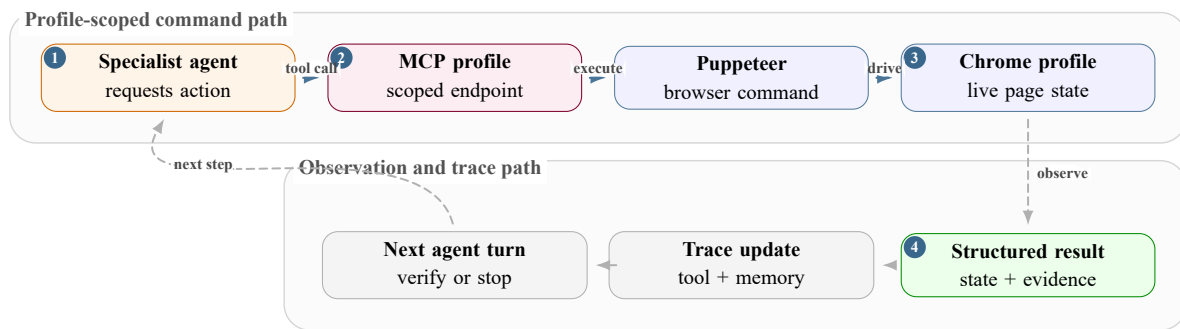


Figure 5.10: Browser tool execution loop through MCP and Puppeteer.

5.2.4 Agents, Models, Memory, and Prompts

OWC uses specialist agents rather than one general browser agent. Each specialist receives a role-specific prompt, a profile-specific tool set, and a bounded execution budget.

Agent	Objective	Main tools used
Classification	Decide whether the seed is landing, hosting, embedded, or other, while preserving obvious channel hints.	Generic inspection, screenshot, frame tree, memory lookup.
Landing	Discover match cards, listing tabs, pagination, and hosting candidates with match and channel context.	Landing inspect, element queries, click/scroll, screenshot, memory tools.
Hosting	Activate server controls, verify player state, and extract streams or embedded fallbacks while checking channel/player labels.	Hosting inspect, click/play tools, media state, frame tree, harvest, screenshot.
Embedded	Work only on embedded/player URLs and harvest final stream evidence.	Embedded inspect, play/media tools, harvest, screenshot, frame diagnostics, iframe-local text checks.

Table 5.7: Specialist agent objectives and assigned tool surfaces.

5.2.4.1 Design constraints implemented in the agent runtime

The agent runtime was shaped by constraints that became clear during prototyping. These constraints are implementation concerns, not only methodology notes, because each one changed how the agents, prompts, tools, memory, and stopping rules were built.

Constraint	Runtime effect	Implementation response
Changing layouts	Selectors and visible labels differ across languages, clones, and match-list layouts.	Agents use broad DOM context first, then scoped element details and page-type-specific tools.
Nested iframe players	The playable media source may not exist in the top-level page.	Hosting agents can hand off embedded URLs; embedded agents inspect frame trees and player-only pages.
Popups and ad traps	Clicks can open new tabs, steal focus, or hide the original player.	Browser tools restore focus, close popup tabs, and retry with safer JavaScript interactions.
Delayed stream generation	Stream manifests often appear only after a play click or server switch.	Prompts require media-state or screenshot verification before harvesting network evidence.
Cost and context growth	Repeated DOM observations, screenshots, and retries can make runs expensive.	The runtime tracks token usage, uses caching, and enforces bounded tool budgets.
Evidence isolation	The reference system must not depend on company production data.	Persistence is limited to traces, metrics, screenshots, tool calls, model usage, and structured outputs.

Table 5.8: Runtime constraints and their implementation responses.

5.2.4.2 Prompt evolution in the implementation

The prompts evolved from linear intent prompts into evidence-driven loops. The early prompt asked the model to solve the page in one pass. That worked on simple pages, but it failed on ambiguous layouts because the agent would stop before opening tabs, activating players, or verifying stream generation. The implemented prompt pattern instead forces each specialist to observe, reason, act, and validate before returning a result.

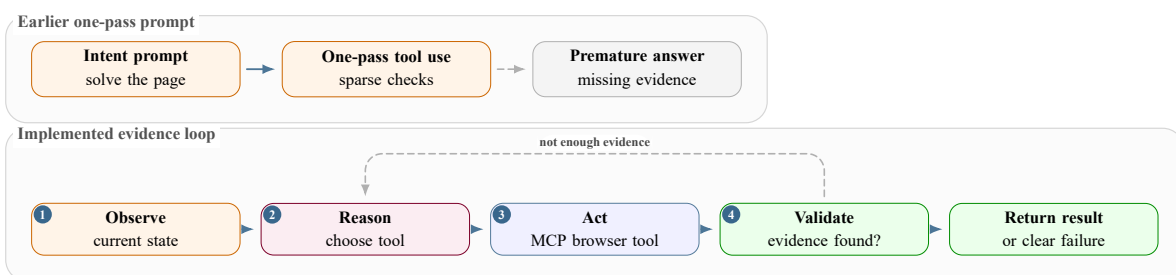


Figure 5.11: Prompt evolution from one-pass intent prompts to evidence-driven agent loops.

5.2.4.3 Agent design principles

The resulting agent design follows seven practical principles:

- classify the page role before deep extraction;
- separate operational n8n validation from the OWC platform architecture;
- ground every conclusion in browser evidence rather than textual guessing;
- preserve traces, screenshots, and structured outputs for each critical step;
- use specialist agents and explicit fallbacks instead of one unconstrained agent;
- optimize reliability and cost together through caching, budgets, and model routing;
- keep deployment loosely coupled across API services, browser workers, observability storage, and queue-based execution.

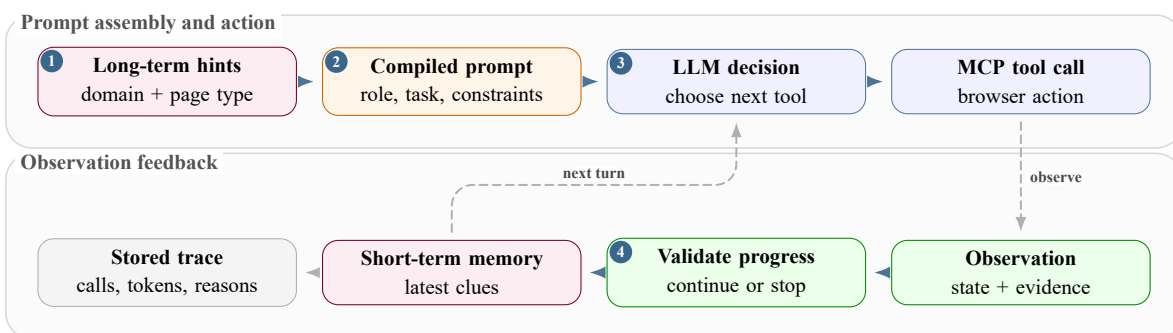


Figure 5.12: Agent reasoning loop with memory and tool feedback.

LangChain is used as the tool-binding and message-loop layer [11]. The implementation does not let the model freely access every browser capability. Instead, the MCP client loads only the tool profile for the current agent. The shared loop records tool calls, model calls, token usage, cache behavior, and stop reasons.

5.2.4.4 Model behavior, token usage, and rate limits

The model layer had to be treated as an operational dependency, not only as a prompting detail. Long browser pages, repeated DOM observations, screenshots, tool traces, and failed retries can grow the context quickly. The implementation therefore tracks context windows, input tokens, output tokens, cached tokens, cache writes, and estimated cost.

Gemini rate limits are evaluated across RPM, TPM, and RPD. Google's current provider reference defines these as requests per minute, input tokens per minute, and requests per day, and states that limits depend on model and usage tier and can change in AI Studio [1]. The implementation therefore treats rate limits as configuration and observability concerns rather than hard-coded constants.

5.2.4.5 Prompt engineering and stopping behavior

The prompts evolved from broad goals into operational procedures. Each prompt defines the objective, the evidence required, which page ownership rules must be respected, and when the

Model concern	Implementation handling	Why it matters
Context window	Store and display model context-window metadata where available.	Prevents long runs from silently approaching context exhaustion.
Token usage	Track input, output, cached input, and cache-write tokens.	Makes cost and prompt growth visible per run.
Tool calling	Bind only profile-specific tools through LangChain and MCP.	Reduces incorrect tool choice and keeps browser actions controlled.
Rate limits	Treat RPM, TPM, and RPD as provider limits to monitor and respect.	Avoids confusing quota failures with browser or prompt failures.
Retries	Apply bounded LLM retry attempts and delays per agent profile.	Recovers transient provider errors without infinite loops.
Caching	Use prompt/provider cache and tool-result cache where safe.	Reduces repeated tokens and repeated browser observations.

Table 5.9: Model-runtime concerns handled by the agent platform.

agent should stop. The strongest prompt pattern was an evidence-based loop: observe, state the current hypothesis, choose one action, verify the result, then either continue or return a structured failure.

Prompt design rule	Implementation effect
Explicit page ownership	Hosting agents stay on hosting pages; embedded agents stay on embedded/player URLs.
Evidence before harvest	Agents confirm player state or screenshot state before claiming stream extraction.
Bounded tool budget	Long runs stop with a clear partial result instead of looping indefinitely.
Structured output contract	Results can be stored, evaluated, and rendered in the UI without manual parsing.
Memory hints as soft guidance	Prior site knowledge helps, but does not replace current evidence.

Table 5.10: Prompt engineering rules enforced by the specialist agents.

5.2.5 PostgreSQL Database and Observability

The PostgreSQL database is used mainly for observability, review, evaluation, and configuration. It is not the product database of the external organization. Its purpose in this project is to preserve what happened during a run so that failures and successes can be inspected later.

Record group	Tables / records	Status in the project
Core run trace	Pipeline runs, agent runs, runtime events, LLM calls, tool calls.	Central to reviewing and debugging runs.
Evidence	Run streams, screenshots, provider analyses, takedown emails.	Central to the evidence bundle.
Model and prompt metadata	Prompt versions, prompt compilations, model usage, pricing configs.	Important for cost, prompt iteration, and reproducibility.
Memory	Memory entries and memory hints used.	Used to connect cross-run knowledge with current agent decisions.
Datasets and batches	Dataset sites, dataset batches, dataset site runs.	Useful for evaluation/batch execution, but not always used in every run.
Operational support	Run decisions, run tasks, tool-call diagnostics, snapshots.	Support/diagnostic records; some remain secondary rather than core report outputs.

Table 5.11: PostgreSQL observability records and their implementation role.

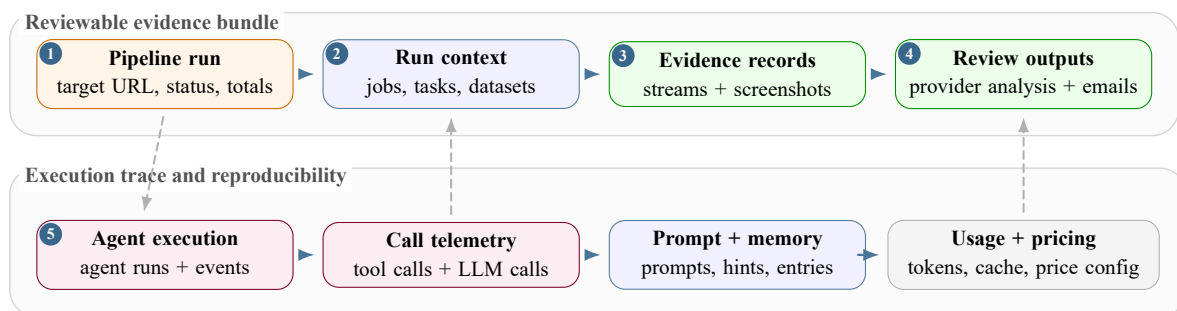


Figure 5.13: Summarized PostgreSQL persistence model derived from the detailed database class diagram.

5.3 Part 3: Deployment

The company deployment decision was intentionally focused. The existing company workflow remained the primary operational system, and the deployment did not attempt to replace it with the complete OWC platform. Instead, the deployed unit was the validated n8n landing-agent backup: a browser-capable agent used when the primary flow cannot extract enough evidence from a dynamic landing page.

The deployment shape follows that practical boundary. A task message containing the target URL and identifiers such as a job or execution ID enters the `agent_tasks` Azure Service Bus queue. A focused Azure Container Apps Job then runs the landing-agent worker with its Puppeteer MCP endpoint and Chrome browser runtime. The agent opens the rendered landing page, uses model reasoning to inspect visible match, channel, pagination, and candidate-link signals, and returns the result through a configured results queue [15, 16].

The returned payload is meant for operational review and integration rather than for automatic enforcement. It can include matched URLs, match or channel metadata, screenshots, rejected URLs with reasons, extraction summary values, a reasoning log, raw JSON for debugging, and failure information when no useful candidates are found. This keeps the company workflow, queue interface, and browser-agent backup loosely coupled: the primary process can continue normal discovery, while the landing agent handles the expensive rendered-page work only for cases that need it.

This deployment choice matches the internship scope. OWC shows how a complete platform could be structured, but the company-side deployment remained focused on the backup landing agent as a cloud-executable single-container unit.

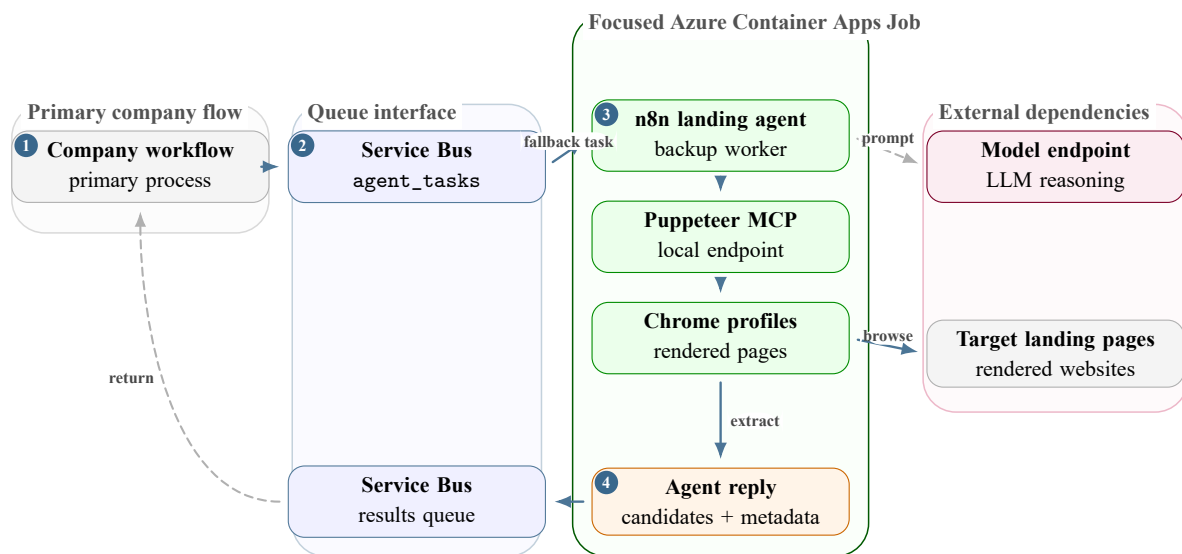


Figure 5.14: Company deployment flow using agent_tasks, a focused browser-agent container, and a configured results queue.

Deployment element	Role	Scope during internship
n8n landing-agent backup	Handles dynamic landing pages when the primary company workflow needs browser reasoning.	Deployed as the focused operational backup.
Azure Service Bus	Decouples fallback submission from browser execution.	Used to pass landing-page jobs to the container worker and return results.
Container Apps Job	Runs one containerized agent unit when a message arrives.	Single image for the landing-agent worker, Puppeteer MCP, and Chrome.
Puppeteer runtime	Opens the page and extracts match/host candidates through the local MCP endpoint.	Runs inside the same job container as the landing agent.
Output queue	Publishes matched URLs, metadata, screenshots, rejected URLs, logs, and failure reasons to the configured results queue.	Used for review and integration back into the workflow.

Table 5.12: Landing-agent deployment scope and responsibilities.

5.4 Implementation Summary

The implementation moved from a useful but limited n8n prototype to the structured OWC platform. The n8n phase validated the feasibility of agentic browser extraction and exposed the main runtime problems. OWC addressed those problems with explicit backend routes, a reviewable frontend, profiled MCP browser tools, specialist agents, memory and caching, model-usage tracking, and PostgreSQL observability.

The result is not only an extraction script. It is a reviewable agent platform where failures can be diagnosed, prompts can be evaluated, tool behavior can be tested independently, and evidence can be inspected after the browser run ends.

Evaluation and Testing

This chapter evaluates Open Web Catcher (OWC) as an evidence-production system for rights-protection review. The goal is not to claim production-grade piracy detection or universal illegal-streaming coverage. The goal is to check whether the platform can take a suspected streaming page and produce a traceable evidence package: route decision, agent activity, screenshots, stream evidence, provider intelligence, email readiness, and operating-cost telemetry.

The evaluation is organized in the same order as the real review workflow. First, the chapter defines the business question. Then it explains the batch-test design and the status taxonomy used to interpret results. After that, it evaluates each specialist agent, studies one concrete run in detail, and reviews the dashboard telemetry and cost metrics. This structure is important because the final run status alone is too weak for this project: a website can fail because an agent behaved badly, but it can also fail because the site is dead, a hosting page is inaccessible, a stream server is down, or an embedded player does not expose media during the capture window.

6.1 Evaluation Objective and Business Context

6.1.1 Business Issue

The practical business issue is manual uncertainty. A suspected live-sports streaming website rarely exposes all useful evidence on the first page. A landing page may only list matches. A hosting page may hide the player behind server buttons, popups, or delayed activation. An embedded page may be reachable only through an iframe or after a specific hosting server is selected. Even when a playlist URL is found, it still needs to be connected to screenshots, provider metadata, and a notice draft before a reviewer can use it.

In a beIN-context rights-protection workflow, the analyst therefore needs an evidence bundle, not only a classification label. Soft Stars provides the internship engineering context, while beIN MEDIA GROUP provides the media-rights context in which live-sports piracy is treated as an operational protection problem.

6.1.2 Evaluation Question

The central evaluation question is:

Does OWC reduce manual uncertainty by producing a traceable, screenshot-supported, and provider-enriched evidence package that a human reviewer can inspect?

This question is more suitable than a generic accuracy score because OWC is an agentic browser system. The reviewer needs to know what the agents did, which page state was observed, which server path was attempted, what evidence was captured, and what remained uncertain.

6.1.3 Reviewable Evidence Bundle

For this report, a run is considered reviewable when it can provide a meaningful subset of the following evidence:

- seed URL and persisted run identifier;
- page-route decision and specialist-agent handoff;
- landing, hosting, or embedded-player navigation evidence;
- player activation, server switching, iframe inspection, or documented blocker evidence;
- stream evidence such as manifest, embedded, media, or playlist URLs;
- screenshots that validate the page, player state, server path, or failure condition;
- provider enrichment for the stream or playlist host;
- abuse-contact and takedown-email draft fields when available;
- trace, tool-call, token, cache, latency, and cost telemetry.

6.1.4 Evidence-First Evaluation

The chapter treats metrics as support for evidence review, not as a replacement for it. A strict success rate shows how many runs reached the strongest terminal status, but it does not explain partial evidence, website availability, server availability, or agent-level behavior. For this reason, the evaluation separates strict success, productive evidence, external blockers, agent/runtime failures, repeatability, and cost.

6.2 Evaluation Protocol and KPI Definitions

6.2.1 Batch Test Design

6.2.1.1 Input Set and Repeated Runs

The batch evaluation is organized around websites. A batch contains suspected streaming URLs with different page structures: landing pages, hosting pages, embedded players, redirecting pages, inactive pages, and pages whose server buttons change over time. Each URL is launched through the same pipeline: classification, specialist routing, evidence extraction, provider enrichment, and review-output generation.

The repeated-run idea is important for this domain. A website that works once may stop working later because a player server goes offline, a page redirects, the site changes its anti-bot behavior, or the stream is available only during a live event. Repeated runs therefore test not only initial success, but also whether evidence-producing sites remain evidence-producing after prompt, tool, browser, or website changes.

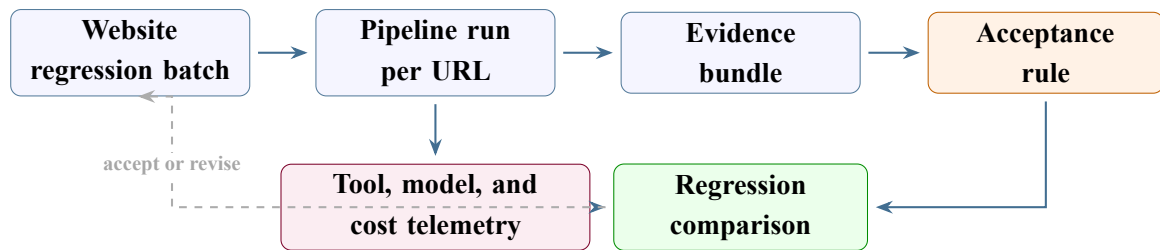


Figure 6.1: Evaluation protocol used for website-batch regression testing.

6.2.1.2 Manual Review After Automated Status

After each batch run, the automated status is reviewed together with the trace, screenshots, stream records, provider rows, and email records. This manual review step is necessary because the same terminal status can hide different causes. For example, a no-stream result may happen because the agent failed to activate the player, but it may also happen because all available servers were down during the run.

6.2.1.3 Website-Level Acceptance Rule

A website is accepted only when the run produces evidence that can be inspected. This is stricter than simply reaching a terminal state.

Checkpoint	Website is accepted when
Page classification	The page is routed to landing, hosting, embedded, or terminal handling according to observable browser behavior.
Landing discovery	Landing pages return relevant match-detail or hosting candidates, or the absence of candidates is explained.
Navigation	The agent reaches the next useful page instead of stopping at the first listing, redirect, advertisement state, or unsupported page state.
Player activation	Hosting or embedded agents interact with play controls, iframe contexts, or server buttons until a player state, media state, or blocker is observed.
Server switching	When server controls exist, the agent tests server paths in a reasonable order and records which path produced evidence or failed.
Network evidence	The run returns at least one useful manifest, embedded URL, media URL, or stream candidate when playback evidence exists.
Screenshot evidence	Screenshots support the final claim by showing the player, activation state, source page, server path, or failure condition.
Provider/email evidence	Stream or playlist evidence is enriched with provider metadata and, when available, abuse-contact or notice-draft fields.

Table 6.1: Website-level acceptance criteria used during batch evaluation.

6.2.2 Failure Taxonomy

6.2.2.1 Why Failure Categories Matter

The dashboard strict success rate is useful, but it must be interpreted carefully. A low strict success rate may indicate weak agent behavior, but it may also reflect a difficult batch containing dead sites, inaccessible hosting pages, inactive streaming servers, and embedded pages that no longer load. These are still important operational results, but they are not the same as a bad classification or an incorrect browser-tool sequence.

Bucket	Status values	Count in snapshot	Interpretation
Productive evidence	success, partial	33 terminal runs	Full or partial evidence was produced. Partials should be reviewed separately instead of being hidden inside failure.
External or expected blocker	page_inaccessible, site_dead, no_hosting_pages, no_streams	22 terminal runs	The page, hosting route, embedded route, player, or stream server did not provide a usable path during the run.
Agent or runtime failure	failed, timeout, redirect	80 terminal runs	The run may have failed because of route choice, tool order, timeout, browser drift, model behavior, or runtime instability.
Excluded from quality score	cancelled	10 terminal runs	The run was stopped, so it should not be used to judge agent quality.
Active	queued, running	4 non-terminal runs	The run is still in progress and is excluded from terminal metrics.

Table 6.2: Failure taxonomy used to separate productive evidence, external blockers, agent/runtime failures, cancelled runs, and active runs.

6.2.2.2 External Blockers Are Not Automatically Agent Faults

This distinction is especially important for live-streaming pages. A site can be down. A match page can exist while all hosting pages have been removed. A hosting page can be reachable while the selected server is down. An embedded player can load visually but produce no manifest request. These cases still reduce operational yield, but the agent evaluation should record them as availability or evidence-path blockers unless the trace shows that the agent skipped an available action.

6.2.3 KPI Definitions

6.2.3.1 Rates Used in This Chapter

The evaluation keeps several rates side by side. This is more transparent than forcing all outcomes into one success metric.

KPI	Formula and interpretation
Strict success rate	$success_runs/terminal_runs$. This is the dashboard's strict success view.
Partial rate	$partial_runs/terminal_runs$. This reports useful but incomplete evidence separately.
Productive evidence rate	$(success_runs + partial_runs)/terminal_runs$. This is the rate of runs that produced full or partial reviewable evidence.
External blocker rate	$external_blocker_runs/terminal_runs$. This identifies rows where website, hosting, embedded, player, or stream availability limited the result.
Agent-adjusted non-failure rate	$(success_runs + partial_runs + external_blocker_runs)/terminal_non_cancelled_runs$. This is not a success rate; it indicates how many non-cancelled runs did not end as agent/runtime failures.
Evidence extraction rate	$runs_with_streams/terminal_runs$. This measures technical stream evidence production.
Email yield rate	$runs_with_emails/terminal_runs$. This measures whether provider and notice fields reached email-draft readiness.
Tool success rate	$successful_tool_calls/observed_tool_calls$. This monitors browser-tool reliability separately from agent judgment.
Regression retention	$previously_accepted_still_accepted/previously_accepted$. This checks that prompt or tool changes do not break earlier working sites.

Table 6.3: KPI definitions used to interpret batch results without hiding partials or external blockers.

6.2.3.2 Current Batch Snapshot

The current dashboard/API snapshot contains 149 total runs, 145 terminal runs, and 4 running runs. The terminal status breakdown is 25 successful runs, 8 partial runs, 80 failed runs, 10 cancelled runs, 14 no-hosting-page runs, 6 no-stream runs, and 2 page-inaccessible runs.

The strict success rate is 17.24%. The partial rate is 5.52%. Combining both gives a productive evidence rate of 22.76%. The external blocker count is 22 terminal runs, or 15.17% of terminal runs. If cancelled runs are removed from the quality denominator, the agent-adjusted non-failure rate is $55/135 = 40.74\%$. This last value is not an evidence success rate; it is a diagnostic rate showing that part of the raw failure picture comes from site and server availability rather than only from agent/runtime failure.

Metric group	Current value	Interpretation
Run volume	149 total runs, 145 terminal runs, 4 running runs.	Active runs are kept outside terminal-quality interpretation.
Strict success	25 successful runs, 17.24% strict success.	This is the strongest terminal-success view, but it excludes partial evidence.
Productive evidence	25 success and 8 partial runs, 22.76% productive evidence.	Partials are incomplete but still useful for review and debugging.
External blockers	2 page-inaccessible, 14 no-hosting-pages, and 6 no-streams.	These rows can come from source-site, hosting-page, embedded-page, player, or stream-server availability.
Agent/runtime failures	80 failed runs.	These rows need trace-level diagnosis because they may indicate route, tool-order, timeout, model, or runtime problems.
Cancelled runs	10 cancelled runs.	Cancelled runs are excluded from agent-quality judgment.
Latency	763.632 seconds average latency.	Browser-agent runs are expensive in time because they navigate, interact, wait, and verify unstable pages.

Table 6.4: Current batch outcome snapshot used for Chapter 6 interpretation.

6.3 Agent-by-Agent Behavior Evaluation

6.3.1 Agent Test Matrix

6.3.1.1 Why Agent-Level Evaluation Is Required

The agent pipeline is made of specialist responsibilities. Because of that, a whole-run label is not detailed enough. The selected run used in this chapter demonstrates the point: the embedded-page agent failed, but hosting-page agents still produced valid stream, screenshot, provider, and email evidence. The evaluation therefore checks the behavior owned by each agent instead of treating the whole workflow as one black box.

6.3.2 Classification and Landing Tests

6.3.2.1 Classification Tests

Classification is tested first because it decides the rest of the workflow. A correct classification result should identify whether the seed page behaves like a landing page, hosting page, embedded page, redirect, inaccessible page, or unsupported state. The review checks the route, the confidence, the visible evidence used for the decision, and whether the next agent receives the correct handoff.

Agent	Tests performed	Passing evidence
Classification	Route correctness, confidence, redirect/blocker recognition, and handoff to the right specialist.	The chosen route matches visible page behavior and the next agent receives the correct task.
Landing	Match-card discovery, schedule/list extraction, duplicate-link filtering, host-candidate discovery, and no-hosting-page handling.	Relevant hosting or match links are returned, or the absence of a hosting route is clearly recorded.
Hosting	Player activation, popup survival, server detection, server switching order, screenshot timing, iframe handoff, and stream capture.	A player path, screenshot, stream candidate, iframe target, or clear server/player blocker is returned.
Embedded	Iframe access, direct player inspection, delayed media harvest, embedded-page inaccessible handling, and no-stream outcomes.	Media evidence is harvested, or the embedded blocker is explained without hiding it as a generic failure.
Provider and email	Playlist-host attribution, IP/provider lookup, country and organization enrichment, abuse-contact lookup, and notice-draft readiness.	A provider row and reviewable email fields are attached to concrete stream evidence.

Table 6.5: Agent-specific tests used during manual and batch evaluation.

6.3.2.2 Landing Tests

Landing-page tests check whether the agent can move beyond a listing page. The reviewer looks for match cards, event links, watch buttons, schedule sections, pagination, and host candidates. A landing page is not considered a failure just because it does not contain a stream. It only becomes an agent problem if useful hosting candidates were visible and the agent missed them. If the page genuinely exposes no usable downstream route, `no_hosting_pages` is the more accurate status.

6.3.3 Hosting Tests

6.3.3.1 Player Activation

Hosting-page tests focus on interaction. The reviewer checks whether the agent clicks the correct player controls, survives popups, returns to the player when focus is stolen, and waits long enough for the page to generate media activity. The screenshot must be taken after a meaningful state change, not before the player has been activated.

6.3.3.2 Server Detection and Switching

Many streaming pages expose several server choices. The hosting agent is expected to detect server controls, try them in a reasonable order, and record which server path was used. If server 1 is down but server 2 works, the agent should continue instead of stopping too early. If all

servers are down, the result is a server/player availability blocker rather than automatically an agent failure.

6.3.3.3 Stream Capture Order

Stream capture is evaluated after activation and server switching. This order matters: harvesting network traffic too early can produce empty results even when the page would generate a playlist after a play action. A passing hosting result therefore connects the browser action, player state, screenshot, and stream evidence in the trace.

6.3.4 Embedded Tests

6.3.4.1 Iframe and Player Inspection

Embedded-page tests check whether the agent can inspect iframe/player contexts without mixing them with landing-page exploration. The agent should identify the embedded URL, inspect the frame or player state, attempt playback where appropriate, and harvest media requests after the page has had time to load.

6.3.4.2 Embedded-Page Inaccessible Cases

An embedded page can be inaccessible even when the hosting page is reachable. It can also load visually but never expose a playlist during the capture window. The evaluation records these as embedded availability or no-stream outcomes unless the trace shows that the agent skipped an obvious available action.

6.3.5 Provider and Email Tests

6.3.5.1 Provider Intelligence Criteria

Provider analysis starts from concrete stream evidence. Playlist URLs are useful because their host identifies the infrastructure that can be used for provider lookup and abuse contact discovery. The provider test checks host extraction, DNS/IP resolution when available, autonomous-system or organization enrichment, country fields, and whether the result is attached to the correct run and stream record.

6.3.5.2 Email Readiness Criteria

The generated email is not treated as an automatic enforcement action. It is evaluated as a draft. A draft is review-ready when it contains the provider/contact, infringing page, playlist host, stream evidence, screenshot references, and a clear need for human verification before sending.

6.3.6 Agent Dashboard Evidence

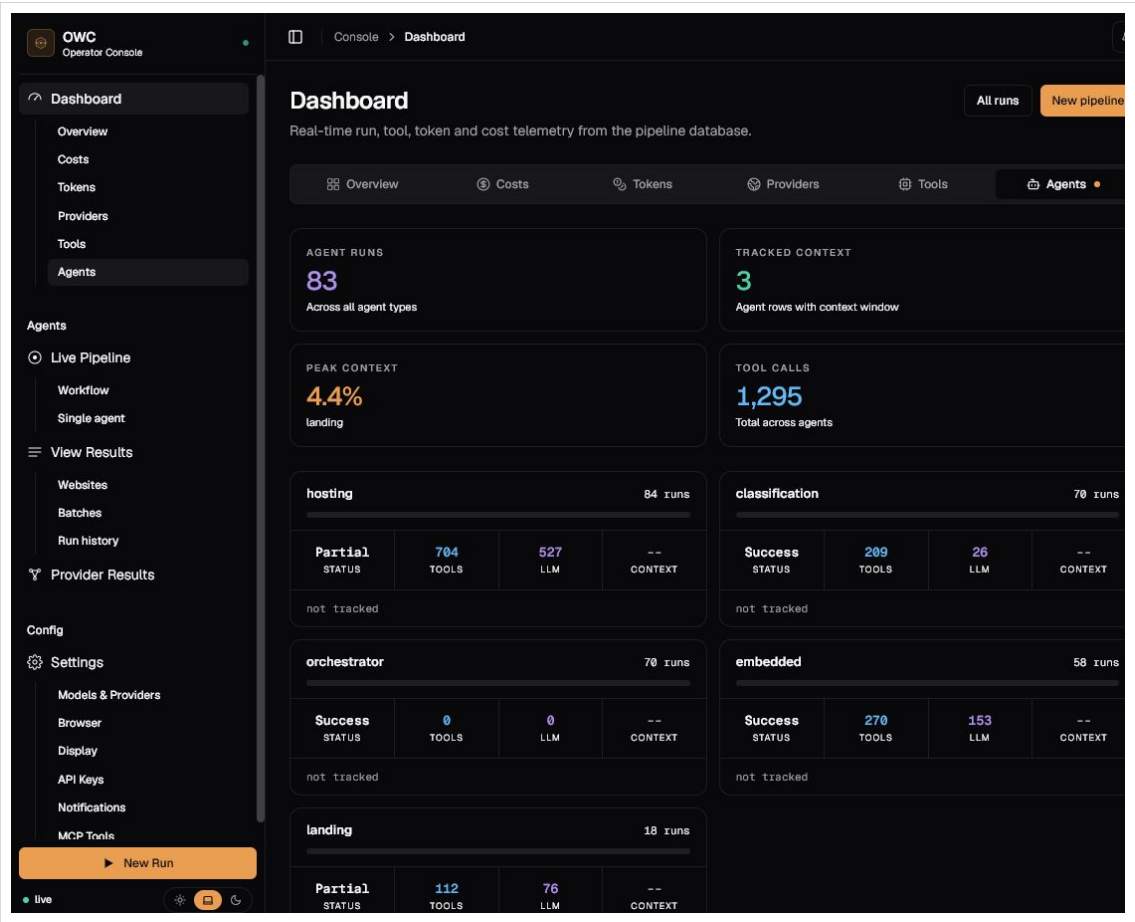


Figure 6.2: Agents dashboard used for behavior evaluation. The screenshot validates that agent runs, tool calls, LLM calls, context tracking, and status are separated by agent type instead of being reduced to one final run label.

The agents dashboard supports behavior-level evaluation by separating agent runs, tool calls, LLM calls, context tracking, and status by agent type. It helps locate where a batch is struggling: classification, landing discovery, hosting activation, embedded inspection, or provider/email enrichment. It does not replace manual review, but it turns manual review into a more targeted process.

6.4 Run-Based Evidence Case Study

6.4.1 Selected Run Summary

The main case study uses run 9566a9be-12ba-434a-afa0-f05f529d2403. It was available from the local backend and operator console during evidence capture, so no fallback run was used.

Field	Observed value
Seed URL	go4kora.life
Run date	May 23, 2026
Duration	302.91 seconds, approximately 5 minutes and 3 seconds
Final status	success
Top-level page type	landing_page
Agent runs	6 agent runs: orchestrator, classification, landing, two hosting attempts, and embedded
LLM and tool footprint	15 LLM calls and 25 tool calls
Evidence produced	2 HLS playlist URLs, 2 screenshots, 2 provider-analysis rows, and 1 email draft
Token footprint	503,127 input tokens, including 430,132 new input and 72,995 cached input, plus 9,636 output tokens
Estimated cost	\$0.12381188

Table 6.6: Selected-run metadata used as the main Chapter 6 evidence case study.

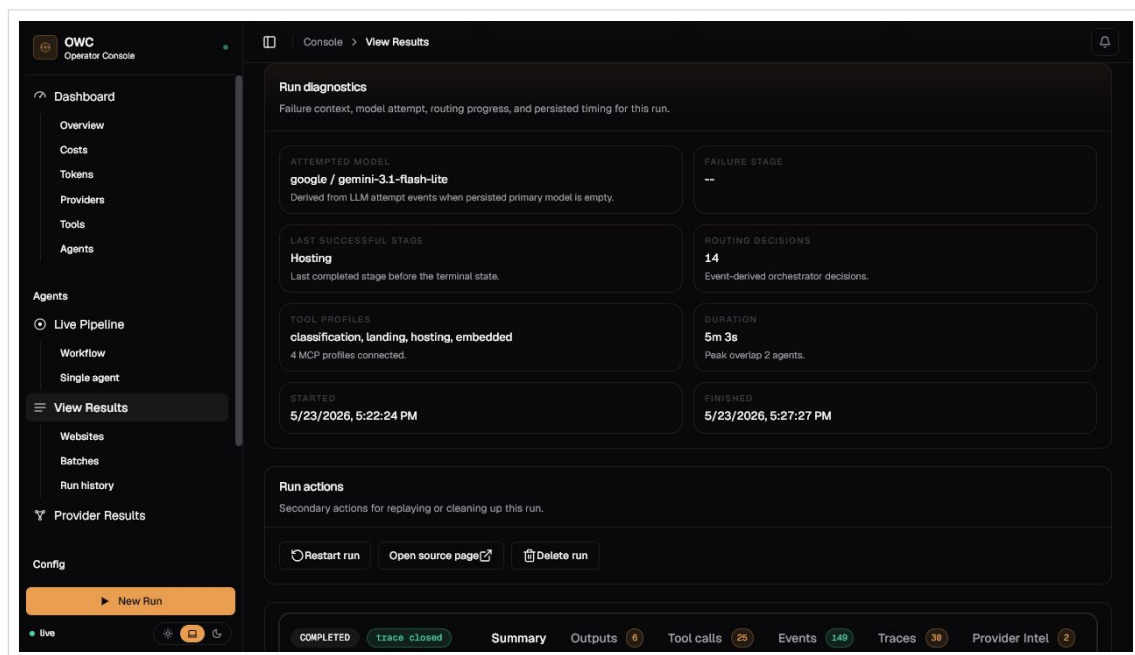


Figure 6.3: Selected run summary and status. The screenshot validates that the chosen run is visible in the operator console with its status, timing, model, tool profiles, and evidence counters.

The run summary screenshot validates that the selected case is a concrete persisted run, not only a manually written example. It shows the final status, run identity, timing, model/tool profile, and evidence counters in the operator console.

6.4.2 Trace and Agent Activity

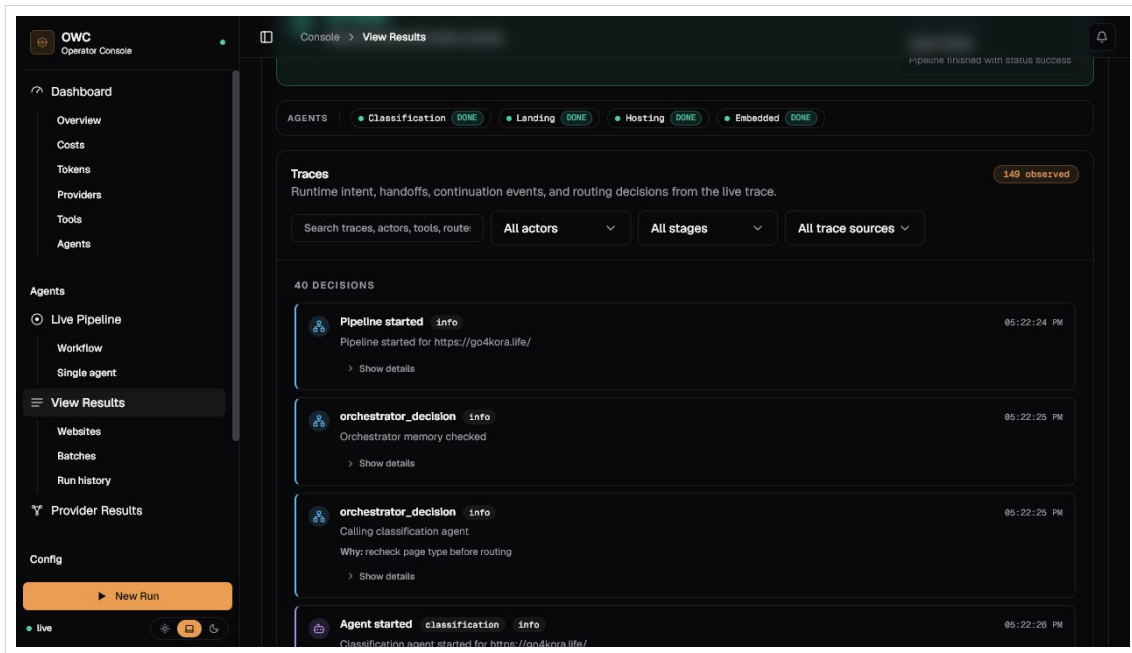


Figure 6.4: Trace rows and agent activity for the selected run. The screenshot validates that pipeline, orchestration, and specialist-agent steps were persisted for audit and manual review.

The trace view is used to reconstruct the path from seed URL to final evidence. In this case, the trace shows the landing-page route, downstream discovery, hosting attempts, and persisted specialist-agent activity. This is also where the embedded-agent nuance is visible: the embedded path did not succeed, but the hosting path still produced enough evidence for the run to be useful.

6.4.3 Landing Evidence



Figure 6.5: Landing-page evidence for the selected run. The screenshot validates that the workflow started from a match-listing context and had to continue toward a hosting/player path rather than stopping after classification.

The landing screenshot shows why specialist routing was needed. The first page is a match/listing context, so the relevant test is whether the landing agent can discover a useful hosting or player path instead of stopping at a surface-level page label.

6.4.4 Hosting and Player Evidence



Figure 6.6: Hosting/player evidence for the selected run. The screenshot validates that the workflow reached a player state on the discovered hosting path, supporting the manual player-activation review.

The first hosting screenshot validates player activation. It shows that the workflow reached a sports video/player state on the discovered path. This supports the hosting checks for navigation, activation, screenshot timing, and media-harvest context.



Figure 6.7: Additional hosting evidence for the selected run. The screenshot supports the server/player review by showing a second captured browser state from the evidence path.

The second hosting screenshot records another captured player state from the same evidence path. This matters because server behavior can vary during a run. Keeping multiple screenshots makes it easier to review whether a server was active, stalled, blocked, or unstable.

6.4.5 Stream, Provider, and Email Evidence

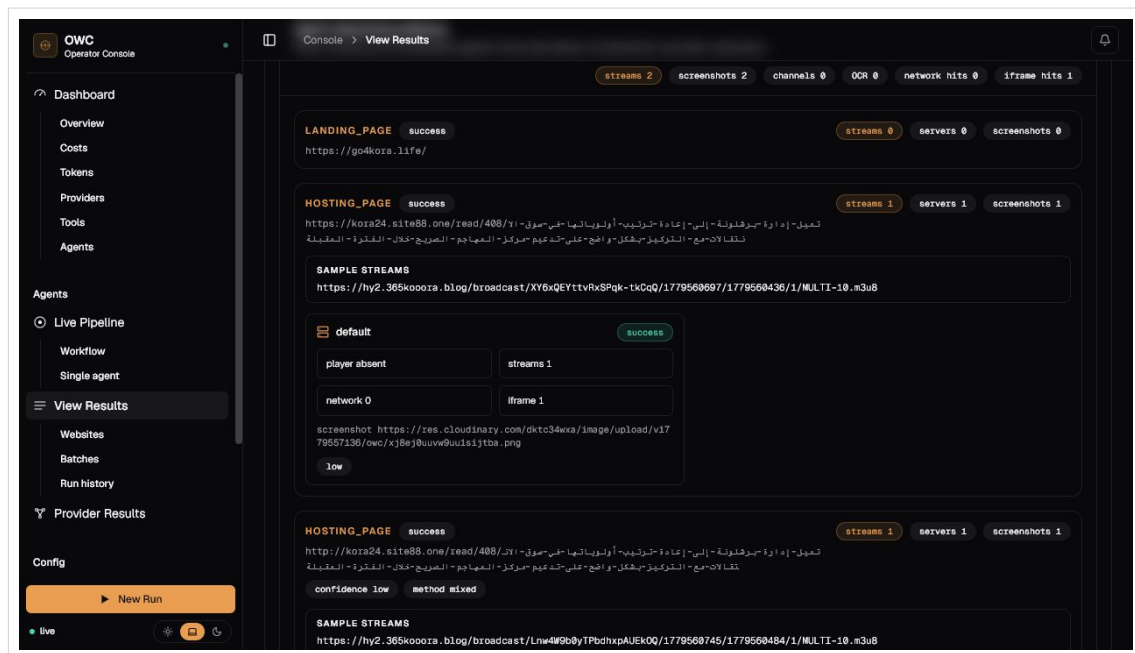


Figure 6.8: Streams and provider evidence for the selected run. The screenshot validates that stream/provider rows were persisted together with stream count and screenshot support.

The streams/provider view confirms that stream evidence was persisted together with source labels and screenshot support. This connection is important: the playlist URL is not an expiring token claim in this report; it is persistent review evidence because the playlist host can be used for provider attribution and email preparation.

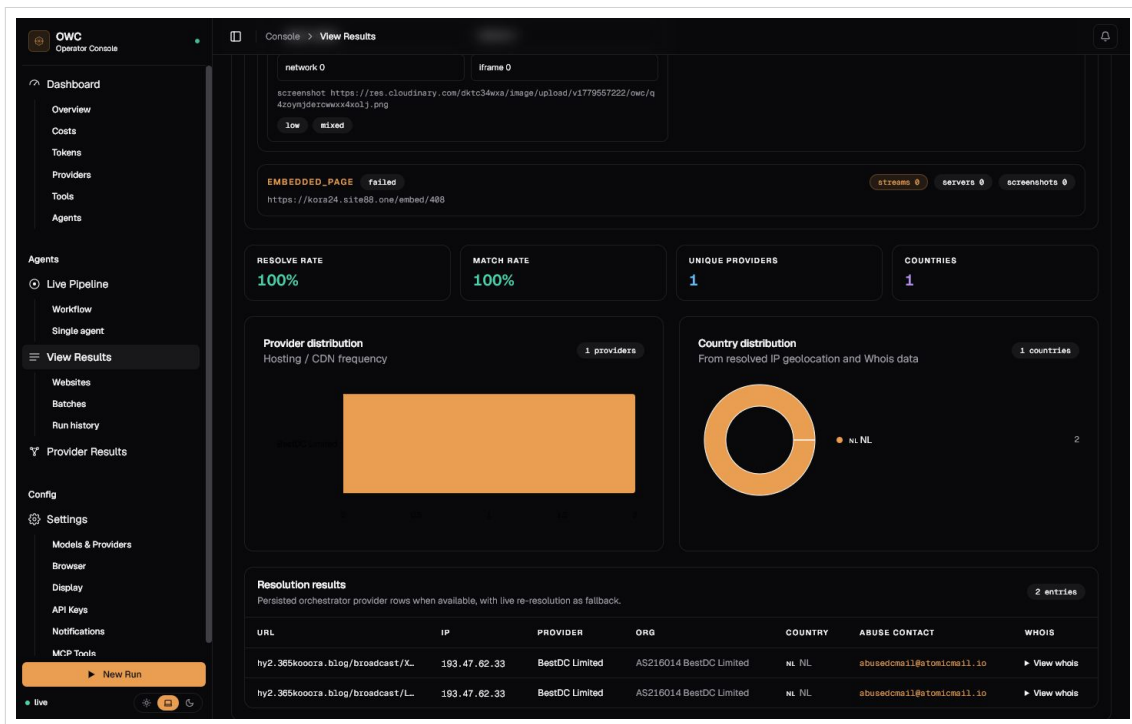


Figure 6.9: Provider-intelligence evidence for the selected run. The screenshot validates that the playlist host was enriched into provider, organization, country, and contact metadata for review.

The provider-intelligence view shows the enrichment step. In this run, the playlist host `hy2.365kooora.blog` was resolved to BestDC Limited, organization AS216014 BestDC Limited, country NL, with an abuse-contact field. This turns stream discovery into provider-intelligence evidence.

Generated email example from run 9566a9be-12ba-434a-afa0-f05f529d2403

Subject: DMCA Notice: unauthorized streaming hosted by BestDC Limited

Provider/contact: BestDC Limited, abusedcm11@atomicmail.io

Infringing page: <https://go4kora.life/>

Playlist host: `hy2.365kooora.blog`

Evidence fields:

- two m3u8 playlist URLs under `https://hy2.365kooora.blog/.../MULTI-10.m3u8`;
- protocol recorded as m3u8;
- server/source recorded as default;
- two screenshot evidence links attached to the stream records.

Review note: the email is an automatically prepared draft. A human reviewer must verify rights ownership, jurisdiction, contact accuracy, and wording before sending.

6.5 Dashboard Telemetry and Cost Review

6.5.1 Overview Metrics

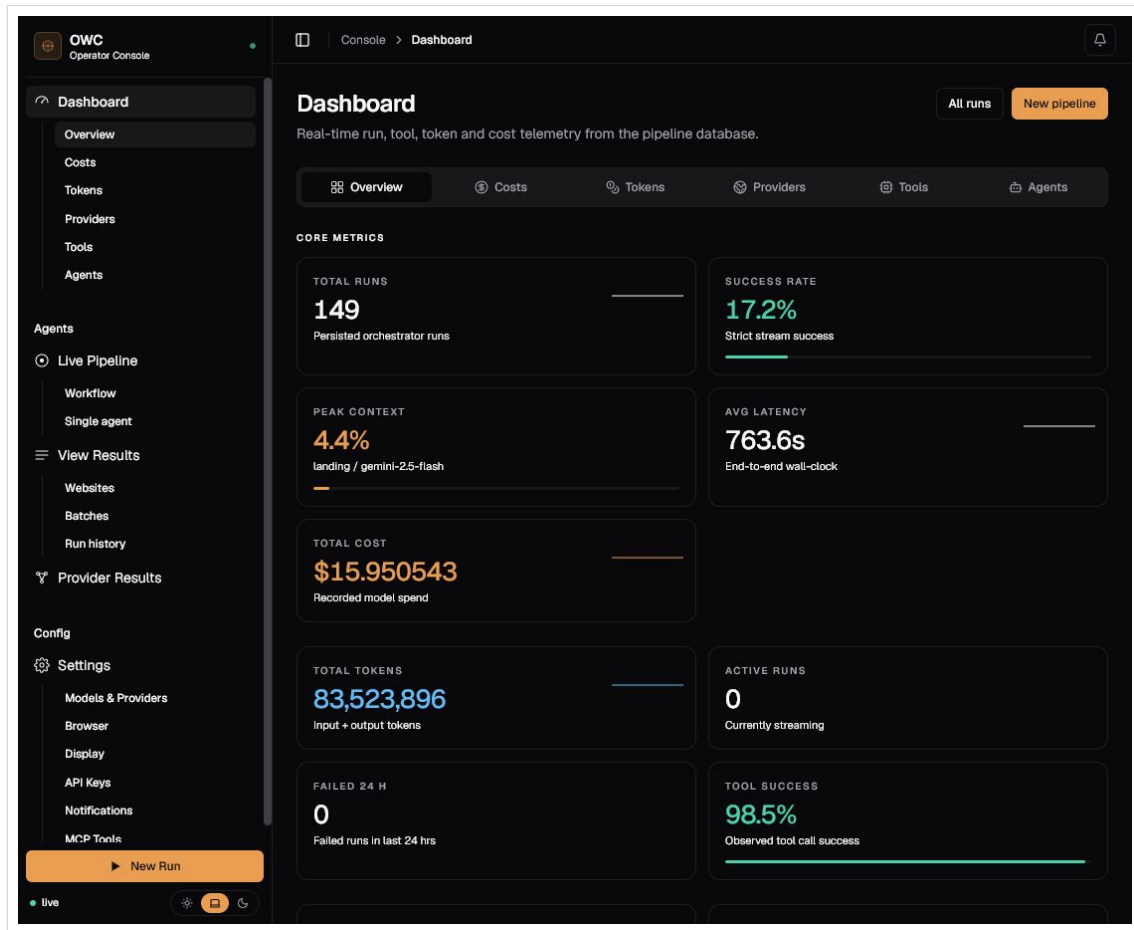


Figure 6.10: Overview dashboard metrics used in Chapter 6. The screenshot validates the current total runs, strict success rate, average latency, total cost, token volume, and tool-success cards behind the telemetry tables.

The overview dashboard gives the operational context for the batch. It shows that OWC is measurable: runs, success rate, cost, token volume, latency, active runs, and tool success are visible from the same console. These values should be read through the failure taxonomy, not as one generic model-accuracy claim.

Metric group	Current value	Interpretation
Evidence yield	25 runs with streams, 25 runs with emails, 136 total streams, 67 total emails.	Successful evidence production is tied to stream and email readiness, not only run completion.
Average yield	0.938 streams/run and 0.462 emails/run across terminal attempts.	Useful for batch-cost planning and expected review output.
Provider telemetry	136 provider analyses, 12 unique providers, 8.82% provider coverage.	Provider enrichment exists but remains incomplete across all attempts.
Tool reliability	3,113 observed tool calls, 3,066 successful, 47 failed, 98.49% success.	Browser-tool execution is mostly reliable; failures are often behavioral, site-specific, or availability-related.
Tool volume	3,209 total tool calls and 7.636 seconds average tool duration.	Agent work is interaction-heavy, especially for hosting pages.
Cost	\$15.950543 total model spend and \$0.110004 average cost per terminal run.	Cost is low enough for experiments but must be monitored for larger batches.
Tokens	83,523,896 total tokens, 54,252,590 cached input tokens, 110,760,668 new input tokens, and 1,017,267 output tokens.	These are dashboard telemetry cards used for cost review; the report does not recompute them into one billing formula.
Cache	32.9% cache hit ratio from cached and new input counters.	Repeated prompt and tool context reuse is visible but not dominant.

Table 6.7: Current evidence, provider, tool, cost, and token telemetry from the overview dashboard/API snapshot.

6.5.2 Token and Runtime Cost

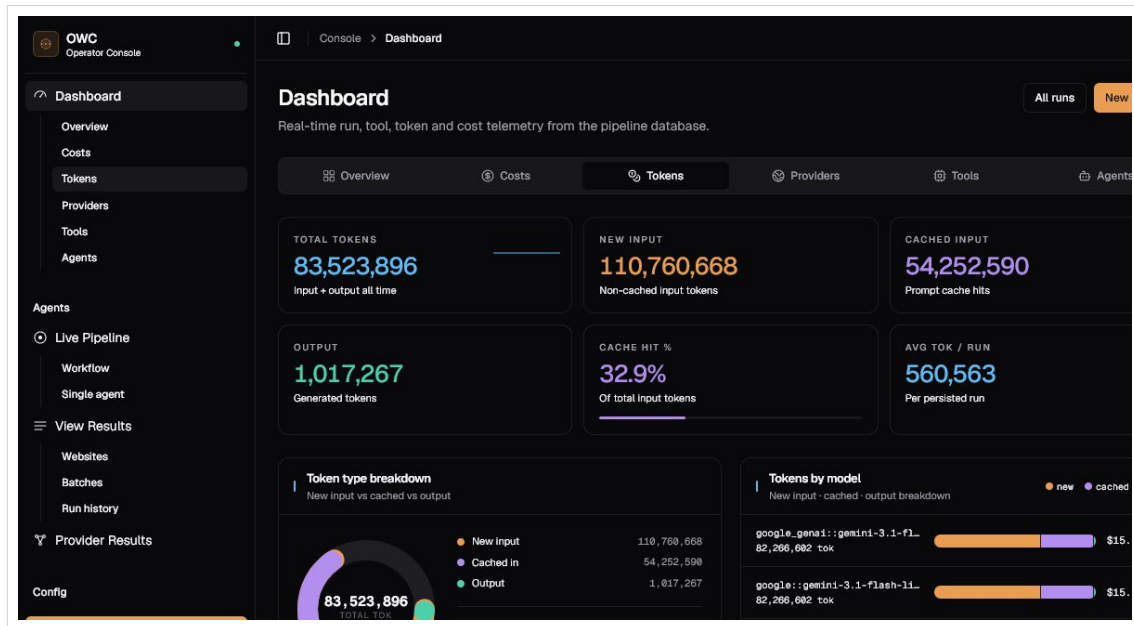


Figure 6.11: Token telemetry dashboard. The screenshot validates that the console separates new input, cached input, output tokens, cache-hit percentage, and average token volume for cost review.

Token telemetry is included because the project is also a data-science system evaluation. The cost of a browser agent depends on context size, cache reuse, output length, tool loops, and repeated attempts. Separating new input, cached input, and output tokens makes it possible to evaluate whether future prompt or tool changes are improving behavior without silently increasing cost.

6.5.3 Model and Runtime Selection Rationale

The model evaluation is not presented as a universal model ranking. The relevant question is which runtime is practical for browser-agent evidence collection. The chosen runtime must support long browser context, disciplined tool use, visible pricing and cache counters, and usable rate-limit behavior. Gemini pricing documentation is cited as operating context for cost interpretation rather than as a model-quality proof [2].

Dimension	Why it matters for OWC	Evidence used here
Context handling	Browser runs contain page state, iframe paths, tool outputs, and trace history.	Selected run used 503,127 input tokens across 15 LLM calls.
Tool discipline	The model must inspect, navigate, activate, harvest, and verify in the right order.	Selected run recorded 25 tool calls and reached stream/provider evidence.
Latency and cost	Large regression batches become impractical if each run is too slow or expensive.	Selected run finished in about 5 minutes at \$0.12381188.
Cache visibility	Repeated prompts and tool descriptions should be observable as cached or new input.	Selected run recorded 72,995 cached input tokens.
Rate-limit practicality	Batch testing depends on token and request limits, not only one successful run.	Dashboard telemetry tracks call, token, and cost volume.
Failure interpretation	Useful models leave enough trace evidence to explain partial success or uncertainty.	The selected run kept trace, provider, screenshot, and email records.

Table 6.8: Model/runtime selection rationale used instead of unsupported model-ranking metrics.

6.6 Limitations and Interpretation

6.6.1 Operational Limitations

6.6.2 Interpretation of the Current Results

The current strict success rate is low, and that is a real limitation of the current system. However, the raw failure rate should not be interpreted as one number meaning “the agents failed”. The batch contains productive partials, external blockers, cancelled runs, and true agent/runtime failures. A data-science interpretation must keep those categories separate before comparing prompt versions, browser settings, or model runtimes.

The most useful conclusion is therefore not that OWC has solved illegal-streaming evidence collection. It has not. The useful conclusion is that OWC now exposes the right evaluation objects: agent route, navigation path, server switching, screenshot timing, stream capture, provider enrichment, email readiness, repeatability, and cost.

Limitation	Effect on evaluation	Review action
Source site down	The seed URL can be inaccessible or dead before the agent can inspect it.	Record as a site blocker and rerun before drawing agent-quality conclusions.
Hosting page inaccessible	A landing page can list matches while the downstream hosting page is removed, blocked, or unreachable.	Treat <code>no_hosting_pages</code> separately from agent failures.
Embedded page inaccessible	A hosting page can expose an iframe URL that no longer loads or redirects away from the player.	Diagnose embedded availability separately from hosting behavior.
Server/player unavailable	Server buttons can point to inactive players or media paths with no stream.	Record server-switching order and screenshots before marking the case as no-stream.
Anti-bot drift	A website can change challenges, redirects, popups, or browser fingerprinting behavior after a successful run.	Treat traces and screenshots as time-bound evidence.
CDN/provider lookup gaps	Provider data can be missing, indirect, or routed through CDN and hosting intermediaries.	Use provider enrichment as triage and verify contacts before sending notices.
Partial-run telemetry	Strict success excludes partials, and some useful evidence can sit outside the headline status.	Report strict success, partial rate, and productive evidence separately.
Human/legal review	Screenshots, streams, and email drafts are evidence candidates, not legal determinations.	A trained reviewer must check rights context, jurisdiction, and notice wording.

Table 6.9: Limitations that remain after evidence-first evaluation.

6.7 Chapter Conclusion

This evaluation judges OWC by agent behavior and evidence quality. The selected run is successful because it provides a reconstructable path from a seed URL to route classification, landing navigation, hosting/player evidence, screenshots, HLS playlist evidence, provider enrichment, cost telemetry, and a generated email draft for human review. The broader batch snapshot shows that the system is measurable, but also that strict success remains low and many terminal rows require failure-category interpretation.

For a final-year data-science project, the main evaluation contribution is the measurement framework itself. Instead of hiding everything inside a generic accuracy score, the report evaluates OWC through strict success, partial evidence, external blockers, agent/runtime failures, repeatability, tool reliability, provider enrichment, email readiness, and operating cost.

General Conclusion

This PFE internship addressed a practical evidence-collection problem in the sports rights-protection domain. Soft Stars needed a workflow that could preserve reviewable evidence from dynamic illegal streaming websites. The most useful evidence is often not present in the first HTML response. It appears after a rendered page loads scripts, after a channel group is opened, after a server button is selected, after a player is activated, or after an embedded iframe requests a short-lived media manifest. The central problem was therefore not only to find suspicious URLs, but to preserve a reviewable browser trace that explains how the evidence was reached.

The work followed Design Science Research Methodology as the sole methodological backbone. The organizational problem was first identified through the limitations of static collection and manual review. The solution objectives were then defined around classification, browser interaction, screenshots, stream evidence, channel metadata, provider enrichment, traceability, and review. The artifact was designed and developed in two steps. The n8n workflow validated the idea quickly and gave the company a landing-agent backup path for dynamic targets. Open Web Catcher (OWC) then organized the same work into a complete platform made of FastAPI, Next.js, PostgreSQL, MCP browser tools, Puppeteer/Chrome runtime, specialist agents, model telemetry, and evidence outputs.

The main contribution is the separation of the investigation into clear page roles and agent responsibilities. The orchestrator controls the run, prepares handoffs, manages queues, and aggregates results. The classification agent decides whether a URL is a landing, hosting, embedded, or irrelevant page. The landing agent discovers match, event, channel, hosting, or embedded candidates. The hosting agent activates server and player controls while preserving screenshots and diagnostics. The embedded agent stays inside direct player contexts and harvests final media evidence. Provider enrichment and takedown-email drafting are downstream review stages, not automatic enforcement actions.

OWC also contributes a more reliable browser-tool layer. The tool profiles separate broad inspection, precise context reads, navigation, interaction, screenshot capture, media-state checks, stream harvesting, and memory hints. Browser runtime controls handle profile-scoped sessions, fingerprint behavior, proxy validation, sticky proxy selection, popup recovery, iframe recovery, and media verification. This made the workflow more explainable: a failure could be linked to classification, a missing hosting candidate, a player activation problem, an iframe issue, a proxy failure, a missing stream, or a skipped provider stage instead of being reduced to a vague failed scrape.

The implementation also produced a review-oriented operator experience. The console shows run status, active traces, screenshots, streams, model usage, cached tokens, costs, tool calls, provider results, persistence health, and runtime events. This is important because

evidence collection is not useful if the operator cannot understand what happened after the browser closes. The database is therefore used for observability and review: it stores runs, agent runs, runtime events, LLM calls, tool calls, screenshots, streams, provider analyses, email drafts, prompt compilations, and memory hints.

The company deployment decision remained pragmatic. The complete OWC platform defines the long-term architecture, but the operational deployment focused on the validated n8n landing-agent backup. Target URLs arrive through the `agent_tasks` Azure Service Bus queue, a focused container runs the landing-agent worker, Puppeteer MCP, and Chrome, and the extracted matched URLs with metadata are returned through a configured results queue. This keeps the primary company workflow and the AI browser agent decoupled while still giving the company a path for dynamic pages that require rendered-state reasoning.

The project remains a foundation rather than a finished production enforcement platform. Some pages will still fail because of anti-bot behavior, unstable player code, server-switching variance, misleading channel labels, popup-heavy navigation, or unavailable providers. Human review is still necessary before evidence is used operationally. The evaluation also needs to grow over larger controlled batches before strong accuracy or cost claims can be made. However, the internship achieved its main objective: it turned a fragile investigation problem into a structured artifact with page-role separation, browser evidence, traces, screenshots, stream extraction, channel detection, provider enrichment, deployment direction, and clear limitations.

The future work is therefore concrete. OWC should be evaluated on larger labeled website batches, with stronger regression checks for classification, landing discovery, hosting activation, embedded extraction, channel detection, stream yield, provider coverage, tool success, token usage, and cost per run. Browser memory can be improved so successful selectors and server patterns are reused without hardcoding sites. The operator console can add screenshot annotation for reviewers. Deployment can expand from the landing-agent backup toward more queue-driven specialist agents once the company decides that the operational risk and cost are justified.

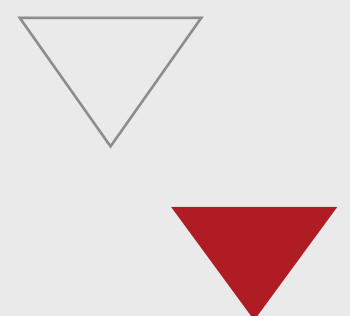
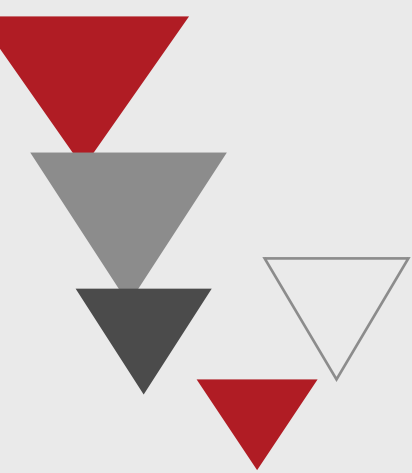
Overall, the internship demonstrated that an AI-assisted browser workflow is a suitable complement to static monitoring when it is constrained, observable, and designed around evidence rather than around unconstrained automation. The value of OWC is not that it permanently solves every illegal streaming website. Its value is that it gives Soft Stars a clear engineering foundation for investigating dynamic streaming pages: classify the page, follow controlled hand-offs, interact with the browser, verify the player state, extract evidence, preserve the trace, and make the result understandable for human review.

References

Bibliography

- [1] Google, *Gemini API Rate Limits*, Google AI for Developers documentation, 2026. Available: <https://ai.google.dev/gemini-api/docs/rate-limits>
- [2] Google, *Gemini Developer API Pricing*, Google AI for Developers documentation, 2026. Available: <https://ai.google.dev/gemini-api/docs/pricing>
- [3] Google, *Context Caching in the Gemini API*, Google AI for Developers documentation, 2026. Available: <https://ai.google.dev/gemini-api/docs/caching>
- [4] Google Chrome Developers, *Puppeteer API Reference*, Official documentation, 2026. Available: <https://pptr.dev>
- [5] Microsoft, *Playwright Documentation*, Official documentation, 2026. Available: <https://playwright.dev/docs/intro>
- [6] Chrome DevTools Protocol, *Chrome DevTools Protocol: Network Domain*, Official documentation, 2026. Available: <https://chromedevtools.github.io/devtools-protocol/>
- [7] A. R. Hevner, S. T. March, J. Park, and S. Ram, *Design Science in Information Systems Research, MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004. Available: <https://www.jstor.org/stable/25148625>
- [8] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, *A Design Science Research Methodology for Information Systems Research, Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007. Available: DOI landing page
- [9] S. Ramírez, *FastAPI: Modern, Fast Web Framework for Building APIs with Python*, 2024. Available: <https://fastapi.tiangolo.com>
- [10] Vercel, *Next.js: The React Framework for the Web*, Vercel, 2024. Available: <https://nextjs.org/docs>
- [11] LangChain, *LangChain Agents and Tools Documentation*, Official documentation, 2026. Available: <https://docs.langchain.com/oss/python/langchain/agents>
- [12] LangChain, *LangGraph Overview*, Official documentation, 2026. Available: <https://docs.langchain.com/oss/python/langgraph/overview>
- [13] Model Context Protocol, *Model Context Protocol Specification*, Official documentation, 2026. Available: Model Context Protocol specification
- [14] n8n GmbH, *n8n: Workflow Automation for Technical People*, 2024. Available: <https://docs.n8n.io>
- [15] Microsoft, *Jobs in Azure Container Apps, Microsoft Learn*, 2024. Available: <https://learn.microsoft.com/en-us/azure/container-apps/jobs>

- [16] Microsoft, *Azure Service Bus Messaging Service*, *Microsoft Learn*, 2024. Available: <https://learn.microsoft.com/en-us/azure/service-bus-messaging/>
- [17] Cloudflare, *How Cloudflare Challenges Work*, Official documentation, 2026. Available: <https://developers.cloudflare.com/cloudflare-challenges/concepts/how-challenges-work/>
- [18] R. Pantos and W. May, *HTTP Live Streaming*, RFC 8216, Internet Engineering Task Force (IETF), August 2017. Available: <https://www.rfc-editor.org/rfc/rfc8216>
- [19] ISO/IEC, *Information technology—Dynamic adaptive streaming over HTTP (DASH)—Part 1: Media presentation description and segment formats*, ISO/IEC 23009–1, 2022.
- [20] R. Hill, *uBlock Origin: An Efficient Blocker for Chromium and Firefox*, GitHub, 2024. Available: <https://github.com/gorhill/uBlock>



ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn – E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax : +216 70 685 454